Foreword

Get a brief look at what we are going to learn in this course!

The Road to learn React teaches you the fundamentals of React. You will build a real-world application along the way in plain React without complicated tooling. Everything from project setup to deployment on a server will be explained for you. The course comes with additional referenced reading material and exercises with each chapter. After taking the course, you will be able to build your own applications in React. The material is kept up to date by me and the community.

In the Road to learn React, I want to offer a foundation before you start to dive into the broader React ecosystem. It has less tooling and less external state management, but a lot of information about React. It explains general concepts, patterns and best practices in a real-world React application.

You will learn to build your own React application. It covers real-world features like pagination, client-side caching and interactions such as searching and sorting. Additionally, you will transition from JavaScript ES5 to JavaScript ES6 along the way. I hope this course captures my enthusiasm for React and JavaScript and helps you to get started with it.

```
import React, { Component } from 'react';
import { sortBy } from 'lodash';
import classNames from 'classnames';

require('./App.css');

const DEFAULT_QUERY = 'redux';
const DEFAULT_PAGE = 0;
const DEFAULT_HPP = '3';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';
const PARAM_HPP = 'hitsPerPage=';

const SORTS = {
```

```
  NONE: list => list,
  TITLE: list => sortBy(list, 'title'),
  AUTHOR: list => sortBy(list, 'author'),

  COMMENTS: list => sortBy(list, 'num_comments').reverse(),
  POINTS: list => sortBy(list, 'points').reverse(),
};

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      results: null,
      searchKey: '',
      searchTerm: DEFAULT_QUERY,
      isLoading: false,
    };

    this.needsToSearchTopstories = this.needsToSearchTopstories.bind(this);
    this.setSearchTopstories = this.setSearchTopstories.bind(this);
    this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  needsToSearchTopstories(searchTerm) {
    return !this.state.results[searchTerm];
  }

  setSearchTopstories(result) {
    const { hits, page } = result;
    const { searchKey, results } = this.state;

    const oldHits = results && results[searchKey]
      ? results[searchKey].hits
      : [];

    const updatedHits = [
      ...oldHits,
      ...hits
    ];

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      },
      isLoading: false
    });
  }

  fetchSearchTopstories(searchTerm, page) {
    this.setState({ isLoading: true });

    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}&${PAR
      .then(response => response.json())
      .then(result => this.setSearchTopstories(result));
  }

  componentDidMount() {
```

```
    const { searchTerm } = this.state;
    this.setState({ searchKey: searchTerm });
    this.fetchSearchTopstories(searchTerm, DEFAULT_PAGE);
  }

  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  onSearchSubmit(event) {
    const { searchTerm } = this.state;
    this.setState({ searchKey: searchTerm });

    if (this.needsToSearchTopstories(searchTerm)) {
      this.fetchSearchTopstories(searchTerm, DEFAULT_PAGE);
    }

    event.preventDefault();
  }

  onDismiss(id) {
    const { searchKey, results } = this.state;
    const { hits, page } = results[searchKey];

    const isNotId = item => item.objectID !== id;
    const updatedHits = hits.filter(isNotId);

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      }
    });
  }

  render() {
    const {
      searchTerm,
      results,
      searchKey,
      isLoading
    } = this.state;

    const page = (
      results &&
      results[searchKey] &&
      results[searchKey].page
    ) || 0;

    const list = (
      results &&
      results[searchKey] &&
      results[searchKey].hits
    ) || [];

    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
```

```
          >
            Search
          </Search>
        </div>
        <Table
          list={list}
          onDismiss={this.onDismiss}
        />
        <div className="interactions">
          <ButtonWithLoading
            isLoading={isLoading}
            onClick={() => this.fetchSearchTopstories(searchKey, page + 1)}>
            More
          </ButtonWithLoading>
        </div>
      </div>
    );
  }
}

const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
    />
    <button type="submit">
      {children}
    </button>
  </form>

class Table extends Component {

  constructor(props) {
    super(props);

    this.state = {
      sortKey: 'NONE',
      isSortReverse: false,
    };

    this.onSort = this.onSort.bind(this);
  }

  onSort(sortKey) {
    const isSortReverse = this.state.sortKey === sortKey && !this.state.isSortReverse;
    this.setState({ sortKey, isSortReverse });
  }

  render() {
    const {
      list,
      onDismiss
    } = this.props;

    const {
```

```
    sortKey,
    isSortReverse,
  } = this.state;

  const sortedList = SORTS[sortKey](list);
  const reverseSortedList = isSortReverse
    ? sortedList.reverse()
    : sortedList;

  return(
    <div className="table">
      <div className="table-header">
        <span style={{ width: '40%' }}>
          <Sort
            sortKey={'TITLE'}
            onSort={this.onSort}
            activeSortKey={sortKey}
          >
            Title
          </Sort>
        </span>
        <span style={{ width: '30%' }}>
          <Sort
            sortKey={'AUTHOR'}
            onSort={this.onSort}
            activeSortKey={sortKey}
          >
            Author
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'COMMENTS'}
            onSort={this.onSort}
            activeSortKey={sortKey}
          >
            Comments
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'POINTS'}
            onSort={this.onSort}
            activeSortKey={sortKey}
          >
            Points
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          Archive
        </span>
      </div>
      { reverseSortedList.map(item =>
        <div key={item.objectID} className="table-row">
          <span style={{ width: '40%' }}>
            <a href={item.url}>{item.title}</a>
          </span>
          <span style={{ width: '30%' }}>
            {item.author}
          </span>
          <span style={{ width: '10%' }}>
            {item.num_comments}
```

```
            </span>
            <span style={{ width: '10%' }}>
              {item.points}
            </span>
            <span style={{ width: '10%' }}>
              <Button
                onClick={() => onDismiss(item.objectID)}
                className="button-inline"
              >
                Dismiss
              </Button>
            </span>
          </div>
        )}
      </div>
    );
  }
}

const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

const Loading = () =>
  <div>Loading ...</div>

const withLoading = (Component) => ({ isLoading, ...rest }) =>
  isLoading ? <Loading /> : <Component { ...rest } />

const ButtonWithLoading = withLoading(Button);

const Sort = ({
  sortKey,
  activeSortKey,
  onSort,
  children
}) => {
  const sortClass = classNames(
    'button-inline',
    { 'button-active': sortKey === activeSortKey }
  );

  return (
    <Button
      onClick={() => onSort(sortKey)}
      className={sortClass}
    >
      {children}
    </Button>
  );
}

export default App;

export {
  Button,
  Search,
```

```
  Table,
};
```

Testimonials

See what the customers have to say about this course!

There are many testimonials, ratings and reviews about this course (originally: book) which should confirm the quality of it. I am so proud of it because I never expected such an overwhelming feedback. If you enjoy this course as well, I would love to find your rating/review somewhere as well. It helps me to spread the word about it. The following shows a short excerpt of these good voices:

Muhammad Kashif: "The Road to Learn React is a unique book that I recommend to any student or professional interested in learning react basics to advanced level. It is packed with insightful tips and techniques that are hard to find elsewhere, and remarkably thorough in its use of examples and references to sample problems, I have 17 years of experience in web and desktop app development, and before reading this book I was having trouble in learning react, but this book works like magic."

Andre Vargas: "The Road to Learn React by Robin Wieruch is such an awesome book! Most of what I learned about React and even ES6 was through it!"

Nicholas Hunt-Walker, Instructor of Python at a Seattle Coding School: "This is one of the most well-written & informative coding books I've ever worked through. A solid React & ES6 introduction."

Austin Green: "Thanks, really loved the book. A perfect blend to learn React, ES6, and higher level programming concepts."

Nicole Ferguson: "I'm doing Robin's Road to Learn React course this weekend & I almost feel guilty for having so much fun."

Karan: "Just finished your Road to React. Best book for a beginner in the world of React and JS. Elegant exposure to ES. Kudos! :)"

Eric Priou: “The Road to learn React by Robin Wieruch is a must read. Clean and concise for React and JavaScript.”

A Rookie Developer: “I just finished the book as a rookie developer, thanks for working on this. It was easy to follow and I feel confident in starting a new app from scratch in the coming days. The book was much better than the official React.js tutorial that I tried earlier (and couldn’t complete due to lack of detail). The exercises at the end of each section were very rewarding.”

Student: “The best book to start learning ReactJS. The project moves along with the concepts being learned which helps to grasp the subject. I have found ‘Code and learn’ as the best way to master programming and this book exactly does that.”

Thomas Lockney: “Pretty solid introduction to React that doesn’t try to be comprehensive. I just wanted a taste to understand what it was about and this book gave me exactly that. I didn’t follow all the little footnotes to learn about the new ES6 features I’ve missed (“I wouldn’t say I’ve been missing it, Bob.”). But I’m sure for those of you who have fallen behind and are diligent about following those, you can probably learn a lot more than just what the book teaches.”

FAQ

Got any questions? This section might help you get the answer.

How do I get updates? You can subscribe to the Newsletter or follow me on Twitter for updates. Once you have a copy of the course, it will stay updated when a new edition gets released. But you have to grab the copy again when an update is announced.

Does it use the recent React version? The course always receives an update when the React version got updated. Usually, courses are outdated pretty soon after their release.

Does it cover Redux? It doesn't. Therefore I have created a second course. The Road to learn React should give you a solid foundation before you dive into advanced topics. The implementation of the sample application in the course will show that you don't need Redux to build an application in React. After you took the course, you should be able to implement a solid application without Redux. Then you can take my second course to learn Redux.

Does it use JavaScript ES6? Yes. But don't worry. You will be fine if you are familiar with JavaScript ES5. All JavaScript ES6 features, that I describe on the journey to learn React, will transition from ES5 to ES6 in the course. Every feature along the way will be explained. The course does not only teach React, but also all useful JavaScript ES6 features for React.

How can I get help while taking the course? The course has a Slack Group for people who are taking the course. You can join the channel to get help or to help others. After all, helping others can improve your learning too.

Is there any troubleshoot area? If you run into problems, please join the Slack Group. In addition, you could have a look into the open issues on GitHub for the course. Perhaps your problem was already mentioned and you can find the solution for it. If your problem wasn't mentioned, don't hesitate to open a new issue where you can explain your problem, maybe provide a

screenshot, and some more details (e.g. lesson in the course, node version). After all, I try to ship all fixes in next editions of the course.

How to do it?

This section provides an insight to what you're going to learn in this course and how you will be progressing with each chapter.

The course is my attempt to teach React by helping you learn how to create your own application. It is a practical guide to learn React and not a reference work about React. You will learn how to create a Hacker News application that interacts with a real-world API. Among several interesting topics, it covers state management in React, caching and interactions (sorting and searching). On the way you will learn best practices and patterns in React.

In addition, the course gives you a transition from JavaScript ES5 to JavaScript ES6. React embraces a lot of JavaScript ES6 features and I want to show you how you can use them.

In general, each lesson of the course will build upon the previous lesson. Each lesson will teach you something new. Don't rush through the course. You should internalize each step. You could apply your own implementations and read more about the topic. After each lesson, I give you some reading material and exercises. If you really want to learn React, I highly recommend to read the extra material and do some hands-on exercises. After you have done a lesson, make yourself comfortable with the learnings before you continue.

In the end, you will have a complete React application in production. I am very keen to see your results, so please text me when you have finished the course. The final lesson of the course will give you a handful of options to continue your React journey. In general, you will find a lot of React related topics on my personal website.

Since you are taking the course, I guess you are new to React. That's perfect. In the end, I hope to get your feedback to improve the material to enable everyone to learn React. You can have a direct impact on GitHub or text me on Twitter.

Hi, my name is React

The lesson gives you an introduction to React. You may ask yourself: Why should I learn React in the first place? The lesson might give you the answer to that question.

This chapter is an introduction to React, a JavaScript library for rendering interfaces in single-page and mobile applications, where I explain why developers should consider adding the React library to their toolbelts. We will dive into the React ecosystem, creating your first React application from scratch with no configuration. Along the way, we will introduce **JSX**, the syntax for React, and **ReactDOM**, so you have an understanding of React's practical uses in modern web applications.

Hi, my name is React.

Single page applications (SPA) have become increasingly popular in recent years, as frameworks like Angular, Ember, and Backbone allow JavaScript developers to build modern web applications using techniques beyond vanilla JavaScript and jQuery. The three mentioned are among the first SPAs, each coming into its own between 2010 and 2011, but there are many more options for single-page development. The first generation of SPA frameworks arrived at the enterprise level, so their frameworks are more rigid. React, on the other hand, remains an innovative library that has been adopted by many technological leaders like Airbnb, Netflix, and Facebook.

React was released by Facebook's web development team in 2013 as a view library, which makes it the 'V' in the MVC (model view controller). As a view, it allows you to render components as viewable elements in a browser, while its ecosystem lets us build single page applications. While the first generation of frameworks tried to solve many things at once, React is only used to build your view layer; specifically, it is a library wherein the view is a hierarchy of composable components. If you haven't heard about MVC before, don't bother about it, because it's just there to put React historically into context for people

about it, because it's just there to put React historically into context for people who come from other programming languages.

In React, the focus remains on the view layer until more aspects are introduced to the application. These are the building blocks for an SPA, which are essential to build a mature application. They come with two advantages:

- You can learn the building blocks one at a time without having to understand them altogether. In contrast, an SPA framework gives you every building block from the start. This course focuses on React as the first building block. More building blocks will eventually follow.
- All building blocks are interchangeable, which makes the ecosystem around React highly innovative. Multiple solutions can compete with each other, and you can choose the most appealing solution for any given challenge.

React is one of the best choices for building modern web applications. Again, it only delivers the view layer, but the surrounding ecosystem makes up an entirely flexible and interchangeable framework. React has a slim API, a robust and evolving ecosystem, and a great community.

Further Readings

If you'd like to know more about why I chose React, or to find a more in-depth look at the topics mentioned above, these articles grant a deeper perspective:

- Why I moved from Angular to React
- React's flexible ecosystem
- How to learn a framework

ES6 const and let

This lesson explains the difference between var and const and let declarations.

Notice that we declared the variable `helloWorld` with a `var` statement. JavaScript ES6 comes with two more ways to declare variables: `const` and `let`. In JavaScript ES6, you will rarely find `var` anymore.

const

A variable declared with `const` cannot be re-assigned or re-declared, and cannot be changed or modified. Once the variable is assigned, you cannot change it:

```
// not allowed
const helloWorld = 'Welcome to the Road to learn React';
helloWorld = 'Bye Bye React';
```

let

Conversely, a variable declared with `let` can be modified:

```
// allowed
let helloWorld = 'Welcome to the Road to learn React';
helloWorld = 'Bye Bye React';
```

TIP: Declare variables with `let` if you think you'll want to re-assign it later on.

Note that a variable declared directly with `const` cannot be modified. However, when the variable is an array or object, the values it holds can get updated through indirect means:

```
// allowed
const helloWorld = {
  text: 'Welcome to the Road to learn React'
};
```

```
helloWorld.text = 'Bye Bye React';
```

There are varying opinions about when to use *const* and when to use *let*. I would recommend using `const` whenever possible to show the intent of keeping your data structures immutable, so you only have to worry about the values contained in objects and arrays. Immutability is embraced in the React ecosystem, so `const` should be your default choice when you define a variable, though it's not really about immutability, but about assigning variables only once. It shows the intent of not changing (re-assigning) the variable even though its content can be changed.

```
import React, { Component } from 'react';
import './App.css';

class App extends Component {
  render() {
    const helloWorld = 'Welcome to the Road to learn React';
    return (
      <div className="App">
        <h2>{helloWorld}</h2>
      </div>
    );
  }
}

export default App;
```

In this course, we will use `const` and `let` over `var` for the rest of the course.

1 The most recommended way of declaring a variable is using:

2

You can add new properties to an instance object any time later after its creation. True or False?

Check Answers

Exercises:

- Gain an understanding of immutable data structures:
  - Why do they make sense in programming?
  - Why are they embraced in React and its ecosystem?

Further Readings:

- Read about ES6 const
- Read about ES6 let

ES6 Arrow Functions

An introduction to Arrow Functions and their importance.

JavaScript ES6 introduced arrow functions expressions, which are shorter than a function expressions.

```
// function declaration
function () { ... }

// arrow function declaration
() => { ... }
```

You can remove the parentheses in an arrow function expression if it only has one argument, but you have to keep the parentheses if it gets multiple arguments:

```
// allowed
item => { ... }

// allowed
(item) => { ... }

// not allowed
item, key => { ... }

// allowed
(item, key) => { ... }
```

You can also write `map` functions more concisely with an ES6 arrow function:

```
{list.map(item => {
  return (
    <div key={item.objectID}>
      <span>
        <a href={item.url}>{item.title}</a>
      </span>
      <span>{item.author}</span>
      <span>{item.num_comments}</span>
      <span>{item.points}</span>
    </div>
  );
```

```
})}
```

You can remove the *block body*, the curly braces, with the ES6 arrow function. In a *concise body*, an implicit return is attached; thus, you can remove the `return` statement. This will happen often in this course, so be sure to understand the difference between a block body and a concise body when using arrow functions.

```
{list.map(item =>
  <div key={item.objectID}>
    <span>
      <a href={item.url}>{item.title}</a>
    </span>
    <span>{item.author}</span>
    <span>{item.num_comments}</span>
    <span>{item.points}</span>
  </div>
)}
```

Q

What would the following code do?

```
render(){
   let users = [
  { 'name' : 'Elton Jhon', 'age' : 60 },
  { 'name' : 'Elvis Persley', 'age' : 50 },
  { 'name' : 'Kurt Cobain', 'age' : 24}
];
   return (<div>
     {users.map(user => <p>user.name</p>)}
   </div>)
}
```

Your JSX should look more concise and readable now, as it omits the `function` statement, the curly braces, and the return statement.

Further Reading:

- Read about ES6 arrow functions

ES6 Classes

This lesson explains JavaScript classes and how to define them in your application.

JavaScript ES6 introduced classes, which are commonly used in object-oriented programming languages. JavaScript, always flexible in its programming paradigms, allows functional programming and object-oriented programming to work side-by-side.

While React embraces functional programming, e.g. immutable data structures and function compositions, classes are used to declare ES6 class components. React mixes the good parts of both programming paradigms.

Consider the following Developer class to examine a JavaScript ES6 class without a component.

```
class Developer {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }

  getName() {
    return this.firstname + ' ' + this.lastname;
  }
}
```

A class has a constructor to make it instantiable. The constructor takes arguments and assigns them to the class instance. A class can also define functions. Because the function is associated with a class, it is called a method, or a class method.

The Developer class is only the class declaration we use here, as you can create multiple instances of a class by invoking it. It is similar to the ES6 class component, which has a declaration, but you have to use it somewhere else to instantiate it:

```
class Developer {
  constructor(firstname, lastname) {
    this.firstname = firstname;

    this.lastname = lastname;
  }

  getName() {
    return this.firstname + ' ' + this.lastname;
  }
}


const robin = new Developer('Robin', 'Wieruch');
console.log(robin.getName());
// output: Robin Wieruch
```

React uses JavaScript ES6 classes for ES6 class components, which you have already used at least once so far:

```
import React, { Component } from 'react';

...

class App extends Component {
  render() {
    ...
  }
}
```

`extends`

When you declare the App component it extends from another component.

In object-oriented programming, the term “extends” refers to the principle of inheritance, which means that functionality can be passed from one class to another. The App class extends from the Component class, meaning it inherits functionality from the Component class. The Component class is used to extend a basic ES6 class to an ES6 component class. It has all the functionalities that a component in React needs. The render method is one function you have already used. You will learn about other component class methods as we move along.

The `Component` class encapsulates all the implementation details of a React component, which allows developers to use classes as components in React.

Methods exposed by a React `Component` are its public interface. One of these methods must be overridden, while the others don't need to be overridden.

You will learn about these when we discuss lifecycle methods later. The `render()` method has to be overridden because it defines the output of a React `Component`, so it must be defined. These are the basics of JavaScript ES6 classes, and how they are used in React to extend them to components.

Further Readings:

- Read about JavaScript fundamentals before learning React
- Read about ES6 classes

Complex JavaScript in JSX

Let's see how to render lists and objects in your JSX!

So far, you have rendered a few primitive variables in your JSX. Now, we will render a list of items. The list will contain sample data in the beginning, but later we will learn how to fetch the data from an external API.

First, you have to define the list of items:

```
import React, { Component } from 'react';
import './App.css';

const list = [
  {
    title: 'React',
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  {
    title: 'Redux',
    url: 'https://redux.js.org/',
    author: 'Dan Abramov, Andrew Clark',
    num_comments: 2,
    points: 5,
    objectID: 1,
  },
];

class App extends Component {
  ...
}
```

The sample data represents information we will fetch from an API later on. Items in this list each have a title, a URL, and an author, as well an identifier, points (which indicate how popular an article is), and a count of comments.

Now you can use the built-in JavaScript map functionality in JSX, which iterates over a list of items to display them according to specific attributes. Again, we use curly braces to encapsulate the JavaScript expression in JSX:

```
import React, { Component } from 'react';
require('./App.css');

const list = [
  {
    title: 'React',
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  {
    title: 'Redux',
    url: 'https://redux.js.org/',
    author: 'Dan Abramov, Andrew Clark',
    num_comments: 2,
    points: 5,
    objectID: 1,
  },
];

class App extends Component {

  render() {
    return (
      <div className="App">
        {list.map(function(item) {
          return <div>{item.title}</div>;
        })}
      </div>
    );
  }
}

export default App;
```

Using JavaScript alongside HTML in JSX is very powerful. For a different task you may have used `map` to convert one list of items to another. This time, we used `map` to convert a list of items to HTML elements.

```
const array = [1, 4, 9, 16];

// pass a function to map
const newArray = array.map(function (x) { return x * 2; });

console.log(newArray);
// expected output: Array [2, 8, 18, 32]
```

So far, only the `title` is displayed for each item. Let's experiment with more of the item's properties:

```
import React, { Component } from 'react';
require('./App.css');

const list = [
  {
    title: 'React',
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  {
    title: 'Redux',
    url: 'https://redux.js.org/',
    author: 'Dan Abramov, Andrew Clark',
    num_comments: 2,
    points: 5,
    objectID: 1,
  },
];

class App extends Component {
  render() {
    return (
      <div className="App">
        {list.map(function(item) {
          return (
            <div>
              <span>
                <a href={item.url}>{item.title}</a>
              </span>
              <span>{item.author}</span>
              <span>{item.num_comments}</span>
              <span>{item.points}</span>
            </div>
          );
        })}
      </div>
    );
  }
}

export default App;
```

Note how the `map` function is inlined in your JSX. Each item property is displayed with a `<span>` tag, and the url property of the item is in the `href` attribute of the anchor tag.

React will display each item, but you can still do more to help React embrace its full potential. By assigning a key attribute to each list element, React can identify modified items when the list changes. These sample list items come with an identifier:

with an identifier.

```
{list.map(function(item) {
  return (
    <div key={item.objectID}>
      <span>
        <a href={item.url}>{item.title}</a>
      </span>
      <span>{item.author}</span>
      <span>{item.num_comments}</span>
      <span>{item.points}</span>
    </div>
  );
})}
```

Make sure that the key attribute is a stable identifier. Avoid using the index of the item in the array, because the array index is not stable. If the list changes its order, for example, React will not be able to identify the items properly.

```
// don't do this
{list.map(function(item, key) {
  return (
    <div key={key}>
      ...
    </div>
  );
})}
```

```
import React, { Component } from 'react';
require('./App.css');

const list = [
  {
    title: 'React',
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  {
    title: 'Redux',
    url: 'https://redux.js.org/',
    author: 'Dan Abramov, Andrew Clark',
    num_comments: 2,
    points: 5,
    objectID: 1,
  },
];

class App extends Component {
  render() {
    return (
      <div className="App">
```

```
        {list.map(function(item) {
          return (
                                <div key={item.objectID}>

              <span>
                <a href={item.url}>{item.title}</a>
              </span>
              <span>{item.author}</span>
              <span>{item.num_comments}</span>
              <span>{item.points}</span>
            </div>
          );
        })}
      </div>
    );
  }
}

export default App;
```

Now you should see both items of the list displayed.

Exercises:

- Use more JavaScript expressions on your own in JSX

Further Readings:

- Read about React lists and keys
- Recap the standard built-in array functionalities in JavaScript

An Introduction to the create-react-app

This lesson gives a brief overview of the create-react-app.

What is the create-react-app?

The Create React App is a tool built by developers at Facebook to help you build React applications. It does not require any time-consuming setup and configuration. You simply run one command and the create react app sets up the tools you need to start your React project. The rest of the course will loosely follow the file structure followed in the create-react-app. You can learn how to set it up in our zero configuration lesson.

A high-level explanation of the create-react-app

You can see the create-react-app below. I won't explain the specifics of it yet, but you should be familiar with the directory structure below. Essentially, everything starts with the `index.js` file. It imports react, the react DOM (don't worry if you don't know what that is yet), the app's code itself which is in `App.js` and `index.css` for styling. It then 'renders' (prints/displays) the app on lines 7-10. Notice the tag `<App />` on line 8. This line actually renders the app. The app is defined as a class or 'component' (these will be explained in depth later) in the `App.js` file.

```
.App {
  text-align: center;
}

.App-logo {
  animation: App-logo-spin infinite 20s linear;
  height: 80px;
}

.App-header {
  background-color: #222;
  height: 150px;
  padding: 20px;
  color: white;
```

```
}

.App-title {
  font-size: 1.5em;
}

.App-intro {
  font-size: large;
}

@keyframes App-logo-spin {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}
```

Local Component State

This section will guide you through the basics of React. It covers state and interactions in components as we move past static components. We will also cover the different ways to declare a component, and how to keep components composable and reusable.

Local component state, also known as the internal component state, allows you to save, modify, and delete properties stored in your component. The ES6 class component then uses a constructor to initialize local component state. The constructor is called only once when the component initializes:

Let's introduce a class constructor.

```
class App extends Component {

  constructor(props) {
    super(props);
  }
  ...

}
```

The App component is a subclass of `Component`, so the `extends Component` is in the App component declaration.

It is mandatory to call `super(props);`. It sets `this.props` in your constructor in case you want to access them there. They would be `undefined` when accessing `this.props` in your constructor otherwise. In this case, the initial state of the component should be the sample list of items:

```
const list = [
  {
    title: 'React',
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  ...
];
```

```
class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      list: list,
    };
  }

  ...

}
```

The state is bound to the class using the `this` object, so you can access the local state of the whole component. For instance, it can be used in the `render()` method. Previously, you have mapped a static list of items in your `render()` method that was defined outside of your component. Now you are about to use the list from your local state in your component.

```
import React, { Component } from 'react';
require('./App.css');

const list = [
  {
    title: 'React',
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  {
    title: 'Redux',
    url: 'https://redux.js.org/',
    author: 'Dan Abramov, Andrew Clark',
    num_comments: 2,
    points: 5,
    objectID: 1,
  },
];

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      list: list,
    };

  }

  render() {
    return (
```

```
      <div className="App">
        { this.state.list.map(item =>
          <div key={item.objectID}>

            <span>
              <a href={item.url}>{item.title}</a>
            </span>
            <span>{item.author}</span>
            <span>{item.num_comments}</span>
            <span>{item.points}</span>
          </div>
        )}
      </div>
    );
  }
}

export default App;
```

The list is part of the component now, in the local component state. You could add, change, or remove items from your list. Every time you change your component state, the `render()` method of your component will run again. That's how you can change your local component state and see the component re-render the correct data from the local state.

Be careful not to mutate the state directly. Instead, you should use a method called `setState()` to modify your states. We will cover these concepts in more depth next chapter.

Exercises:

- Experiment with the local state
  - Define more initial state in the constructor
  - Use and access the state in your `render()` method

Further Readings:

- Read about the ES6 class constructor

ES6 Object Initializer

This lesson teaches how to use object initializer and how to initialize methods within them.

In JavaScript ES6, you can use a shorthand property syntax to initialize your objects more concisely, like following object initialization:

```
const name = 'Robin';

const user = {
  name: name,
};

console.log('ES5', user);
```

When the property name in your object is the same as your variable name, you can do the following:

```
const name = 'Robin';

const user = {
  name,
};

console.log('ES6', user);
```

We can do the same thing in the application that we are building in this course. The list variable name and the state property name share the same name.

```
// ES5
this.state = {
  list: list,
};
```

```
// ES6
this.state = {

  list,
};
```

Shorthand method names are also useful. In JavaScript ES6, you can initialize methods in an object more concisely:

```
// comment either ES5 or ES6 code

// ES5
var userService = {
  getUserName: function (user) {
    return user.firstname + ' ' + user.lastname;
  },
};

// ES6
// var userService = {
//   getUserName(user) {
//     return user.firstname + ' ' + user.lastname;
//   },
// };

var user = {
  firstname: 'Robin',
  lastname: 'Wieruch',
};

console.log(userService.getUserName(user));
```

Finally, you are allowed to use computed property names in JavaScript ES6:

```
// comment either ES5 or ES6 code

// ES5
var user = {
  name: 'Robin',
};

// ES6
// var key = 'name';
// var user = {
//   [key]: 'Robin',
// };

console.log(user.name);
```

Later, you will be able to use computed property names to allocate values by key in an object dynamically, a handy way to generate lookup tables in JavaScript.

Exercises:

- Experiment with ES6 object initializer

Further Readings:

- Read about ES6 object initializer

Unidirectional Data Flow

Learn how to manipulate a static state and its component.

Now you have a local state in your App component, but haven't manipulated the state yet. The local state is static, so its component is as well. A good way to experience state manipulation is to engage in component interaction.

We will practice this concept by adding a button for each item in the displayed list. The button will read "Dismiss", as its purpose will be to remove an item from the list. In an email client, for example, it would be a useful way to mark some list items as 'read' while keeping the unread items separate.

```
class App extends Component {

  ...

  render() {
    return (
      <div className="App">
        {this.state.list.map(item =>
          <div key={item.objectID}>
            <span>
              <a href={item.url}>{item.title}</a>
            </span>
            <span>{item.author}</span>
            <span>{item.num_comments}</span>
            <span>{item.points}</span>
            <span>
              <button
                onClick={() => this.onDismiss(item.objectID)}
                type="button"
              >
                Dismiss
              </button>
            </span>
          </div>
        )}
      </div>
    );
  }
}
```

The `onDismiss()` class method is not defined yet. We will do it in a moment,

but for now, let's focus on the `onClick` handler of the button element. As you can see, the `onDismiss()` method in the `onClick` handler is enclosed by an arrow function. You can use it to peek at the `objectID` property of the `item` object and identify the item to be dismissed. An alternative way would be to define the function outside of the `onClick` handler and only pass the defined function to it. We will cover handlers more in-depth as we move along.

Note the use of multilines for the button element, and how elements with multiple attributes can get disorganized easily. The button element is used with multilines and indentations to keep it readable. While this practice isn't specific to React development, it is a programming style I recommend for cleanliness and your own peace of mind.

Now we will implement the `onDismiss()` functionality. It takes an id to identify the item to dismiss. The function is bound to the class and thus becomes a class method. That's why you access it with `this.onDismiss()` and not `onDismiss()`. The `this` object is your class instance. In order to define the `onDismiss()` as class method, you have to bind it in the constructor:

```
class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      list,
    };

    this.onDismiss = this.onDismiss.bind(this);
  }

  onDismiss(id) {
    ...
  }

  render() {
    ...
  }
}
```

In the next step, we define its functionality, the business logic, in the class:

```
onDismiss(id) {
  const updatedList = this.state.list.filter(function isNotId(item) {
    return item.objectID !== id;
  });
}
```

Now we can define what happens inside a class method. Remember, the objective is to remove the item identified by the id from the list and store an updated list to the local state. The updated list will be used in the re-running `render()` method to display it, where the removed item should no longer appear.

You can remove an item from a list using JavaScript's built-in filter functionality, which takes a function as input. The function has access to each value in the list because it iterates over each item, so you can evaluate them based on certain conditions.

```
const words = ['spray', 'limit', 'elite', 'exuberant', 'destruction', 'present'];

const filteredWords = words.filter(function (word) { return word.length > 6; });

console.log(filteredWords);
// expected output: Array ["exuberant", "destruction", "present"]
```

The function returns a new list instead of mutating the old one, and it supports the React convention of using immutable data structures.

```
onDismiss(id) {
  const updatedList = this.state.list.filter(function isNotId(item) {
    return item.objectID !== id;
  });
}
```

In the next step, we extract the function and pass it to the filter function:

```
onDismiss(id) {
  function isNotId(item) {
    return item.objectID !== id;
  }

  const updatedList = this.state.list.filter(isNotId);
}
```

Remember: You can filter more efficiently using a JavaScript ES6 arrow function.

```
onDismiss(id) {
  const isNotId = item => item.objectID !== id;
  const updatedList = this.state.list.filter(isNotId);
}
```

You could even inline it again like you did in the `onClick` handler of the button, though it might get less readable:

```
onDismiss(id) {
  const updatedList = this.state.list.filter(item => item.objectID !== id);
}
```

The list removes the clicked item now, but the state hasn't updated yet. Use the `setState()` class method to update the list in the local component state:

```
onDismiss(id) {
  const isNotId = item => item.objectID !== id;
  const updatedList = this.state.list.filter(isNotId);
  this.setState({ list: updatedList });
}
```

Run the code again and try the "Dismiss" button. What you experienced is the **unidirectional data flow** of React. An action is triggered in the view layer with `onClick()`, a function or class method modifies the local component state, and then the `render()` method of the component runs again to update the view.

```
import React, { Component } from 'react';
require('./App.css');

const list = [
  {
    title: 'React',
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  {
    title: 'Redux',
    url: 'https://redux.js.org/',
    author: 'Dan Abramov, Andrew Clark',
    num_comments: 2,
    points: 5,
```

```
    objectID: 1,
  },
];


class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      list,
    };

    this.onDismiss = this.onDismiss.bind(this);
  }

  onDismiss(id) {
    const isNotId = item => item.objectID !== id;
    const updatedList = this.state.list.filter(isNotId);
    this.setState({ list: updatedList });
  }

  render() {
    return (
      <div className="App">
        { this.state.list.map(item =>
          <div key={item.objectID}>
            <span>
              <a href={item.url}>{item.title}</a>
            </span>
            <span>{item.author}</span>
            <span>{item.num_comments}</span>
            <span>{item.points}</span>
            <span>
              <button
                onClick={() => this.onDismiss(item.objectID)}
                type="button"
              >
                Dismiss
              </button>
            </span>
          </div>
        )}
      </div>
    );
  }
}

export default App;
```

Further Reading:

- Read about the state and lifecycle in React

Bindings

Binding comes in very handy while creating React apps. You'll see how by as you go read this lesson.

It is important to learn about bindings in JavaScript classes when using React ES6 class components. In the previous chapter, you have bound your class method `onDismiss()` in the constructor.

```
class App extends Component {
  constructor(props) {
    super(props);

    this.state = {
      list,
    };

    this.onDismiss = this.onDismiss.bind(this);
  }

  ...
}
```

The binding step is necessary because class methods don't automatically bind `this` to the class instance. Let's demonstrate it with the help of the following ES6 class component:

```
class ExplainBindingsComponent extends Component {
  onClickMe() {
    console.log(this);
  }

  render() {
    return (
      <button
        onClick={this.onClickMe}
        type="button"
      >
        Click Me
      </button>
    );
  }
}
```

The component renders just fine, but when you click the button, you see `undefined` in your developer console log. This is one of the main sources of bugs developers encounter in React. If you want to access `this.state` in your class method, it cannot be retrieved because `this` is `undefined`. To make `this` accessible in your class methods, you have to bind the class methods to `this`.

In the following class component the class method is properly bound in the class constructor:

```
class ExplainBindingsComponent extends Component {
  constructor() {
    super();

    this.onClickMe = this.onClickMe.bind(this);
  }

  onClickMe() {
    console.log(this);
  }

  render() {
    return (
      <button
        onClick={this.onClickMe}
        type="button"
      >
        Click Me
      </button>
    );
  }
}
```

Class method binding can happen somewhere else too. For instance, it can happen in the `render()` class method:

```
class ExplainBindingsComponent extends Component {
  onClickMe() {
    console.log(this);
  }

  render() {
    return (
      <button
        onClick={this.onClickMe.bind(this)}
        type="button"
      >
        Click Me
      </button>
    );
  }
}
```

Avoid this practice, however, because it binds the class method every time the `render()` method runs, meaning every time the component updates, which will hurt your application's performance eventually. Binding the class method in the constructor need only be done once, when the component is instantiated.

Some developers will define the business logic of their class methods in the constructor:

```
class ExplainBindingsComponent extends Component {
  constructor() {
    super();

    this.onClickMe = () => {
      console.log(this);
    }
  }

  render() {
    return (
      <button
        onClick={this.onClickMe}
        type="button"
      >
        Click Me
      </button>
    );
  }
}
```

Avoid this approach as well, as it will clutter your constructor over time. The constructor is only there to instantiate your class with all its properties, so the business logic of class methods should be defined outside the constructor.

```
class ExplainBindingsComponent extends Component {
  constructor() {
    super();

    this.doSomething = this.doSomething.bind(this);
    this.doSomethingElse = this.doSomethingElse.bind(this);
  }

  doSomething() {
    // do something
  }

  doSomethingElse() {
    // do something else
  }
```

```
  }

  ...
}
```

Class methods can be auto-bound using JavaScript ES6 arrow functions:

```
class ExplainBindingsComponent extends Component {
  onClickMe = () => {
    console.log(this);
  }

  render() {
    return (
      <button
        onClick={this.onClickMe}
        type="button"
      >
        Click Me
      </button>
    );
  }
}
```

Use this method if the repetitive binding in the constructor annoys you. The official React documentation sticks to the class method bindings in the constructor, so this course will stick with those as well.

Exercises:

- Try different approaches of bindings and console log the `this` object

Further Reading:

- Learn more about an alternative React component syntax

Event Handler

Learn how to define event handlers in a React app. You will also get to learn the importance of using Arrow function inside event handlers.

Now we'll cover event handlers in elements. In the application that you are building, you are using the following button element to dismiss an item from the list.

```
...

<button
  onClick={() => this.onDismiss(item.objectID)}
  type="button"
>
  Dismiss
</button>

...
```

This function is already complex because it passes a value to the class method and has to wrap it in another (arrow) function. Essentially, it has to be a function that is passed to the event handler. The code given below wouldn't work if you are following along on a local React setup, because the class method would be executed immediately when you open the application in the browser:

```
...

<button
  onClick={this.onDismiss(item.objectID)}
  type="button"
>
  Dismiss
</button>

...
```

`onClick`

When using `onClick={doSomething()}`, the `doSomething()` function executes

When using `onClick={doSomething()}`, the `doSomething()` function executes immediately when the application is opened in a browser. The expression in the handler is evaluated. Since the returned value of the function isn't a function anymore, nothing would happen when you click the button. But using `onClick={doSomething}` where `doSomething` is a function, it would only be executed if the button is clicked.

The same rules apply for the `onDismiss()` class method.

However, using `onClick={this.onDismiss}` wouldn't suffice, because the `item.objectID` property needs to be passed to the class method to identify the item that should be dismissed. We wrap it into another function to sneak in the property. This concept is called higher-order functions in JavaScript, which we will cover briefly later.

```
...

<button
  onClick={() => this.onDismiss(item.objectID)}
  type="button"
>
  Dismiss
</button>

...
```

We can also define wrapping function outside the method, to pass only the defined function to the handler. Since it needs access to the individual item, it has to live inside the map function block.

```
class App extends Component {

  ...

  render() {
    return (
      <div className="App">
        {this.state.list.map(item => {
          const onHandleDismiss = () =>
            this.onDismiss(item.objectID);

          return (
            <div key={item.objectID}>
              <span>
                <a href={item.url}>{item.title}</a>
              </span>
              <span>{item.author}</span>
              <span>{item.num_comments}</span>
              <span>{item.points}</span>
```

```
              <span>
                <button
                  onClick={onHandleDismiss}
                  type="button"
                >
                  Dismiss
                </button>
              </span>
            </div>
          );
        }
        )}
      </div>
    );
  }
}
```

A function has to be passed to the element's handler. As an example, try this code instead:

```
class App extends Component {

  ...

  render() {
    return (
      <div className="App">
        {this.state.list.map(item =>
            ...
            <span>
              <button
                onClick={console.log(item.objectID)}
                type="button"
              >
                Dismiss
              </button>
            </span>
          </div>
        )}
      </div>
    );
  }
}
```

This method will run when you open your application in the browser, but not when you click the button. The following code would only run when you click the button, a function that is executed when you trigger the handler:

```
...

<button
  onClick={function () {
    console.log(item.objectID)
  }}
```

```
  type="button"
>
  Dismiss
</button>

...
```

Remember: You can transform functions into a JavaScript ES6 arrow function, just as we did with the `onDismiss()` class method:

```
...

<button
  onClick={() => console.log(item.objectID)}
  type="button"
>
  Dismiss
</button>

...
```

Newcomers to React often have difficulty using functions in event handlers, so don't get discouraged if you have trouble on the first pass. You should end up with an inlined JavaScript ES6 arrow function with access to the `objectID` property of the `item` object:

```
class App extends Component {
  ...

  render() {
    return (
      <div className="App">
        {this.state.list.map(item =>
          <div key={item.objectID}>
            ...
            <span>
              <button
                onClick={() => this.onDismiss(item.objectID)}
                type="button"
              >
                Dismiss
              </button>
            </span>
          </div>
        )}
      </div>
    );
  }
}
```

Using arrow functions in event handlers directly impacts your application's

performance. For instance, the `onClick` handler for the `onDismiss()` method wraps the method in another arrow function to pass the item identifier. Every time the `render()` method runs, the handler instantiates the higher-order arrow function. It can have an impact on your application performance, but in most cases you won't notice. If you have a huge table of data with 1000 items and each row or column has an arrow function in an event handler, it is worth thinking about the performance implications, so you could implement a dedicated Button component to bind the method in the constructor. Before that, though, it is premature optimization, and it is more prudent learn the basics of React before thinking about optimization.

Exercises:

- Try the different approaches of using functions in the `onClick` handler of your button

Interactions with Forms and Events

This lesson will explain how to add search functionality by introducing forms and events in React.

We'll add another interaction to see forms and events in React, a search functionality where the input the search field temporarily filters a list based on the title property of an item.

In the first step, we define a form with an input field in JSX:

```
class App extends Component {

  ...

  render() {
    return (
      <div className="App">
        <form>
          <input type="text" />
        </form>
        {this.state.list.map(item =>
          ...
        )}
      </div>
    );
  }
}
```

In the following scenario you will type into the input field and filter the list temporarily by the search term that is used in the input field. To filter the list based on the value of the input field, we store the value of the input field in the local state. We use **synthetic events** in React to access a value in an event payload.

Let's define a `onChange` handler for the input field:

```
class App extends Component {

  ...

  render() {
    return (
```

```
      <div className="App">
        <form>
          <input

            type="text"
            onChange={this.onSearchChange}
          />
        </form>
        ...
      </div>
    );
  }
}
```

The function is bound to the component, so it is a class method again. You just need to bind and define the method:

```
class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      list,
    };

    this.onSearchChange = this.onSearchChange.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  onSearchChange() {
    ...
  }

  ...
}
```

When using a handler in your element, you get access to the synthetic React event in your callback function's signature.

```
class App extends Component {

  ...

  onSearchChange(event) {
    ...
  }

  ...
}
```

The event has the value of the input field in its target object, so you can update the local state with a search term using `this.setState()`.

```
class App extends Component {

  ...

  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  ...
}
```

Don't forget to define the initial state for the `searchTerm` property in the constructor. The input field should be empty in the beginning, so its value is an empty string.

```
class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      list,
      searchTerm: '',
    };

    this.onSearchChange = this.onSearchChange.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  ...
}
```

We store the input value to the local state every time the value in the input field changes.

We can assume that when we update `searchTerm` with `this.setState()`, the list also needs to be passed to preserve it. React's `this.setState()` is a shallow merge, however, so it preserves the sibling properties in the state object when it updates a property. The list state, though you have already dismissed an item from it, stays the same when updating the `searchTerm` property.

Returning to our application, we see the list isn't filtered yet, based on the input field value stored in the local state. We need to filter the list temporarily based on the `searchTerm`, and we have everything we need to perform this operation. In the `render()` method, before mapping over the list, we apply a

filter to it. The filter will only evaluate if the `searchTerm` matches the title property of the item. We've already used the built-in JavaScript filter functionality, so let's use it again to sneak in the filter function before the map function. The filter function returns a new array, so the map function can be used on it.

```
class App extends Component {

  ...

  render() {
    return (
      <div className="App">
        <form>
          <input
            type="text"
            onChange={this.onSearchChange}
          />
        </form>
        {this.state.list.filter(...).map(item =>
          ...
        )}
      </div>
    );
  }
}
```

Let's approach the filter function in a different way this time. We want to define the filter argument, which is the function passed to the filter function outside the ES6 class component. We don't have access to the state of the component, so we have no access to the `searchTerm` property to evaluate the filter condition. This means we'll need to pass the `searchTerm` to the filter function, returning a new function to evaluate the condition. This is called a higher-order function.

It makes sense to know about higher-order functions, because React deals with a concept called higher-order components. You will get to know the concept later in the course. Now again, let's focus on the filter functionality.

First, you have to define the higher-order function outside of your App component.

```
function isSearched(searchTerm) {
  return function(item) {
    // some condition which returns true or false
  }
}
```

```
class App extends Component {


  ...

}
```

The function takes the `searchTerm` and returns another function, because the filter function only takes that type as its input. The returned function has access to the item object, because it is the one passed to the filter function.

It will also be used to filter the list based on the condition defined in the function, so let's define the condition:

```
function isSearched(searchTerm) {
  return function(item) {
    return item.title.toLowerCase().includes(searchTerm.toLowerCase());
  }
}

class App extends Component {

  ...

}
```

The condition matches the incoming `searchTerm` pattern with the title property of the item from your list. You can do that with the built-in `includes` JavaScript functionality. When the pattern matches, it returns true and the item stays in the list; when the pattern doesn't match, the item is removed from the list. Don't forget to match the capitalizion on both strings to the letter, as there will be mismatches between the search term 'redux' and an item title 'Redux'. Since we are working on a immutable list and return a new list by using the filter function, the original list in the local state isn't modified at all.

We cheated a bit using JavaScript ES7 features, but these aren't present in ES5. For ES5, use the `indexOf()` function to get the index of the item in the list instead. When the item is in the list, `indexOf()` will return its index in the array.

```
// comment either ES5 or ES6 code

var string = 'Robin';
var pattern = 'Rob';
```

```
// ES5
var result = string.indexOf(pattern) !== -1;


// ES6
// var result = string.includes(pattern);

console.log(result);
```

Another neat refactoring can be done with an ES6 arrow function again. It makes the function more concise:

```
// comment either ES5 or ES6 code

// ES5
function isSearched(searchTerm) {
  return function(item) {
    return !searchTerm || item.title.toLowerCase().includes(searchTerm.toLowerCase());
  }
}

// ES6
// const isSearched = (searchTerm) => (item) =>
//   !searchTerm || item.title.toLowerCase().includes(searchTerm.toLowerCase());

var searchTerm = 'rea';
var item = { title: 'React' };

console.log(isSearched(searchTerm)(item));
```

The React ecosystem uses a lot of functional programming concepts, often using functions that return functions (the concept is called high-order functions) to pass information. JavaScript ES6 lets us express these even more concisely with arrow functions.

Last but not least, use the defined `isSearched()` function to filter lists. We pass it the `searchTerm` property from the local state, so that it returns the filter's input function and filters your list based on the filter condition. After that it maps over the filtered list to display an element for each list item.

```
class App extends Component {

  ...
```

```
  render() {
    return (

      <div className="App">
        <form>
          <input
            type="text"
            onChange={this.onSearchChange}
          />
        </form>
        {this.state.list.filter(isSearched(this.state.searchTerm)).map(item =>
          ...
        )}
      </div>
    );
  }
}
```

The search functionality should work now. Try it.

```
import React, { Component } from 'react';
require('./App.css');

const list = [
  {
    title: 'React',
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  {
    title: 'Redux',
    url: 'https://redux.js.org/',
    author: 'Dan Abramov, Andrew Clark',
    num_comments: 2,
    points: 5,
    objectID: 1,
  },
];

const isSearched = (searchTerm) => (item) =>
  item.title.toLowerCase().includes(searchTerm.toLowerCase());

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      list,
      searchTerm: '',
    };

    this.onSearchChange = this.onSearchChange.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }
```

```
  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  onDismiss(id) {
    const isNotId = item => item.objectID !== id;
    const updatedList = this.state.list.filter(isNotId);
    this.setState({ list: updatedList });
  }

  render() {
    return (
      <div className="App">
        <form>
          <input
            type="text"
            onChange={this.onSearchChange}
          />
        </form>
        {this.state.list.filter(isSearched(this.state.searchTerm)).map(item =>
          <div key={item.objectID}>
            <span>
              <a href={item.url}>{item.title}</a>
            </span>
            <span>{item.author}</span>
            <span>{item.num_comments}</span>
            <span>{item.points}</span>
            <span>
              <button
                onClick={() => this.onDismiss(item.objectID)}
                type="button"
              >
                Dismiss
              </button>
            </span>
          </div>
        )}
      </div>
    );
  }
}

export default App;
```

Further Reading:

- Read more about React events
- Read more about higher-order functions

ES6 Destructuring

A brief introduction to destructuring and things we can access through it.

Destructuring in JavaScript ES6 provides easier access to properties in objects and arrays. Compare the following snippet in JavaScript ES5 and ES6:

```
// comment either ES5 or ES6 code

const user = {
  firstname: 'Robin',
  lastname: 'Wieruch',
};

// ES5
// var firstname = user.firstname;
// var lastname = user.lastname;

// ES6
var { firstname, lastname } = user;

console.log(firstname + ' ' + lastname);
```

While we add an extra line each time we access an object property in JavaScript ES5, it takes just one line in JavaScript ES6. For readability, use multi lines when you destructure an object into multiple properties.

```
var {
  firstname,
  lastname
} = user;
```

The same concept applies to arrays. You can destructure them, too, again using multilines to keep your code scannable and readable.

```
var users = ['Robin', 'Andrew', 'Dan'];

var [
```

```
  userOne,
  userTwo,
  userThree
] = users;

console.log(userOne, userTwo, userThree);
```

Note that the local state object in the App component can get destructured the same way. You can shorten the filter and map line of code.

```
  render() {
    const { searchTerm, list } = this.state;
    return (
      <div className="App">
        ...
        {list.filter(isSearched(searchTerm)).map(item =>
          ...
        )}
      </div>
    );
```

You can do it the ES5 or ES6 way:

```
// ES5
var searchTerm = this.state.searchTerm;
var list = this.state.list;

// ES6
var { searchTerm, list } = this.state;
```

But since the course uses JavaScript ES6 most of the time, you should stick to it.

Further Reading:

- Read about ES6 destructuring

Controlled Components

A brief introduction to Controlled components and how it is different from uncontrolled components.

We covered unidirectional data flows before, and the same law applies for the input field, which updates the local state with the `searchTerm` to filter the list. When the state changes, the `render()` method runs again and uses the recent `searchTerm` from the local state to apply the filter condition.

But didn't we forget something in the input element? An HTML input tag comes with a `value` attribute. The value attribute usually contains the value shown in the input field. In this case, that is the `searchTerm` property. Form elements such as `<input>`, `<textarea>`, and `<select>` hold their own state in plain HTML. They modify the value internally once someone changes it from the outside. In React, that's called an **uncontrolled component**, because it handles its own state. We want to make sure those elements are **controlled components** instead.

To do this, we set the value attribute of the input field, which is already saved in the `searchTerm` state property, so we can access it from there:

```
class App extends Component {

  ...

  render() {
    const { searchTerm, list } = this.state;
    return (
      <div className="App">
        <form>
          <input
            type="text"
            value={searchTerm}
            onChange={this.onSearchChange}
          />
        </form>
        ...
      </div>
    );
  }
}
```

The unidirectional data flow loop for the input field is self-contained, and the local component state is the single source of truth for the input field.

Local state management and unidirectional data flow might be new to you, but once you adjust to it, it will likely become your natural flow of React implementation. React brings novel patterns with the unidirectional data flow, which have been adopted by several frameworks and libraries which create single page applications.

Further Readings:

- Read about React forms
- Learn more about different controlled components

Split Up Components

This lesson explains how to break-down a large component into smaller components.

Now we have one large App component that keeps growing and may eventually become too complex to manage efficiently. We need to split it into smaller, more manageable parts by creating separate components for search input and the items list.

```
class App extends Component {

  ...

  render() {
    const { searchTerm, list } = this.state;
    return (
      <div className="App">
        <Search />
        <Table />
      </div>
    );
  }
}
```

We pass the components properties that they can use themselves. The App component needs to pass the properties managed in the local state and its class methods.

```
class App extends Component {

  ...

  render() {
    const { searchTerm, list } = this.state;
    return (
      <div className="App">
        <Search
          value={searchTerm}
          onChange={this.onSearchChange}
        />
        <Table
          list={list}
          pattern={searchTerm}
          onDismiss={this.onDismiss}
```

```
        />
      </div>
    );

  }
}
```

Now we define the components next to the App component, which will be done using JavaScript ES6 by using classes. They render the same elements as before.

The first one is the Search component:

```
class App extends Component {
  ...
}

class Search extends Component {
  render() {
    const { value, onChange } = this.props;
    return (
      <form>
        <input
          type="text"
          value={value}
          onChange={onChange}
        />
      </form>
    );
  }
}
```

The second one is the Table component.

```
...

class Table extends Component {
  render() {
    const { list, pattern, onDismiss } = this.props;
    return (
      <div>
        {list.filter(isSearched(pattern)).map(item =>
          <div key={item.objectID}>
            <span>
              <a href={item.url}>{item.title}</a>
            </span>
            <span>{item.author}</span>
            <span>{item.num_comments}</span>
            <span>{item.points}</span>
            <span>
              <button
                onClick={() => onDismiss(item.objectID)}
                type="button"
              >
                Dismiss
```

```
              </button>
            </span>
          </div>

        )}
      </div>
    );
  }
}
```

Now you have three ES6 class components. Notice the `props` object is accessible via the class instance by using `this`. Props, short for properties, have all the values passed to the components when we used App component. That way, components can pass properties down the component tree.

By extracting these components from the App component, they become reusable. Since components get their values using the `props` object, you can pass different props to your components every time you use them somewhere else.

```
import React, { Component } from 'react';
require('./App.css');

const list = [
  {
    title: 'React',
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  {
    title: 'Redux',
    url: 'https://redux.js.org/',
    author: 'Dan Abramov, Andrew Clark',
    num_comments: 2,
    points: 5,
    objectID: 1,
  },
];

const isSearched = (searchTerm) => (item) =>
  item.title.toLowerCase().includes(searchTerm.toLowerCase());

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      list,
      searchTerm: '',
    };
```

```
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }


  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  onDismiss(id) {
    const isNotId = item => item.objectID !== id;
    const updatedList = this.state.list.filter(isNotId);
    this.setState({ list: updatedList });
  }

  render() {
    const { searchTerm, list } = this.state;
    return (
      <div className="App">
        <Search
          value={searchTerm}
          onChange={this.onSearchChange}
        />
        <Table
          list={list}
          pattern={searchTerm}
          onDismiss={this.onDismiss}
        />
      </div>
    );
  }
}

class Search extends Component {
  render() {
    const { value, onChange } = this.props;
    return (
      <form>
        <input
          type="text"
          value={value}
          onChange={onChange}
        />
      </form>
    );
  }
}

class Table extends Component {
  render() {
    const { list, pattern, onDismiss } = this.props;
    return (
      <div>
        {list.filter(isSearched(pattern)).map(item =>
          <div key={item.objectID}>
            <span>
              <a href={item.url}>{item.title}</a>
            </span>
            <span>{item.author}</span>
            <span>{item.num_comments}</span>
            <span>{item.points}</span>
            <span>
              <button
```

```
                onClick={() => onDismiss(item.objectID)}
                type="button"
              >
                Dismiss
              </button>
            </span>
          </div>
        )}
      </div>
    );
  }
}

export default App;
```

Exercises:

- Discover more components that can be split up like the Search and Table components, but wait until we've covered more of its concepts before you implement any of them.

Composable Components

In this lesson, you will get to learn about composable components. You will also extend the search functionality that we implemented in the previous lessons.

The `children` prop is used to pass elements to components from above, which are unknown to the component itself but make it possible to compose components together. We'll see how this looks when you pass a text string as a child to the Search component.

```
class App extends Component {

  ...

  render() {
    const { searchTerm, list } = this.state;
    return (
      <div className="App">
        <Search
          value={searchTerm}
          onChange={this.onSearchChange}
        >
          Search
        </Search>
        <Table
          list={list}
          pattern={searchTerm}
          onDismiss={this.onDismiss}
        />
      </div>
    );
  }
}
```

Now the Search component can destructure the `children` property from the `props` object, and specify where it should be displayed.

```
class Search extends Component {
  render() {
    const { value, onChange, children } = this.props;
    return (
      <form>
        {children} <input
          type="text"
          value={value}
```

```
          value={value}
          onChange={onChange}
        />
      </form>
    );
  }
}
```

The “Search” text should now be visible next to your input field. When you use the Search component elsewhere, you can use different entities, since it’s not just text that can be passed as children. You can also pass an element, or element trees that can be encapsulated by components, as children. The children property makes it possible to weave components into each other.

```
import React, { Component } from 'react';
require('./App.css');

const list = [
  {
    title: 'React',
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  {
    title: 'Redux',
    url: 'https://redux.js.org/',
    author: 'Dan Abramov, Andrew Clark',
    num_comments: 2,
    points: 5,
    objectID: 1,
  },
];

const isSearched = (searchTerm) => (item) =>
  item.title.toLowerCase().includes(searchTerm.toLowerCase());

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      list,
      searchTerm: '',
    };

    this.onSearchChange = this.onSearchChange.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }
```

```
  onDismiss(id) {
    const isNotId = item => item.objectID !== id;
    const updatedList = this.state.list.filter(isNotId);

    this.setState({ list: updatedList });
  }

  render() {
    const { searchTerm, list } = this.state;
    return (
      <div className="App">
        <Search
          value={searchTerm}
          onChange={this.onSearchChange}
        >
          Search
        </Search>
        <Table
          list={list}
          pattern={searchTerm}
          onDismiss={this.onDismiss}
        />
      </div>
    );
  }
}

class Search extends Component {
  render() {
    const { value, onChange, children } = this.props;
    return (
      <form>
        {children} <input
          type="text"
          value={value}
          onChange={onChange}
        />
      </form>
    );
  }
}

class Table extends Component {
  render() {
    const { list, pattern, onDismiss } = this.props;
    return (
      <div>
        {list.filter(isSearched(pattern)).map(item =>
          <div key={item.objectID}>
            <span>
              <a href={item.url}>{item.title}</a>
            </span>
            <span>{item.author}</span>
            <span>{item.num_comments}</span>
            <span>{item.points}</span>
            <span>
              <button
                onClick={() => onDismiss(item.objectID)}
                type="button"
              >
                Dismiss
              </button>
            </span>
```

```
          </div>
        )}
      </div>
    );
  }
}

export default App;
```

Further Reading:

- Read about the composition model of React

Reusable Components

This lesson explains how we can implement component hierarchies in React to achieve reusability.

Reusable and composable components empower you to come up with capable component hierarchies, the foundation of React's view layer. The last sections mentioned reusability, and now we can see how reusing the Table and Search components works in our case. Even the App component is reusable, as it can be instantiated elsewhere as well.

Let's define one more reusable component, a Button component which we'll eventually reuse often:

```
class Button extends Component {
  render() {
    const {
      onClick,
      className,
      children,
    } = this.props;

    return (
      <button
        onClick={onClick}
        className={className}
        type="button"
      >
        {children}
      </button>
    );
  }
}
```

It might seem redundant to declare components like this, but it's not. We use a `Button` component instead of a `button` element, which spares only the `type="button"`. It might not seem like a huge win, but these measures are about long-term, however. Imagine you have several buttons in your application, and you want to change an attribute, style, or behavior for just one. Without the component, you'd have to change (refactor) each one. The Button component ensures that the operation has a single source of truth, or

one Button to refactor all the others at once.

Since you already have a button element, you can use the Button component instead. It omits the type attribute, because the Button component specifies it.

```
class Table extends Component {
  render() {
    const { list, pattern, onDismiss } = this.props;
    return (
      <div>
        { list.filter(isSearched(pattern)).map(item =>
          <div key={item.objectID}>
            <span>
              <a href={item.url}>{item.title}</a>
            </span>
            <span>{item.author}</span>
            <span>{item.num_comments}</span>
            <span>{item.points}</span>
            <span>
              <Button onClick={() => onDismiss(item.objectID)}>
                Dismiss
              </Button>
            </span>
          </div>
        )}
      </div>
    );
  }
}
```

The Button component expects a `className` property in the `props`. The `className` attribute is another React derivate for the HTML attribute class. We didn't pass any `className` when the Button was used, though. It should be more explicit in our Button component that the `className` is optional, so we'll assign a default value in the object destructuring.

```
class Button extends Component {
  render() {
    const {
      onClick,
      className = '',
      children,
    } = this.props;

    ...
  }
}
```

Now, whenever there is no `className` property specified in the Button component, the value will be an empty string instead of `undefined`.

Further Reading:

- Read about how to pass props in React

Component Declarations

In this lesson, we will discuss the three type of class components that we are using in our application i.e. Functional Stateless Components, ES6 Class components, and React.createClass

Now we have four ES6 class components, but our application can still be improved using functional stateless components as an alternative for ES6 class components. Before you refactor your components, let's introduce the different types.

- **Functional Stateless Components** are functions that take input and return an output. The inputs are the props, and the output is a component instance in plain JSX. So far, it is quite similar to an ES6 class component. However, functional stateless components are functions (functional) and they have no local state (stateless). You cannot access or update the state with `this.state` or `this.setState()` because there is no `this` object. Additionally, they have no lifecycle methods except for the `render()` method which will be applied implicitly in functional stateless components. You didn't learn about lifecycle methods yet, but you already used two: `constructor()` and `render()`. The constructor runs only once in the lifetime of a component, whereas the `render()` class method runs once at the beginning and every time the component updates. Keep in mind that functional stateless components have no lifecycle methods when we arrive at lifecycle methods chapter later.

- **ES6 Class Components** extend from the React component. The `extend` hooks all the lifecycle methods, available in the React component API, to the component. This is how we were able to use the `render()` class method. You can also store and manipulate state in ES6 class components using `this.state` and `this.setState()`.

- **React.createClass** was used in older versions of React and is still used in JavaScript ES5 React applications. But Facebook declared it as deprecated

in favor of JavaScript ES6. They even added a deprecation warning in version 15.5, so we will not use it in the course.

When deciding when to use functional stateless components over ES6 class components, a good rule of thumb is to use functional stateless components when you don't need local state or component lifecycle methods. Usually, we implement components as functional stateless components, but once access to the state or lifecycle methods is required, we have to refactor it to an ES6 class component. We started the other way around in our application for the sake of learning.

Returning to the application, we see the App component uses local state, so it has to stay as an ES6 class component. The other three ES6 class components are stateless, so they don't need access to `this.state` or `this.setState()`, and they have no lifecycle methods. We're going to refactor the Search component to a stateless functional component. The Table and Button component refactoring will become your exercise.

```
function Search(props) {
  const { value, onChange, children } = props;
  return (
    <form>
      {children} <input
        type="text"
        value={value}
        onChange={onChange}
      />
    </form>
  );
}
```

The `props` are accessible in the function signature, and the return value is JSX; but we can do more with the code in a functional stateless component using ES6 destructuring. The best practice is to use it in the function signature to destructure the `props`:

```
function Search({ value, onChange, children }) {
  return (
    <form>
      {children} <input
        type="text"
        value={value}
        onChange={onChange}
```

```
      />
    </form>

  );
}
```

Remember that ES6 arrow functions allow you to keep your functions concise and let you remove the block body of the function. In a concise body, an implicit return is attached, letting us remove the `return` statement. Since the functional stateless component is a function, it can be made more concise as well:

```
const Search = ({ value, onChange, children }) =>
  <form>
    {children} <input
      type="text"
      value={value}
      onChange={onChange}
    />
  </form>
```

The last step is especially useful to enforce props as input and JSX as output. Still, you could *do something* in between by using a block body in your ES6 arrow function:

```
const Search = ({ value, onChange, children }) => {

  // do something

  return (
    <form>
      {children} <input
        type="text"
        value={value}
        onChange={onChange}
      />
    </form>
  );
}
```

Now you have one lightweight functional stateless component. When we need access to the local component state or lifecycle methods, we can refactor it to an ES6 class component. When using block bodies, programmers tend to make their functions more complex, but leaving the block body out lets you focus on the input and output. JavaScript ES6 in React components makes components more readable and elegant.

Exercises:

- Refactor the Table and Button component to stateless functional components

Further Reading

- Read about ES6 class components and functional stateless components

Styling Components

In this section, we'll add some basic styling to our application and components using the src/App.css and src/index.css files.

These two files `src/App.css` and `src/index.css` should already be in your project since you have bootstrapped it with *create-react-app*. They should be imported in your `src/App.js` and `src/index.js` files too. The following is CSS that can be copied and pastes to these files but feel free to use your own if you're comfortable with CSS.

First, styling for our overall application:

`src/index.css`

```
body {
  color: #222;
  background: #f4f4f4;
  font: 400 14px CoreSans, Arial, sans-serif;
}

a {
  color: #222;
}

a:hover {
  text-decoration: underline;
}

ul, li {
  list-style: none;
  padding: 0;
  margin: 0;
}

input {
  padding: 10px;
  border-radius: 5px;
  outline: none;
  margin-right: 10px;
  border: 1px solid #dddddd;
}

button {
  padding: 10px;
  border-radius: 5px;
```

```
  border: 1px solid #dddddd;
  background: transparent;
  color: #808080;

  cursor: pointer;
}

button:hover {
  color: #222;
}

*:focus {
  outline: none;
}
```

Second, styling for your components in the App file:

`src/App.css`

```
.page {
  margin: 20px;
}

.interactions {
  text-align: center;
}

.table {
  margin: 20px 0;
}

.table-header {
  display: flex;
  line-height: 24px;
  font-size: 16px;
  padding: 0 10px;
  justify-content: space-between;
}

.table-empty {
  margin: 200px;
  text-align: center;
  font-size: 16px;
}

.table-row {
  display: flex;
  line-height: 24px;
  white-space: nowrap;
  margin: 10px 0;
  padding: 10px;
  background: #ffffff;
  border: 1px solid #e3e3e3;
}

.table-header > span {
  overflow: hidden;
  text-overflow: ellipsis;
  padding: 0 5px;
```

```
}

.table-row > span {

  overflow: hidden;
  text-overflow: ellipsis;
  padding: 0 5px;
}

.button-inline {
  border-width: 0;
  background: transparent;
  color: inherit;
  text-align: inherit;
  -webkit-font-smoothing: inherit;
  padding: 0;
  font-size: inherit;
  cursor: pointer;
}

.button-active {
  border-radius: 0;
  border-bottom: 1px solid #38BB6C;
}
```

Now we can use this style with some of our components. Remember to use React `className` instead of `class` as an HTML attribute.

First, apply it in your App ES6 class component:

```
class App extends Component {

  ...

  render() {
    const { searchTerm, list } = this.state;
    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
          >
            Search
          </Search>
        </div>
        <Table
          list={list}
          pattern={searchTerm}
          onDismiss={this.onDismiss}
        />
      </div>
    );
  }
}
```

Second, apply it in your Table functional stateless component:

```
const Table = ({ list, pattern, onDismiss }) =>
  <div className="table">
    {list.filter(isSearched(pattern)).map(item =>
      <div key={item.objectID} className="table-row">
        <span>
          <a href={item.url}>{item.title}</a>
        </span>
        <span>{item.author}</span>
        <span>{item.num_comments}</span>
        <span>{item.points}</span>
        <span>
          <Button
            onClick={() => onDismiss(item.objectID)}
            className="button-inline"
          >
            Dismiss
          </Button>
        </span>
      </div>
    )}
  </div>
```

Now the application and components have been styled with basic CSS. Further, we know JSX mixes up HTML and JavaScript, and now we could arguably add CSS to that mix. That's called inline style, where you can define JavaScript objects and pass them to the style attribute of an element.

Let's keep the Table column width flexible by using inline style.

```
const Table = ({ list, pattern, onDismiss }) =>
  <div className="table">
    {list.filter(isSearched(pattern)).map(item =>
      <div key={item.objectID} className="table-row">
        <span style={{ width: '40%' }}>
          <a href={item.url}>{item.title}</a>
        </span>
        <span style={{ width: '30%' }}>
          {item.author}
        </span>
        <span style={{ width: '10%' }}>
          {item.num_comments}
        </span>
        <span style={{ width: '10%' }}>
          {item.points}
        </span>
        <span style={{ width: '10%' }}>
          <Button
            onClick={() => onDismiss(item.objectID)}
            className="button-inline"
          >
            Dismiss
          </Button>
        </span>
```

```
      </span>
    </div>
  )}
</div>
```

The style is inlined now. Define the style objects outside of your elements to make it cleaner.

```
const largeColumn = {
  width: '40%',
};

const midColumn = {
  width: '30%',
};

const smallColumn = {
  width: '10%',
};
```

After that, we use them in the columns: `<span style={smallColumn}>`. There are different opinions and solutions about styling in React, but the pure inline CSS we used is sufficient for this tutorial.

Further Readings:

I don't want to be opinionated here, but I want to leave you some more options. You can read about them and apply them on your own:

- styled-components
- CSS Modules

But if you are new to React, I would recommend sticking to pure CSS and inline style for now.

Recap

You have learned the basics on how to write your own React application! This lesson will provide you a recap to what you have learned so far and will also share how your App.js code looks by now.

Let's recap the last chapter:

- **React**
  - Use `this.state` and `setState()` to manage your local component state
  - Pass functions or class methods to your element handler
  - Use forms and events in React to add interactions
  - Unidirectional data flow is an important concept in React
  - Embrace controlled components
  - Compose components with children and reusable components
  - Usage and implementation of ES6 class components and functional stateless components
  - Approaches to style your components
- **ES6**
  - Functions that are bound to a class are class methods
  - Destructuring of objects and arrays
  - Default parameters
- **General**
  - Higher-order functions

Again, it makes sense to take a break, internalize the lessons, and apply them on your own. Experiment with the source code you have written so far. The source code for this project is found in the official repository.

Your `src/App.js` should look like the following by now:

```
import React, { Component } from 'react';
require('./App.css');
```

```
require('./App.css');

const list = [
  {
    title: 'React',
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  {
    title: 'Redux',
    url: 'https://redux.js.org/',
    author: 'Dan Abramov, Andrew Clark',
    num_comments: 2,
    points: 5,
    objectID: 1,
  },
];

const isSearched = (searchTerm) => (item) =>
  item.title.toLowerCase().includes(searchTerm.toLowerCase());

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      list,
      searchTerm: '',
    };

    this.onSearchChange = this.onSearchChange.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  onDismiss(id) {
    const isNotId = item => item.objectID !== id;
    const updatedList = this.state.list.filter(isNotId);
    this.setState({ list: updatedList });
  }

  render() {
    const { searchTerm, list } = this.state;
    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
          >
            Search
          </Search>
        </div>
        <Table
          list={list}
          pattern={searchTerm}
```

```
          onDismiss={this.onDismiss}
        />
      </div>
    );
  }
}


const Search = ({ value, onChange, children }) =>
  <form>
    {children} <input
      type="text"
      value={value}
      onChange={onChange}
    />
  </form>

const Table = ({ list, pattern, onDismiss }) =>
  <div className="table">
    {list.filter(isSearched(pattern)).map(item =>
      <div key={item.objectID} className="table-row">
        <span style={{ width: '40%' }}>
          <a href={item.url}>{item.title}</a>
        </span>
        <span style={{ width: '30%' }}>
          {item.author}
        </span>
        <span style={{ width: '10%' }}>
          {item.num_comments}
        </span>
        <span style={{ width: '10%' }}>
          {item.points}
        </span>
        <span style={{ width: '10%' }}>
          <Button
            onClick={() => onDismiss(item.objectID)}
            className="button-inline"
          >
            Dismiss
          </Button>
        </span>
      </div>
    )}
  </div>

const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

export default App;
```

Lifecycle Methods

This lesson briefly introduces the lifecycle methods that can be overridden in a React ES6 class component

Now it's time to get real with APIs and move past sample data. If you are not familiar, I encourage you to read my article on how I got to know APIs.

For our first foray into the concept, we will be using Hacker News, a solid news aggregator about tech topics. In this exercise, we will use the Hacker News API to fetch trending stories. There are basic and search APIs to get data from the platform. Search makes sense in the application that we are building in this course because we want to be able to search Hacker News stories. Visit the API specification to get an understanding of the data structure.

Lifecycle Methods

You may remember lifecycle methods were mentioned briefly in the last chapter, as a hook into the lifecycle of a React component. They can be used in ES6 class components, but not in functional stateless components. Besides the `render()` method, there are several methods that can be overridden in a React ES6 class component. All of these are the lifecycle methods.

Mounting

We have already covered two lifecycle methods that can be used in an ES6 class component:

- The constructor is only called when an instance of the component is created and inserted in the DOM. The component gets instantiated in a process called mounting.
- The `render()` method is called during the mount process too, but also when the component updates. Each time the state or the props of a

component changes, the `render()` method is called.

There are two more lifecycle methods when mounting a component: `getDerivedStateFromProps()` and `componentDidMount()`. The constructor is called first, `getDerivedStateFromProps()` is called before the `render()` method, and `componentDidMount()` is called after the `render()` method.

Overall, the mounting process has 4 lifecycle methods, invoked in the following order:

- constructor()
- getDerivedStateFromProps()
- render()
- componentDidMount()

Updating

For the update lifecycle of a component when the state or the props change, there are 5 lifecycle methods, in the following order:

- getDerivedStateFromProps()
- shouldComponentUpdate()
- render()
- getSnapshotBeforeUpdate()
- componentDidUpdate()

Unmounting

Lastly, there is the unmounting lifecycle. It has only one lifecycle method: `componentWillUnmount()`.

Description of Lifecycle Methods

You don't need to know all the lifecycle methods from the beginning, and even in a large React application you'll only use a few of them besides the

`constructor()` and the `render()` methods. Still, it is good to know each lifecycle method can be used for specific purposes:

- **constructor(props)** is called when the component gets initialized. You can set an initial component state and bind class methods during that lifecycle method.

- **static getDerivedStateFromProps(props, state)** is called before the `render()` lifecycle method, both on the initial mount and on the subsequent updates. It should return an object to update the state, or null to update nothing. It exists for **rare** use cases where the state depends on changes in props over time. It is important to know that this is a static method and it doesn't have access to the component instance.

- **render()** is a mandatory lifecycle method that returns elements as an output of the component. The method should be pure, so it shouldn't modify the component state. It gets an input as props and state and returns an element.

- **componentDidMount()** is called once, when the component mounted. That's the perfect time to do an asynchronous request to fetch data from an API. The fetched data is stored in the local component state to display it in the `render()` lifecycle method.

- **shouldComponentUpdate(nextProps, nextState)** is always called when the component updates due to state or props changes. You will use it in mature React applications for performance optimization. Depending on a boolean that you return from this lifecycle method, the component and all its children will render or will not render on an update lifecycle. You can prevent the render lifecycle method of a component.

- **getSnapshotBeforeUpdate(prevProps, prevState)** is a lifecycle method, invoked before the most recently rendered output is committed to the DOM. In rare cases, the component needs to capture information from the DOM before it is potentially changed. This lifecycle method enables the component to do it. Another method (`componentDidUpdate()`) will receive any value returned by `getSnapshotBeforeUpdate()` as a parameter.

- **componentDidUpdate(prevProps, prevState, snapshot)** is a lifecycle method that is invoked immediately after updating, but not for the initial render. You can use it as to perform DOM operations or to perform more

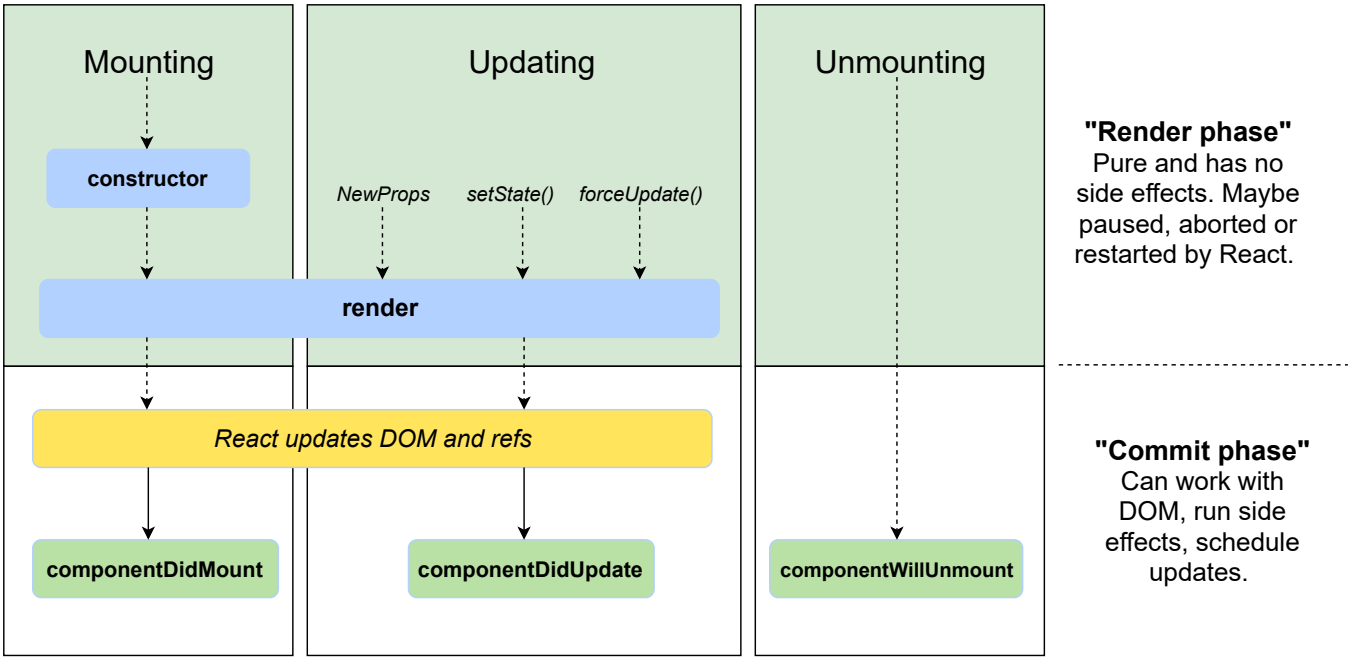
asynchronous requests. If your component implements the

`getSnapshotBeforeUpdate()` method, the value it returns will be received as the `snapshot` parameter.

- **componentWillUnmount()** is called before you destroy your component. You can use this lifecycle method to perform any cleanup tasks.

As you may have gathered, the `constructor()` and `render()` lifecycle methods are the most commonly used lifecycle methods for ES6 class components. The `render()` method is always required to return a component instance.

Lastly, `componentDidCatch(error, info)` was introduced in React 16 as a way to catch errors in components. For instance, displaying the sample list in your application works fine, but there could be a time when a list in the local state is set to `null` by accident (e.g. when fetching the list from an external API, but the request failed and you set the local state of the list to null). It becomes impossible to filter and map the list because it is `null` and not an empty list. The component would be broken, and the whole application would fail. Using `componentDidCatch()`, you can catch the error, store it in your local state, and show a message to the user.



The React LifeCycle Diagram (Taken from http://projects.wojtekmaj.pl/react-lifecycle-methods-diagram/)

Q

There is a lifecyle method in React component which is overidden to prevent updating the components. Which of the following is that method?

COMPLETED 0%

1 of 1

Further Reading:

- Read about lifecycle methods in React
- Read about the state related to lifecycle methods in React
- Read about error handling in components

Fetching Data

You will get to learn how to fetch data from an API using one the lifecycle methods that we discussed in the previous lesson.

Now we're prepared to fetch data from the Hacker News API. There was one lifecycle method mentioned that can be used to fetch data: `componentDidMount()`. Before we use it, let's set up the URL constants and default parameters to break the URL endpoint for the API request into smaller pieces.

```
import React, { Component } from 'react';
import './App.css';

const DEFAULT_QUERY = 'redux';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
```

In JavaScript ES6, you can use template literals for string concatenation or interpolation. You will use it to concatenate your URL for the API endpoint.

```
const DEFAULT_QUERY = 'redux';
const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';

// comment either ES5 or ES6

// ES6
const url1 = `${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${DEFAULT_QUERY}`;

// ES5
const url2 = PATH_BASE + PATH_SEARCH + '?' + PARAM_SEARCH + DEFAULT_QUERY;

console.log(url1);
// output: https://hn.algolia.com/api/v1/search?query=redux
// output
console.log(url2);
```

This will keep your URL composition flexible in the future. Below, the entire data fetch process will be presented, and each step will be explained afterward.

```
import React, { Component } from 'react'
class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      result: null,
      searchTerm: DEFAULT_QUERY,
    };
    this.setSearchTopStories = this.setSearchTopStories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  setSearchTopStories(result) {
    this.setState({ result });
  }

  componentDidMount() {
    const { searchTerm } = this.state;

    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}`)
      .then(response => response.json())
      .then(result => this.setSearchTopStories(result))
      .catch(error => error);
  }
//  ...
}
```

First, we remove the sample list of items, because we will return a real list from the Hacker News API, so the sample data is no longer used. The initial state of your component has an empty result and default search term now. The same default search term is used in the input field of the Search component, and in your first request.

Second, you use the `componentDidMount()` lifecycle method to fetch the data after the component mounted. The first fetch uses default search term from the local state. It will fetch “redux” related stories, because that is the default parameter.

Third, the native fetch API is used. The JavaScript ES6 template strings allow it to compose the URL with the `searchTerm`. The URL is the argument for the native fetch API function. The response is transformed to a JSON data structure, a mandatory step in a native fetch with JSON data structures, after which it can be set as result in the local component state. If an error occurs during the request, the function will run into the catch block instead of the then block.

Last, remember to bind your new component method in the constructor.

Now you can use the fetched data instead of the sample list. Note that the result is not only a list of data, but a complex object with meta information and a list of hits that are news stories. You can output the lcoal state with `console.log(this.state);` in your `render()` method to visualize it.

In the next step, we use the result to render it. But we will prevent it from rendering anything, so we will return null, when there is no result in the first place. Once the request to the API has succeeded, the result is saved to the state and the App component will re-render with the updated state.

```
class App extends Component {

  ...

  render() {
    const { searchTerm, result } = this.state;

    if (!result) { return null; }

    return (
      <div className="page">
        ...
        <Table
          list={result.hits}
          pattern={searchTerm}
          onDismiss={this.onDismiss}
        />
      </div>
    );
  }
}
```

Now, let's recap what happens during the component lifecycle. Your component is initialized by the constructor, after which it renders for the first time. We prevented it from displaying anything, because the result in the local state is null. It is allowed to return null for a component to display nothing.

Then the `componentDidMount()` lifecycle method fetches the data from the Hacker News API asynchronously. Once the data arrives, it changes your local component state in `setSearchTopStories()`. The update lifecycle activates because the local state was updated. The component runs the `render()` method again, but this time with populated result in your local component state. The component and the Table component will be rendered with its content.

We used the native fetch API supported by most browsers to perform an asynchronous request to an API. The *create-react-app* configuration makes sure it is supported by all browsers. There are also third-party node packages that you can use to substitute the native fetch API: axios. You will use axios later in this course.

In this course, we build on JavaScript's shorthand notation for truthfulness checks. In the previous example, `if (!result)` was used in favor of `if (result === null)`. The same applies for other cases as well. For instance, `if (!list.length)` is used in favor of `if (list.length === 0)` or `if (someString)` is used in favor of `if (someString !== '')`.

The list of hits should now be visible in our application; however, two regression bugs have appeared. First, the "Dismiss" button is broken, because it doesn't know about the complex result object, but it still operates on the plain list from the sample data when dismissing an item. Second, when the list is displayed and you try to search for something else, it gets filtered on the client-side, though the initial search was made by searching for stories on the server-side. The perfect behavior would be to fetch another result object from the API when using the Search component. Both regression bugs will be fixed in the following chapters.

```
import React, { Component } from 'react';
require('./App.css');

const DEFAULT_QUERY = 'redux';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';

const isSearched = (searchTerm) => (item) =>
  item.title.toLowerCase().includes(searchTerm.toLowerCase());

class App extends Component {
```

```
  constructor(props) {
    super(props);

    this.state = {
      result: null,
      searchTerm: DEFAULT_QUERY,
    };

    this.setSearchTopstories = this.setSearchTopstories.bind(this);
    this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  setSearchTopstories(result) {
    this.setState({ result });
  }

  fetchSearchTopstories(searchTerm) {
    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}`)
      .then(response => response.json())
      .then(result => this.setSearchTopstories(result))
      .catch(e => e);
  }

  componentDidMount() {
    const { searchTerm } = this.state;
    this.fetchSearchTopstories(searchTerm);
  }

  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  onDismiss(id) {
    const isNotId = item => item.objectID !== id;
    const updatedList = this.state.list.filter(isNotId);
    this.setState({ list: updatedList });
  }

  render() {
    const { searchTerm, result } = this.state;

    if (!result) { return null; }

    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
          >
            Search
          </Search>
        </div>
        <Table
          list={result.hits}
          pattern={searchTerm}
          onDismiss={this.onDismiss}
        />
      </div>
    );
```

```
  }
}


const Search = ({ value, onChange, children }) =>
  <form>
    {children} <input
      type="text"
      value={value}
      onChange={onChange}
    />
  </form>

const Table = ({ list, pattern, onDismiss }) =>
  <div className="table">
    { list.filter(isSearched(pattern)).map(item =>
      <div key={item.objectID} className="table-row">
        <span style={{ width: '40%' }}>
          <a href={item.url}>{item.title}</a>
        </span>
        <span style={{ width: '30%' }}>
          {item.author}
        </span>
        <span style={{ width: '10%' }}>
          {item.num_comments}
        </span>
        <span style={{ width: '10%' }}>
          {item.points}
        </span>
        <span style={{ width: '10%' }}>
          <Button
            onClick={() => onDismiss(item.objectID)}
            className="button-inline"
          >
            Dismiss
          </Button>
        </span>
      </div>
    )}
  </div>

const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

export default App;
```

Further Reading:

- Read about ES6 template literals
- Read about the native fetch API
- Read about data fetching in React

ES6 Spread Operators

We will further extend our application here by modifying the onDismiss() method to tackle with the result object.

The “Dismiss” button doesn’t work because the `onDismiss()` method is not aware of the complex result object. It only knows about a plain list in the local state. But it isn’t a plain list anymore. Let’s change it to operate on the result object instead of the list itself.

```
onDismiss(id) {
  const isNotId = item => item.objectID !== id;
  const updatedHits = this.state.result.hits.filter(isNotId);
  this.setState({
    ...
  });
}
```

In `setState()`, the result is now a complex object, and the list of hits is only one of multiple properties in the object. Only the list gets updated when an item gets removed in the result object, though, while the other properties stay the same.

We could alleviate this challenge by mutating the hits in the result object. It is demonstrated below for the sake of understanding, but we’ll end up using another way.

```
// don`t do this
this.state.result.hits = updatedHits;
```

As we know, React embraces immutable data structures, so we don’t want to mutate an object (or mutate the state directly). We want to generate a new object based on the information given, so none of the objects get altered and we keep the immutable data structures. You will always return a new object, but never alter the original object.

For this, we use JavaScript ES6’s `Object.assign()`. It takes a target object as

first argument. All following arguments are source objects, which are merged

into the target object. The target object can be an empty object. It embraces immutability, because no source object gets mutated.

```
const updatedHits = { hits: updatedHits };
const updatedResult = Object.assign({}, this.state.result, updatedHits);
```

Source objects will override previously merged objects with the same property names. Let's do this on the `onDismiss()` method:

```
onDismiss(id) {
  const isNotId = item => item.objectID !== id;
  const updatedHits = this.state.result.hits.filter(isNotId);
  this.setState({
    result: Object.assign({}, this.state.result, { hits: updatedHits })
  });
}
```

That's one solution, but there is a simpler way in JavaScript ES6, called the *spread operator*. The spread operator is shown with three dots: `...` When it is used, every value from an array or object gets copied to another array or object.

We'll examine the ES6 **array** spread operator even though you don't need it yet:

```
const userList = ['Robin', 'Andrew', 'Dan'];
const additionalUser = 'Jordan';
const allUsers = [ ...userList, additionalUser ];

console.log(allUsers);
// output: ['Robin', 'Andrew', 'Dan', 'Jordan']
```

The `allUsers` variable is a completely new array. The other variables `userList` and `additionalUser` stay the same. You can even merge two arrays into a new array.

```
const oldUsers = ['Robin', 'Andrew'];
const newUsers = ['Dan', 'Jordan'];
```

```
const allUsers = [ ...oldUsers, ...newUsers ];

console.log(allUsers);
```

Now let's have a look at the object spread operator, which is not JavaScript ES6. It is part of a proposal for a next JavaScript version that is already being used by the React community, which is why *create-react-app* incorporated the feature in the configuration. The object spread operator works just like the JavaScript ES6 array spread operator, except with objects. Each key value pair is copied into a new object:

```
const userNames = { firstname: 'Robin', lastname: 'Wieruch' };
const age = 28;
const user = { ...userNames, age };

console.log(user);
// output: { firstname: 'Robin', lastname: 'Wieruch', age: 28 }
```

Multiple objects can be spread, as with the array spread example.

```
const userNames = { firstname: 'Robin', lastname: 'Wieruch' };
const userAge = { age: 28 };
const user = { ...userNames, ...userAge };

console.log(user);
// output: { firstname: 'Robin', lastname: 'Wieruch', age: 28 }
```

It can be used to replace `Object.assign()`:

```
onDismiss(id) {
  const isNotId = item => item.objectID !== id;
  const updatedHits = this.state.result.hits.filter(isNotId);
  this.setState({
    result: { ...this.state.result, hits: updatedHits }
  });
}
```

Q

In React, you can combine two objects using Spread Operator and add extra properties to that object too. Is this statement True or False?

COMPLETED 0%

1 of 1

The “Dismiss” button should work now, because the `onDismiss()` method is aware of the complex result object, and how to update it after dismissing an item from the list.

```
import React, { Component } from 'react';
require('./App.css');

const DEFAULT_QUERY = 'redux';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';

const isSearched = (searchTerm) => (item) =>
  item.title.toLowerCase().includes(searchTerm.toLowerCase());

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      result: null,
      searchTerm: DEFAULT_QUERY,
    };

    this.setSearchTopstories = this.setSearchTopstories.bind(this);
    this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  setSearchTopstories(result) {
    this.setState({ result });
  }

  fetchSearchTopstories(searchTerm) {
```

```
    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}`)
      .then(response => response.json())
      .then(result => this.setSearchTopstories(result))
      .catch(e => e);
  }

  componentDidMount() {
    const { searchTerm } = this.state;
    this.fetchSearchTopstories(searchTerm);
  }

  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  onDismiss(id) {
    const isNotId = item => item.objectID !== id;
    const updatedHits = this.state.result.hits.filter(isNotId);
    this.setState({
      result: { ...this.state.result, hits: updatedHits }
    });
  }

  render() {
    const { searchTerm, result } = this.state;

    if (!result) { return null; }

    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
          >
            Search
          </Search>
        </div>
        <Table
          list={result.hits}
          pattern={searchTerm}
          onDismiss={this.onDismiss}
        />
      </div>
    );
  }
}


const Search = ({ value, onChange, children }) =>
  <form>
    {children} <input
      type="text"
      value={value}
      onChange={onChange}
    />
  </form>

const Table = ({ list, pattern, onDismiss }) =>
  <div className="table">
    { list.filter(isSearched(pattern)).map(item =>
      <div key={item.objectID} className="table-row">
```

```
        <span style={{ width: '40%' }}>
          <a href={item.url}>{item.title}</a>
        </span>
        <span style={{ width: '30%' }}>
          {item.author}
        </span>
        <span style={{ width: '10%' }}>
          {item.num_comments}
        </span>
        <span style={{ width: '10%' }}>
          {item.points}
        </span>
        <span style={{ width: '10%' }}>
          <Button
            onClick={() => onDismiss(item.objectID)}
            className="button-inline"
          >
            Dismiss
          </Button>
        </span>
      </div>
    )}
  </div>

const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

export default App;
```

Further Reading:

- Read about the ES6 Object.assign()
- Read about the ES6 array (and object) spread operator

Conditional Rendering

This lesson will teach you how to add Conditional Rendering in your application by using ternary operator.

Conditional rendering is usually introduced early in React applications, though not in our working example. It happens when you decide to render either of two elements, which sometimes is a choice between rendering an element or nothing. The simplest usage of a conditional rendering can be expressed by an if-else statement in JSX.

The `result` object in the local component state is `null` in the beginning. So far, the App component returned no elements when the `result` hasn't arrived from the API. That's already a conditional rendering, because you return earlier from the `render()` lifecycle method for a certain condition. The App component either renders nothing or its elements.

But let's go one step further. It makes more sense to wrap the Table component, which is the only component that depends on `result` in an independent conditional rendering. Everything else should be displayed, even though there is no `result` yet. You can simply use a ternary operator in the JSX:

```
class App extends Component {

  ...

  render() {
    const { searchTerm, result } = this.state;
    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
          >
            Search
          </Search>
        </div>
        { result
          ? <Table
            list={result.hits}
```

```
            pattern={searchTerm}
            onDismiss={this.onDismiss}
          />

          : null
        }
      </div>
    );
  }
}
```

That's your second option to express a conditional rendering. A third option is the logical `&&` operator. In JavaScript a `true && 'Hello World'` always evaluates to 'Hello World'. A `false && 'Hello World'` always evaluates to false.

```
const result = true && 'Hello World';
console.log(result);
// output: Hello World

const result2 = false && 'Hello World';
console.log(result2);
// output: false
```

In React, you can make use of that behavior. If the condition is true, the expression after the logical `&&` operator will be the output. If the condition is false, React ignores the expression. It is applicable in the Table component's conditional rendering case, because it should either return a Table or nothing.

```
{ result &&
  <Table
    list={result.hits}
    pattern={searchTerm}
    onDismiss={this.onDismiss}
  />
}
```

These were a few approaches to use conditional rendering in React. You can read about more alternatives in an exhaustive list of examples. Moreover, you will get to know their different uses and when to apply them.

if you are following along on a local react setup, you should be able to see the fetched data in your application by now. Everything except the Table is displayed when data fetching is pending. Once the request resolves the result and stores it into the local state, the Table is displayed because the `render()`

method runs again, and the condition in the conditional rendering resolves in favor of displaying the Table component.

Further Readings:

- Read about different ways for conditional renderings
- Read about React conditional rendering

Client- or Server-side Search

Extending the search functionality to now search on the server side instead of client side.

Now, when you use the Search component with its input field, you will filter the list. That's happening on the client-side, though. We want to use the Hacker News API to search on the server-side. Otherwise, you would only deal with the first API response you got on `componentDidMount()`, the default search term parameter.

You can define an `onSearchSubmit()` method in your App component, which fetches results from the Hacker News API when executing a search in the Search component.

```
class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      result: null,
      searchTerm: DEFAULT_QUERY,
    };

    this.setSearchTopStories = this.setSearchTopStories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  ...

  onSearchSubmit() {
    const { searchTerm } = this.state;
  }

  ...
}
```

The `onSearchSubmit()` method should use the same functionality as the `componentDidMount()` lifecycle method, but this time with a modified search term from the local state and not with the initial default search term. Thus

you can extract the functionality as a reusable class method.

```
class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      result: null,
      searchTerm: DEFAULT_QUERY,
    };

    this.setSearchTopStories = this.setSearchTopStories.bind(this);
    this.fetchSearchTopStories = this.fetchSearchTopStories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  ...

  fetchSearchTopStories(searchTerm) {
    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}`)
      .then(response => response.json())
      .then(result => this.setSearchTopStories(result))
      .catch(error => error);
  }

  componentDidMount() {
    const { searchTerm } = this.state;
    this.fetchSearchTopStories(searchTerm);
  }

  ...

  onSearchSubmit() {
    const { searchTerm } = this.state;
    this.fetchSearchTopStories(searchTerm);
  }

  ...
}
```

Now the Search component has to add an additional button to trigger the search request explicitly. Otherwise, we'd fetch data from the Hacker News API every time the input field changes. We want to do it explicitly in a `onClick()` handler.

As an alternative, you could debounce (delay) the `onChange()` function and spare the button, but it would add more complexity, and we want to keep things simple for now.

First, pass the `onSearchSubmit()` method to your Search component.

```
class App extends Component {

  ...

  render() {
    const { searchTerm, result } = this.state;
    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
          >
            Search
          </Search>
        </div>
        { result &&
          <Table
            list={result.hits}
            pattern={searchTerm}
            onDismiss={this.onDismiss}
          />
        }
      </div>
    );
  }
}
```

Second, introduce a button in your Search component. The button has the `type="submit"` and the form uses its `onSubmit` attribute to pass the `onSubmit()` method. You can reuse the `children` property, but this time it will be used as the content of the button.

```
const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
    />
    <button type="submit">
      {children}
    </button>
  </form>
```

In the Table, you can remove the filter functionality, because there will be no client-side filter (search) anymore. Don't forget to remove the `isSearched()`

function as well, as it won't be used anymore. Now, the result comes directly from the Hacker News API when the user clicks the "Search" button.

```
class App extends Component {

  ...

  render() {
    const { searchTerm, result } = this.state;
    return (
      <div className="page">
        ...
        { result &&
          <Table
            list={result.hits}
            onDismiss={this.onDismiss}
          />
        }
      </div>
    );
  }
}

...

const Table = ({ list, onDismiss }) =>
  <div className="table">
    {list.map(item =>
      ...
    )}
  </div>
```

Now when you try to search, you will notice the browser reloads. That's a native browser behavior for a submit callback in an HTML form. In React, you will often come across the `preventDefault()` event method to suppress the native browser behavior.

```
onSearchSubmit(event) {
  const { searchTerm } = this.state;
  this.fetchSearchTopStories(searchTerm);
  event.preventDefault();
}
```

You should be able to search different Hacker News stories now. We have interacted with a real API, and there should be no more client-side searches.

```
import React, { Component } from 'react';
require('./App.css');

const DEFAULT_QUERY = 'redux';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
```

```
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      result: null,
      searchTerm: DEFAULT_QUERY,
    };

    this.setSearchTopstories = this.setSearchTopstories.bind(this);
    this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  setSearchTopstories(result) {
    this.setState({ result });
  }

  fetchSearchTopstories(searchTerm) {
    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}`)
      .then(response => response.json())
      .then(result => this.setSearchTopstories(result))
        .catch(e => e);
  }

  componentDidMount() {
    const { searchTerm } = this.state;
    this.fetchSearchTopstories(searchTerm);
  }

  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  onSearchSubmit(event) {
    const { searchTerm } = this.state;
    this.fetchSearchTopstories(searchTerm);
    event.preventDefault();
  }

  onDismiss(id) {
    const isNotId = item => item.objectID !== id;
    const updatedHits = this.state.result.hits.filter(isNotId);
    this.setState({
      result: { ...this.state.result, hits: updatedHits }
    });
  }

  render() {
    const { searchTerm, result } = this.state;
    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
```

```
          >
            Search
          </Search>
        </div>
        { result &&
          <Table
            list={result.hits}
            onDismiss={this.onDismiss}
          />
        }
      </div>
    );
  }
}


const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
    />
    <button type="submit">
      {children}
    </button>
  </form>

const Table = ({ list, onDismiss }) =>
  <div className="table">
    { list.map(item =>
      <div key={item.objectID} className="table-row">
        <span style={{ width: '40%' }}>
          <a href={item.url}>{item.title}</a>
        </span>
        <span style={{ width: '30%' }}>
          {item.author}
        </span>
        <span style={{ width: '10%' }}>
          {item.num_comments}
        </span>
        <span style={{ width: '10%' }}>
          {item.points}
        </span>
        <span style={{ width: '10%' }}>
          <Button
            onClick={() => onDismiss(item.objectID)}
            className="button-inline"
          >
            Dismiss
          </Button>
        </span>
      </div>
    )}
  </div>

const Button = ({ onClick, className = '', children }) =>
```

```
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

export default App;
```

Exercises:

- Experiment with the Hacker News API

Further Reading:

- Read about synthetic events in React

Paginated Fetch

Let's extend the composable API constants so it can deal with paginated data!

Take a closer look at the data structure and observe how the Hacker News API returns more than a list of hits. Precisely, it returns a paginated list. The page property, which is `0` in the first response, can be used to fetch more paginated sublists as results. You only need to pass the next page with the same search term to the API.

We'll start with:

```
const DEFAULT_QUERY = 'redux';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';
```

Now you can use the new constant to add the page parameter to your API request:

```
const DEFAULT_QUERY = 'redux';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';

const url = `${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${DEFAULT_QUERY}&${PARAM_PAGE}`;

console.log(url);
// output: https://hn.algolia.com/api/v1/search?query=redux&page=
```

The `fetchSearchTopStories()` method will take the page as second argument. If you don't provide the second argument, it will fallback to the `0` page for the initial request. Thus the `componentDidMount()` and `onSearchSubmit()` methods

initial request. Thus the `componentDidMount()` and `onSearchSubmit()` methods

fetch the first page on the first request. Every additional fetch should fetch the next page by providing the second argument.

```
class App extends Component {

  ...

  fetchSearchTopStories(searchTerm, page = 0) {
    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}`)
      .then(response => response.json())
      .then(result => this.setSearchTopStories(result))
      .catch(error => error);
  }

  ...

}
```

The page argument uses the JavaScript ES6 default parameter to introduce the fallback to page `0` in case no defined page argument is provided for the function.

Now you can use the current page from the API response in `fetchSearchTopStories()`. You can use this method in a button to fetch more stories with an `onClick` button handler. Let's use the Button to fetch more paginated data from the Hacker News API. For this, we'll define the `onClick()` handler, which takes the current search term and the next page (current page + 1).

```
class App extends Component {

  ...

  render() {
    const { searchTerm, result } = this.state;
    const page = (result && result.page) || 0;
    return (
      <div className="page">
        <div className="interactions">
        ...
        { result &&
          <Table
            list={result.hits}
            onDismiss={this.onDismiss}
          />
        }
        <div className="interactions">
          <Button onClick={() => this.fetchSearchTopStories(searchTerm, page + 1)}>
```

```
            More
          </Button>
        </div>

      </div>
    );
  }
}
```

In your `render()` method, make sure to default to page 0 when there is no result. Remember, the `render()` method is called before the data is fetched asynchronously in the `componentDidMount()` lifecycle method.

There is still one step missing, because fetching the next page of data will override your previous page of data. We want to concatenate the old and new list of hits from the local state and new result object, so we'll adjust its functionality to add new data rather than override it.

```
setSearchTopStories(result) {
  const { hits, page } = result;

  const oldHits = page !== 0
    ? this.state.result.hits
    : [];

  const updatedHits = [
    ...oldHits,
    ...hits
  ];

  this.setState({
    result: { hits: updatedHits, page }
  });
}
```

A couple things happen in the `setSearchTopStories()` method now. First, you get the hits and page from the result.

Second, you have to check if there are already old hits. When the page is 0, it is a new search request from `componentDidMount()` or `onSearchSubmit()`. The hits are empty. But when you click the "More" button to fetch paginated data the page isn't 0. The old hits are already stored in your state and thus can be used.

Third, you don't want to override the old hits. You can merge old and new hits from the recent API request, which can be done with a JavaScript ES6 array spread operator.

Fourth, you set the merged hits and page in the local component state.

Now we'll make one last adjustment. When you try the "More" button it only fetches a few list items. The API URL can be extended to fetch more list items with each request, so we add even more composable path constants:

```
const DEFAULT_QUERY = 'redux';
const DEFAULT_HPP = '100';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';
const PARAM_HPP = 'hitsPerPage=';
```

Now you can use the constants to extend the API URL.

```
fetchSearchTopStories(searchTerm, page = 0) {
  fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}&${PARAM
    .then(response => response.json())
    .then(result => this.setSearchTopStories(result))
    .catch(error => error);
}
```

The request to the Hacker News API fetches more list items in one request than before. As you can see, a powerful API such as the Hacker News API gives plenty of ways to experiment with real world data. You should make use of it to make your endeavours when learning something new more exciting. That's how I learned about the empowerment that APIs provide when learning a new programming language or library.

Here's the code so far:

```
import React, { Component } from 'react';
require('./App.css');

const DEFAULT_QUERY = 'redux';
const DEFAULT_HPP = '100';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';
```

```
class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      result: null,
      searchTerm: DEFAULT_QUERY,
    };

    this.setSearchTopstories = this.setSearchTopstories.bind(this);
    this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  setSearchTopstories(result) {
    const { hits, page } = result;

    const oldHits = page !== 0
      ? this.state.result.hits
      : [];

    const updatedHits = [
      ...oldHits,
      ...hits
    ];

    this.setState({
      result: { hits: updatedHits, page }
    });
  }

  fetchSearchTopstories(searchTerm, page = 0) {
    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}`)
      .then(response => response.json())
      .then(result => this.setSearchTopstories(result))
        .catch(e => e);
  }

  componentDidMount() {
    const { searchTerm } = this.state;
    this.fetchSearchTopstories(searchTerm);
  }

  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  onSearchSubmit(event) {
    const { searchTerm } = this.state;
    this.fetchSearchTopstories(searchTerm);
    event.preventDefault();
  }

  onDismiss(id) {
    const isNotId = item => item.objectID !== id;
    const updatedHits = this.state.result.hits.filter(isNotId);
    this.setState({
      result: { ...this.state.result, hits: updatedHits }
    });
```

```
  }

  render() {
    const { searchTerm, result } = this.state;
    const page = (result && result.page) || 0;
    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
          >
            Search
          </Search>
        </div>
        { result &&
          <Table
            list={result.hits}
            onDismiss={this.onDismiss}
          />
        }
        <div className="interactions">
          <Button onClick={() => this.fetchSearchTopstories(searchTerm, page + 1)}>
            More
          </Button>
        </div>
      </div>
    );
  }
}


const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
    />
    <button type="submit">
      {children}
    </button>
  </form>

const Table = ({ list, onDismiss }) =>
  <div className="table">
    { list.map(item =>
      <div key={item.objectID} className="table-row">
        <span style={{ width: '40%' }}>
          <a href={item.url}>{item.title}</a>
        </span>
        <span style={{ width: '30%' }}>
          {item.author}
        </span>
        <span style={{ width: '10%' }}>
          {item.num_comments}
```

```
        </span>
        <span style={{ width: '10%' }}>
          {item.points}
        </span>
        <span style={{ width: '10%' }}>
          <Button
            onClick={() => onDismiss(item.objectID)}
            className="button-inline"
          >
            Dismiss
          </Button>
        </span>
      </div>
    )}
  </div>

const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

export default App;
```

Exercise:

- Experiment with the Hacker News API parameters

Further Reading:

- Read about ES6 default parameters

Client Cache

This lesson will teach you how to optimize your search functionality by using the client-side cache.

Each search submit makes a request to the Hacker News API. You might search for “redux”, followed by “react” and eventually “redux” again. In total it makes 3 requests. But you searched for “redux” twice and both times it took a whole asynchronous roundtrip to fetch the data. In a client-side cache, you would store each result. When a request to the API is made, it checks if a result is already there and uses the cache if it is. Otherwise, an API request is made to fetch the data.

To have a client cache for each result, you have to store multiple `results` rather than one `result` in your local component state. The results object will be a map with the search term as key and the result as value. Each result from the API will be saved by the search term (key).

At the moment, your result in the local state looks like this:

```
result: {
  hits: [ ... ],
  page: 2,
}
```

Imagine you have made two API requests. One for the search term “redux” and another one for “react”. The results object should look like the following:

```
results: {
  redux: {
    hits: [ ... ],
    page: 2,
  },
  react: {
    hits: [ ... ],
    page: 1,
  },
  ...
}
```

Let's implement a client-side cache with React `setState()`. First, rename the `result` object to `results` in the initial component state. Second, define a temporary `searchKey` to store each `result`.

```
class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      results: null,
      searchKey: '',
      searchTerm: DEFAULT_QUERY,
    };

    ...

  }

  ...

}
```

The `searchKey` has to be set before each request is made. It reflects the `searchTerm`. Understanding why we don't use `searchTerm` here is a crucial part to understand before continuing with the implementation. The `searchTerm` is a fluctuant variable, because it gets changed every time you type into the search input field. We want a non fluctuant variable that determines the recent submitted search term to the API and can be used to retrieve the correct result from the map of results. It is a pointer to your current result in the cache, which can be used to display the current result in the `render()` method.

```
componentDidMount() {
  const { searchTerm } = this.state;
  this.setState({ searchKey: searchTerm });
  this.fetchSearchTopStories(searchTerm);
}

onSearchSubmit(event) {
  const { searchTerm } = this.state;
  this.setState({ searchKey: searchTerm });
  this.fetchSearchTopStories(searchTerm);
  event.preventDefault();
}
```

Now we will change where the result is stored to the local component state. It should store each result by `searchKey`.

```
class App extends Component {

  ...

  setSearchTopStories(result) {
    const { hits, page } = result;
    const { searchKey, results } = this.state;

    const oldHits = results && results[searchKey]
      ? results[searchKey].hits
      : [];

    const updatedHits = [
      ...oldHits,
      ...hits
    ];

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      }
    });
  }

  ...

}
```

The `searchKey` will be used as the key to save the updated hits and page in a `results` map.

First, you have to retrieve the `searchKey` from the component state. Remember that the `searchKey` gets set on `componentDidMount()` and `onSearchSubmit()`.

Second, the old hits have to get merged with the new hits as before. But this time the old hits get retrieved from the `results` map with the `searchKey` as key.

Third, a new result can be set in the `results` map in the state. Let's examine the `results` object in `setState()`.

```
results: {
  ...results,
  [searchKey]: { hits: updatedHits, page }
}
```

The bottom part makes sure to store the updated result by `searchKey` in the results map. The value is an object with a hits and page property. The `searchKey` is the search term. You already learned the `[searchKey]: ...` syntax. It is an ES6 computed property name. It helps you allocate values dynamically in an object.

The upper part needs to spread all other results by `searchKey` in the state using the object spread operator. Otherwise, you would lose all the results that you have stored before.

Now you store all results by search term. That's the first step to enable your cache. In the next step, you can retrieve the result depending on the non fluctuant `searchKey` from your map of results. That's why you had to introduce the `searchKey` in the first place as non fluctuant variable. Otherwise, the retrieval would be broken when you would use the fluctuant `searchTerm` to retrieve the current result because this value might change when we use the Search component.

```
class App extends Component {

  ...

  render() {
    const {
      searchTerm,
      results,
      searchKey
    } = this.state;

    const page = (
      results &&
      results[searchKey] &&
      results[searchKey].page
    ) || 0;

    const list = (
      results &&
      results[searchKey] &&
      results[searchKey].hits
    ) || [];

    return (
      <div className="page">
        <div className="interactions">
          ...
        </div>
        <Table
          list={list}
          onDismiss={this.onDismiss}
        />
        <div className="interactions">
```

```
          <Button onClick={() => this.fetchSearchTopStories(searchKey, page + 1)}>
            More
          </Button>

        </div>
      </div>
    );
  }
}
```

Since you default to an empty list when there is no result by `searchKey`, you can spare the conditional rendering for the Table component. Additionally, we need to pass the `searchKey` rather than the `searchTerm` to the “More” button. Otherwise, your paginated fetch depends on the `searchTerm` value which is fluctuant. Moreover, make sure to keep the fluctuant `searchTerm` property for the input field in the Search component.

The search functionality should store all results from the Hacker News API now.

Now, we can see the `onDismiss()` method needs improvement, as it still deals with the `result` object. We want it deal with multiple `results`:

```
  onDismiss(id) {
    const { searchKey, results } = this.state;
    const { hits, page } = results[searchKey];

    const isNotId = item => item.objectID !== id;
    const updatedHits = hits.filter(isNotId);

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      }
    });
  }
```

Check to make sure the “Dismiss” button is working again.

Note that nothing will stop the application from sending an API request on each search submit. Though there might be already a result, there is no check that prevents the request, so the cache functionality is not complete yet. It caches the results, but it doesn’t make use of them. The last step is to prevent the API request when a result is available in the cache:

```
class App extends Component {
```

```
  constructor(props) {

    ...

    this.needsToSearchTopStories = this.needsToSearchTopStories.bind(this);
    this.setSearchTopStories = this.setSearchTopStories.bind(this);
    this.fetchSearchTopStories = this.fetchSearchTopStories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  needsToSearchTopStories(searchTerm) {
    return !this.state.results[searchTerm];
  }

  ...

  onSearchSubmit(event) {
    const { searchTerm } = this.state;
    this.setState({ searchKey: searchTerm });

    if (this.needsToSearchTopStories(searchTerm)) {
      this.fetchSearchTopStories(searchTerm);
    }

    event.preventDefault();
  }

  ...

}
```

Now your client only makes a request to the API once, though you searched for a term twice. Even paginated data with several pages gets cached that way, because you always save the last page for each result in the `results` map. This is a powerful approach to introduce caching into an application. The Hacker News API provides you with everything you need to cache paginated data effectively.

Error Handling

Let's catch some errors in your application and debug them!

Now we've taken care of interactions with the Hacker News API. We've introduced an elegant way to cache results from the API and make use of its paginated list functionality to fetch an endless list of sublists of stories from the API. But no application is complete without error handling.

In this chapter, we introduce an efficient solution to add error handling for the application that we are building in case of an erroneous API request. We have learned the necessary building block to introduce error handling in React: local state and conditional rendering. The error is just another state, which we store in the local state and display with conditional rendering in the component.

Now, we'll implement it in the App component, since that's where we fetch data from the Hacker News API. First, introduce the error in the local state. It is initialized as null but will be set to the error object in case of an error.

```
class App extends Component {
  constructor(props) {
    super(props);

    this.state = {
      results: null,
      searchKey: '',
      searchTerm: DEFAULT_QUERY,
      error: null,
    };

    ...
  }

...

}
```

Second, use the catch block in your native fetch to store the error object in the local state by using `setState()`. Every time the API request isn't successful, the

catch block is executed.

```
class App extends Component {

  ...

  fetchSearchTopStories(searchTerm, page = 0) {
    // https://github.com/the-road-to-learn-react/the-road-to-learn-react/issues/43
    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}&${PAR
      .then(response => response.json())
      .then(result => this.setSearchTopStories(result))
      .catch(error => this.setState({ error }));
  }

  ...

}
```

Third, retrieve the error object from your local state in the `render()` method, and display a message in case of an error using React's conditional rendering.

```
class App extends Component {

  ...

  render() {
    const {
      searchTerm,
      results,
      searchKey,
      error
    } = this.state;

    ...

    if (error) {
      return <p>Something went wrong.</p>;
    }

    return (
      <div className="page">
        ...
      </div>
    );
  }
}
```

If you want to test that your error handling is working, change the API URL to something non-existent to force an error.

```
const PATH_BASE = 'https://hn.mydomain.com/api/v1';
```

Now you should see an error message instead of your application. It's up to you where you want to place the conditional rendering for the error message. In this case, the app is overridden by the error message, but that wouldn't be the best user experience. The remaining application would still be visible so the user knows it's still running:

```
class App extends Component {

  ...

  render() {
    const {
      searchTerm,
      results,
      searchKey,
      error
    } = this.state;

    const page = (
      results &&
      results[searchKey] &&
      results[searchKey].page
    ) || 0;

    const list = (
      results &&
      results[searchKey] &&
      results[searchKey].hits
    ) || [];

    return (
      <div className="page">
        <div className="interactions">
          ...
        </div>
        { error
          ? <div className="interactions">
            <p>Something went wrong.</p>
          </div>
          : <Table
            list={list}
            onDismiss={this.onDismiss}
          />
        }
        ...
      </div>
    );
  }
}
```

Remember to revert the URL for the API to the existent one:

```
const PATH_BASE = 'https://hn.algolia.com/api/v1';
```

Our application now has error handling in case the API request fails.

Further Reading:

- Read about React's Error Handling for Components

Axios instead of Fetch

In this lesson, you will learn how to shift from Fetch API to other APIs like Axios in your React app.

Before, we introduced the native fetch API to perform a request to the Hacker News platform. The browser allows you to use this native fetch API. However, not all browsers support this, older browsers especially. Once you start testing your application in a headless browser environment (where there is no browser, it is mocked), there can be issues with the fetch API. There are a couple of ways to make fetch work in older browsers (polyfills) and in tests (isomorphic-fetch), but these concepts are bit off-task for the purpose of learning React.

One alternative is to substitute the native fetch API with a stable library such as axios, which performs asynchronous requests to remote APIs. In this chapter, we will discover how to substitute a library, a native API of the browser in this case, with another library.

Below, the native fetch API is substituted with axios. First, we install axios on the command line:

```
npm install axios
```

Second, import axios in your App component's file:

```
import React, { Component } from 'react';
import axios from 'axios';
import './App.css';

...
```

You can use axios instead of `fetch()`, and its usage looks almost identical to the native fetch API. It takes the URL as argument and returns a promise. You don't have to transform the returned response to JSON anymore, since axios wraps the result into a `data` object in JavaScript. Just make sure to adapt your

code to the returned data structure:

```
class App extends Component {

  ...

  fetchSearchTopStories(searchTerm, page = 0) {
    axios(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}&${PAR
      .then(result => this.setSearchTopStories(result.data))
      .catch(error => this.setState({ error }));
  }

  ...

}
```

In this code, we call `axios()`, which uses an HTTP GET request by default. You can make the GET request explicit by calling `axios.get()`, or you can use another HTTP method such as HTTP POST with `axios.post()`. With these examples alone, we can see that axios is a powerful library to perform requests to remote APIs. I recommend you use it instead of the native fetch API when requests become complex, or you have to deal with promises.

Now we'll introduce another improvement for the Hacker News request in the App component. Imagine the component mounts when the page is rendered for the first time in the browser. In `componentDidMount()` the component starts to make the request, but then the user navigates away from the page with this rendered component. Then the App component unmounts, but there is still a pending request from `componentDidMount()` lifecycle method. It will attempt to use `this.setState()` eventually in the `then()` or `catch()` block of the promise. You will likely see the following warning on your command line, or in your browser's developer output: *Warning: Can only update a mounted or mounting component. This usually means you called setState, replaceState, or forceUpdate on an unmounted component. This is a no-op.*

You can handle this issue by aborting the request when your component unmounts or preventing `this.setState()` on an unmounted component. It is considered a best practice in React to preserve a clean application without warnings. However, the current promise API doesn't implement aborting a request, so we add a workaround, introducing a class field that holds the lifecycle state of your component. It can be initialized as `false` when the component initializes, changed to `true` when the component mounted, and then reset to `false` when the component unmounted. This way, you can keep

then reset to `false` when the component unmounted. This way, you can keep track of your component's lifecycle state. It doesn't affect the local state stored

and modified with `this.state` and `this.setState()`, because you can access it directly on the component instance without relying on React's local state management. Moreover, it doesn't lead to any re-rendering of the component when the class field is changed.

```
class App extends Component {

  _isMounted = false;
  constructor(props) {
    ...
  }

  ...

  componentDidMount() {
    this._isMounted = true;

    const { searchTerm } = this.state;
    this.setState({ searchKey: searchTerm });
    this.fetchSearchTopStories(searchTerm);
  }

  componentWillUnmount() {
    this._isMounted = false;
  }

  ...

}
```

Finally, you can use this knowledge not to abort the request itself, but avoid calling `this.setState()` on your component instance, even though the component already unmounted. It will prevent the warning.

```
class App extends Component {

  ...

  fetchSearchTopStories(searchTerm, page = 0) {
    axios(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}&${PAR
      .then(result => this._isMounted && this.setSearchTopStories(result.data))
      .catch(error => this._isMounted && this.setState({ error }));
  }

  ...

}
```

This chapter has shown you how you can replace one library with another in

React. If you run into any issues, you can use the vast library ecosystem in JavaScript to find a solution. Also, we've learned how to avoid calling `this.setState()` in React on an unmounted component. If you dig deeper into the axios library, you will find a way to cancel the request beforehand.

Further Reading:

- Read about why frameworks matter

Recap

Now you've learned to interact with an API in React! This lesson will provide you a recap to what you have learned so far and will also share how your App.js code looks by now.

Let's recap the last chapter:

- **React**
  - ES6 class component lifecycle methods for different use cases
  - `componentDidMount()` for API interactions
  - Conditional renderings
  - Synthetic events on forms
  - Error handling
  - Aborting a remote API request
- **ES6 and beyond**
  - Template strings to compose strings
  - Spread operator for immutable data structures
  - Computed property names
  - Class fields
- **General**
  - Hacker News API interaction
  - Native fetch browser API
  - Client and server-side search
  - Pagination of data
  - Client-side caching
  - Axios as an alternative for the native fetch API

Again, it makes sense to take a break, internalize the lessons and apply them on your own. Experiment with the parameters for the API endpoint to query

different results. You can find the source code in the official repository.

Your `src/App.js` should look like the following by now:

```
import React, { Component } from 'react';
require('./App.css');

const DEFAULT_QUERY = 'redux';
const DEFAULT_HPP = '100';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';
const PARAM_HPP = 'hitsPerPage=';

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      results: null,
      searchKey: '',
      searchTerm: DEFAULT_QUERY,
      error: null
    };

    this.needsToSearchTopstories = this.needsToSearchTopstories.bind(this);
    this.setSearchTopstories = this.setSearchTopstories.bind(this);
    this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  componentDidMount() {
    const { searchTerm } = this.state;
    this.setState({ searchKey: searchTerm });
    this.fetchSearchTopstories(searchTerm);
  }

  setSearchTopstories(result) {
    const { hits, page } = result;
    const { searchKey, results } = this.state;

    const oldHits = results && results[searchKey]
      ? results[searchKey].hits
      : [];

    const updatedHits = [
      ...oldHits,
      ...hits
    ];

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
```

```
    }
  });
}

fetchSearchTopstories(searchTerm, page = 0) {
  fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}&${PAR
    .then(response => response.json())
    .then(result => this.setSearchTopstories(result))
      .catch(e => this.setState({ error: e }));
}

needsToSearchTopstories(searchTerm) {
  return !this.state.results[searchTerm];
}

onSearchChange(event) {
  this.setState({ searchTerm: event.target.value });
}

onSearchSubmit(event) {
  const { searchTerm } = this.state;
  this.setState({ searchKey: searchTerm });

  if (this.needsToSearchTopstories(searchTerm)) {
    this.fetchSearchTopstories(searchTerm);
  }

  event.preventDefault();
}

onDismiss(id) {
  const { searchKey, results } = this.state;
  const { hits, page } = results[searchKey];

  const isNotId = item => item.objectID !== id;
  const updatedHits = hits.filter(isNotId);

  this.setState({
    results: {
      ...results,
      [searchKey]: { hits: updatedHits, page }
    }
  });
}

render() {
  const {
    searchTerm,
    results,
    searchKey,
    error
  } = this.state;

  const page = (
    results &&
    results[searchKey] &&
    results[searchKey].page
  ) || 0;

  const list = (
    results &&
    results[searchKey] &&
```

```
      results[searchKey].hits
    ) || [];

    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
          >
            Search
          </Search>
        </div>
        { error
          ? <div className="interactions">
            <p>Something went wrong.</p>
          </div>
          : <Table
            list={list}
            onDismiss={this.onDismiss}
          />
        }
        <div className="interactions">
          <Button onClick={() => this.fetchSearchTopstories(searchKey, page + 1)}>
            More
          </Button>
        </div>
      </div>
    );
  }
}

const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
    />
    <button type="submit">
      {children}
    </button>
  </form>

const Table = ({ list, onDismiss }) =>
  <div className="table">
    { list.map(item =>
      <div key={item.objectID} className="table-row">
        <span style={{ width: '40%' }}>
          <a href={item.url}>{item.title}</a>
        </span>
        <span style={{ width: '30%' }}>
          {item.author}
        </span>
        <span style={{ width: '10%' }}>
          {item.num_comments}
```

```
        </span>
        <span style={{ width: '10%' }}>
          {item.points}
        </span>
        <span style={{ width: '10%' }}>
          <Button
            onClick={() => onDismiss(item.objectID)}
            className="button-inline"
          >
            Dismiss
          </Button>
        </span>
      </div>
    )}
  </div>

const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

export default App;
```

ES6 Modules: Import and Export

In this lesson, we cover all the different ways to import and export functionaliities from modules in ES6.

In JavaScript ES6, you can import and export functionalities from modules. These can be functions, classes, components, constants, essentially anything you can assign to a variable. Modules can be single files or whole folders with one index file as entry point.

After we bootstrapped our application with *create-react-app* at the beginning, we already used several `import` and `export` statements in the initial files. The `import` and `export` statements help you to share code across multiple files. Historically there were already several solutions for this in the JavaScript environment, but it was a mess because there wasn't a standardized method of performing this task. JavaScript ES6 added it as a native behavior eventually.

These statements embrace code splitting, where we distribute code across multiple files to keep it reusable and maintainable. The former is true because we can import a piece of code into multiple files. The latter is true because there is only one source where you maintain the piece of code.

We also want to think about code encapsulation, since not every functionality needs to be exported from a file. Some of these functionalities should only be used in files where they have been defined. File exports are basically a public API to a file, where only the exported functionalities are available to be reused elsewhere. This follows the best practice of encapsulation.

The following examples showcase the statements by sharing one or multiple variables across two files. In the end, the approach can scale to multiple files and could share more than simple variables.

The act of exporting one or multiple variables is called a named export:

```
// file1
```

```
const firstname = 'Robin';
const lastname = 'Wieruch';


export { firstname, lastname };
```

And import them in another file with a relative path to the first file.

index.js

file1.js

```
// file2.js
import { firstname, lastname } from './file1.js';

console.log(firstname);
// output: robin
```

You can also import all exported variables from another file as one object.

index.js

file1.js

```
// file2.js
import * as person from './file1.js';

console.log(person.firstname);
// output: robin
```

Imports can have aliases, which are necessary when we import functionalities from multiple files that have the same named export.

index.js

file1.js

```
// file2.js
import { firstname as username } from './file1.js';

console.log(username);
// output: Robin
```

There is also the `default` statement, which can be used for a few cases:

- to export and import a single functionality
- to highlight the main functionality of the exported API of a module

- to have a fallback import functionality

```
// file1.js
const robin = {
  firstname: 'robin',
  lastname: 'wieruch',
};

export default robin;
```

You have to leave out the curly braces to import the default export.

index.js

file1.js

```
// file2.js
import developer from './file1.js';

console.log(developer);
// output: { firstname: 'robin', lastname: 'wieruch' }
```

The import name can differ from the exported default name, and it can be used with the named export and import statements:

```
// file1.js
const firstname = 'robin';
const lastname = 'wieruch';

const person = {
  firstname,
  lastname,
};

export {
  firstname,
  lastname,
};

export default person;
```

Import the default or the named exports in another file:

index.js

file1.js

```
// file2.js
```

```
import developer, { firstname, lastname } from './file1.js';

console.log(developer);

// output: { firstname: 'robin', lastname: 'wieruch' }
console.log(firstname, lastname);
// output: robin wieruch
```

You can spare the extra lines, and export the variables directly for named exports as in `file1.js`.

index.js

file1.js

```
import { firstname, lastname } from './file1.js';

console.log(firstname);
console.log(lastname);
```

These are the main functionalities for ES6 modules. They help you to organize your code, to maintain it, and to design reusable module APIs. You can also export and import functionalities to test them which you will do in a later chapter.

Further Reading:

- Read about ES6 import
- Read about ES6 export

Code Organization with ES6 Modules

In this lesson, we'll discuss the best practices of React specific code organization on a local setup for a large React application i.e., the most efficient way to put components into modules (files) and modules into folders.

File structure: Best Practices

You might wonder why we didn't follow the best practices of putting components into different files for the *App.js* file. We already have multiple components in the file that can be defined in their own files/folders (modules). For the sake of learning React, it is practical to keep these items in one place. Once your React application grows, you should consider splitting these components into multiple modules so it can scale properly.

In the following, I propose several module structures you can apply. You can apply them as an exercise later, but we will continue developing our app with the *App.js* file to keep things simple. Most React setups have a *src* folder though which contains the *App.js* file.

Module Structure #1: Naming by component

Here is one possible module structure:

```
src/
  index.js
  index.css
  App.js
  App.test.js
  App.css
  Button.js
  Button.test.js
  Button.css
  Table.js
  Table.test.js
  Table.css
  Search.js
  Search.test.js
  Search.css
```

This one separates the components into their own files, but it doesn't look too

promising. You can see a lot of naming duplications, and only the file extension differs.

Module Structure #2: Categorizing by Component

Another module structure could be:

```
src/
  index.js
  index.css
  App/
    index.js
    test.js
    index.css
  Button/
    index.js
    test.js
    index.css
  Table/
    index.js
    test.js
    index.css
  Search/
    index.js
    test.js
    index.css
```

Now it looks cleaner than before. The index naming of a file describes it as an entry point file to the folder. It is just a common naming convention, but you can use your own naming as well. In this module structure, a component is defined by its component declaration in the JavaScript file, but also by its style and tests. If you have trouble finding your components this way while searching for them in your editor/IDE, search for paths rather than files (e.g. search for “Table index”).

Another step is extracting the constant variables from the App component, which were used to compose the Hacker News API URL.

```
src/
  index.js
  index.css
  constants/
    index.js
  components/
    App/
      index.js
      test.js
      index.css
    Button/
      index.js
      test.js
```

```
      index.css
    ...
```

Naturally, the modules would split up into *src/constants/* and *src/components/*. The *src/constants/index.js* file could look like the following:

```
export const DEFAULT_QUERY = 'redux';
export const DEFAULT_HPP = '100';
export const PATH_BASE = 'https://hn.algolia.com/api/v1';
export const PATH_SEARCH = '/search';
export const PARAM_SEARCH = 'query=';
export const PARAM_PAGE = 'page=';
export const PARAM_HPP = 'hitsPerPage=';
```

The *App/index.js* file could import these variables in order to use them.

```
import {
  DEFAULT_QUERY,
  DEFAULT_HPP,
  PATH_BASE,
  PATH_SEARCH,
  PARAM_SEARCH,
  PARAM_PAGE,
  PARAM_HPP,
} from '../../constants/index.js';

...
```

When you use the *index.js* naming convention, you can omit the filename from the relative path.

```
import {
  DEFAULT_QUERY,
  DEFAULT_HPP,
  PATH_BASE,
  PATH_SEARCH,
  PARAM_SEARCH,
  PARAM_PAGE,
  PARAM_HPP,
} from '../../constants';

...
```

The *index.js* file naming is convention was introduced in the Node.js world, where the index file is the entry point to a module. It describes the public API to the module. External modules are only allowed to use the *index.js* file to

import shared code from the module. Consider the following module structure to demonstrate it:

```
src/
  index.js
  App/
    index.js
  Buttons/
    index.js
    SubmitButton.js
    SaveButton.js
    CancelButton.js
```

The *Buttons/* folder has multiple button components defined in its distinct files. Each file can `export default` the specific component, making it available to *Buttons/index.js*. The *Buttons/index.js* file imports all different button representations and exports them as public module API.

```
import SubmitButton from './SubmitButton';
import SaveButton from './SaveButton';
import CancelButton from './CancelButton';

export {
  SubmitButton,
  SaveButton,
  CancelButton,
};
```

Now the *src/App/index.js* can import the buttons from the public module API located in the *index.js* file.

```
import {
  SubmitButton,
  SaveButton,
  CancelButton
} from '../Buttons';
```

However, it can be seen as bad practice to reach into other files than the *index.js* in the module, as it breaks the rules of encapsulation.

```
// bad practice, don't do it
import SubmitButton from '../Buttons/SubmitButton';
```

We refactored the source code in a module with the constraints of encapsulation. Again, we'll keep things simple in this course, but you should always refactor your source code to keep it clean.

Exercises

- Refactor your *src/App.js* file into multiple component modules when you finished the course on a local React setup

Snapshot Tests with Jest

In the next couple of lessons, we'll cover testing, an important practice to keep source code robust. In this lesson, we'll brush upon the basics of software testing and then learn the basics of testing snapshot testing React components with Jest - Facebook's JavaScript testing library.

Software Testing

Testing source code is essential to programming, and should be seen as mandatory. We want to keep the quality of your code high and make sure everything works before using it in production. We will use the testing pyramid as our guide.

The testing pyramid includes end-to-end tests, integration tests, and unit tests. A unit test is for an isolated and small block of code, such a single function. However, sometimes units work well in isolation but not in combination with other units, so they need to be tested as a group. That's where integration tests can help us figure out if units work well together. An end-to-end test is a simulation of a real-life scenario, such as the automated setup of a browser simulating the login flow in a web application. Where unit tests are fast and easy to write and maintain, end-to-end tests are at the opposite of the spectrum.

We want to have many unit tests covering the isolated functions. After that, we can use several integration tests to make sure the most important functions work in combination as expected. Finally, we may need a few end-to-end tests to simulate critical scenarios.

Testing React

The foundation for testing in React are component tests, generalized partly as unit tests and partly as snapshot tests. Unit tests for our components will be covered in the next chapter with a library called Enzyme. In this section, we focus on snapshot tests, using Jest.

Introduction to Jest

Jest is a JavaScript testing framework used by Facebook. It is used for component tests by the React community. Fortunately, *create-react-app* already comes with Jest, so you don't need to set it up.

Jest Setup

Before you can test your first components, you have to export them from your *App.js* file. Afterward, you can test them in a different file.

```
...

class App extends Component {
  ...
}

...

export default App;

export {
  Button,
  Search,
  Table,
};
```

In your *App.test.js* file, you will find the first test that came with *create-react-app*. It verifies the App component will render without errors.

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';

it('renders without crashing', () => {
  const div = document.createElement('div');
  ReactDOM.render(<App />, div);
  ReactDOM.unmountComponentAtNode(div);
});
```

The "it"-block describes one test case. It comes with a test description, and when you test it, it can either succeed or fail. Furthermore, you could wrap it into a "describe"-block that defines your test suite. A test suite could include a bunch of the "it"-blocks for one specific component. You will see "describe"-blocks later. Both blocks are used to separate and organize your test cases.

Note that the `it` function is acknowledged in the JavaScript community as the

function where you run a single test. However, in Jest it is often found as an alias `test` function.

You can run the test cases using the interactive *create-react-app* test script on the command line. You will get the output for all test cases on your command line interface.

```
npm test
```

Jest enables you to write snapshot tests. These tests make a snapshot of your rendered component and runs it against future snapshots. When a future snapshot changes, you will get notified in the test. You can either accept the snapshot change, because you changed the component implementation on purpose, or deny the change and investigate for the error. It complements unit tests very well, because you only test the differences of the rendered output. It doesn't add big maintenance costs since you can accept snapshots for intentional changes.

Jest stores snapshots in a folder so it can validate the diff against a future snapshot. This also lets use share snapshots across teams when having version control such as git in place.

Before writing your first snapshot test with Jest, you have to install its utility library:

```
npm install --save-dev react-test-renderer
```

Now you can extend the App component test with your first snapshot test. First, import the new functionality from the node package and wrap your previous "it"-block for the App component into a descriptive "describe"-block. In this case, the test suite is only for the App component.

```
import React from 'react';
import ReactDOM from 'react-dom';
import renderer from 'react-test-renderer';
import App from './App';

describe('App', () => {

  it('renders without crashing', () => {
    const div = document.createElement('div');
    ReactDOM.render(<App />, div);
    ReactDOM.unmountComponentAtNode(div);
```

```
  });

});
```

Testing with Jest

Now you can implement your first snapshot test by using a “test”-block:

```
import React from 'react';
import ReactDOM from 'react-dom';
import renderer from 'react-test-renderer';
import App from './App';

describe('App', () => {

  it('renders without crashing', () => {
    const div = document.createElement('div');
    ReactDOM.render(<App />, div);
    ReactDOM.unmountComponentAtNode(div);
  });

  test('has a valid snapshot', () => {
    const component = renderer.create(
      <App />
    );
    const tree = component.toJSON();
    expect(tree).toMatchSnapshot();
  });
});
```

Run your tests again and observe how they succeed or fail. Once you change the output of the render block in your App component, the snapshot test should fail. Then you can decide to update the snapshot or investigate in your App component.

The `renderer.create()` function creates a snapshot of your App component. It renders it virtually, and then stores the DOM into a snapshot. Afterward, the snapshot is expected to match the previous version from the last test. This is how we make sure the DOM stays the same and doesn’t change anything by accident.

Testing the Search component

Let’s add more tests for our independent components. First, the Search component:

```
import React from 'react';
import ReactDOM from 'react-dom';
import renderer from 'react-test-renderer';
```

```
import renderer from 'react-test-renderer';
import App, { Search } from './App';

...

describe('Search', () => {

  it('renders without crashing', () => {
    const div = document.createElement('div');
    ReactDOM.render(<Search>Search</Search>, div);
    ReactDOM.unmountComponentAtNode(div);
  });

  test('has a valid snapshot', () => {
    const component = renderer.create(
      <Search>Search</Search>
    );
    const tree = component.toJSON();
    expect(tree).toMatchSnapshot();
  });

});
```

The Search component has two tests similar to the App component. The first test simply renders the Search component to the DOM and verifies that there is no error during the rendering process. If there would be an error, the test would break even though there isn't any assertion (e.g. expect, match, equal) in the test block. The second snapshot test is used to store a snapshot of the rendered component and to run it against a previous snapshot. It fails when the snapshot has changed.

Second, you can test the Button component whereas the same test rules as in the Search component apply.

```
...
import App, { Search, Button } from './App';

...

describe('Button', () => {

  it('renders without crashing', () => {
    const div = document.createElement('div');
    ReactDOM.render(<Button>Give Me More</Button>, div);
    ReactDOM.unmountComponentAtNode(div);
  });

  test('has a valid snapshot', () => {
    const component = renderer.create(
      <Button>Give Me More</Button>
    );
    const tree = component.toJSON();
    expect(tree).toMatchSnapshot();
  });
```

```
});
```

Finally, the Table component that you can pass a bunch of initial props to render it with a sample list.

```
...
import App, { Search, Button, Table } from './App';
...

describe('Table', () => {

  const props = {
    list: [
      { title: '1', author: '1', num_comments: 1, points: 2, objectID: 'y' },
      { title: '2', author: '2', num_comments: 1, points: 2, objectID: 'z' },
    ],
  };

  it('renders without crashing', () => {
    const div = document.createElement('div');
    ReactDOM.render(<Table { ...props } />, div);
  });

  test('has a valid snapshot', () => {
    const component = renderer.create(
      <Table { ...props } />
    );
    const tree = component.toJSON();
    expect(tree).toMatchSnapshot();
  });
});
```

Snapshot tests usually stay pretty basic. You only want to make sure the component doesn't change its output. Once it does, you have to decide if you accept the changes, otherwise you have to fix the component.

Exercises

- See how a snapshot test fails once you change your component's return value in the `render()` method
  - Accept or deny the snapshot change(s)
- Keep your snapshots tests up to date when the implementation of components change in next chapters

Further Reading:

- Read about Jest in React

Quick quiz on Jest!

Q

The `it` block can contain multiple test cases.

COMPLETED 0%

1 of 1

Unit Tests with Enzyme

In this lesson, we'll discuss unit testing in React using a testing utility called Enzyme.

Introduction to Enzyme

Enzyme is a testing utility by Airbnb to assert, manipulate, and traverse React components. It is used to conduct unit tests to complement snapshot tests in React. First we have to install it along with its extension, since it doesn't come with *create-react-app*.

Installing Enzyme

```
npm install --save-dev enzyme react-addons-test-utils enzyme-adapter-react-16
```

Second, we include it in the test setup and we initialize its adapter.

```
import React from 'react';
import ReactDOM from 'react-dom';
import renderer from 'react-test-renderer';
import Enzyme from 'enzyme';
import Adapter from 'enzyme-adapter-react-16';
import App, { Search, Button, Table } from './App';

Enzyme.configure({ adapter: new Adapter() });
```

Now you can write your first unit test in the Table "describe"-block. You will use `shallow()` to render your component and assert that the Table was passed two items. The assertion simply checks if the element has two elements with the class `table-row`.

```
import React from 'react';
import ReactDOM from 'react-dom';
import renderer from 'react-test-renderer';
import Enzyme, { shallow } from 'enzyme';
import Adapter from 'enzyme-adapter-react-16';
import App, { Search, Button, Table } from './App';

Enzyme.configure({ adapter: new Adapter() });
```

```
describe('Table', () => {


  const props = {
    list: [
      { title: '1', author: '1', num_comments: 1, points: 2, objectID: 'y' },
      { title: '2', author: '2', num_comments: 1, points: 2, objectID: 'z' },
    ],
  };

  ...

  it('shows two items in list', () => {
    const element = shallow(
      <Table { ...props } />
    );

    expect(element.find('.table-row').length).toBe(2);
  });

});
```

Shallow renders the component without its child components, so you can dedicate the test to one component.

Enzyme has three rendering mechanisms in its API. You already know `shallow()`, but there is also `mount()` and `render()`. Both instantiate instances of the parent component and all child components. `mount()` gives you access to the component lifecycle methods.

When to use what

- Always begin with a shallow test
- If `componentDidMount()` or `componentDidUpdate()` should be tested, use `mount()`
- If you want to test component lifecycle and children behavior, use `mount()`
- If you want to test a component's children rendering with less overhead than `mount()` and you are not interested in lifecycle methods, use `render()`

Continue to unit test your components, but be sure to keep the tests simple and maintainable. Otherwise, you will have to refactor them once you change your components. This is one the main reason Facebook introduced snapshot tests with Jest.

Exercises

- Write a unit test with Enzyme for your Button component
- Keep your unit tests up to date during the following chapters

Further Reading:

- Read about Enzyme and its rendering API
- Read about testing React applications

Component Interface with PropTypes

In this lesson, we introduce a way to make React components type-safe using PropTypes.

TypeScript and Flow are used to introduce a type system to JavaScript. A typed language is less error prone because the code gets validated based on its program text. Editors and other utilities can catch these errors before the program runs.

In the course, you will not introduce Flow or TypeScript, but another useful way to check your types in components: React comes with a built-in type checker to prevent bugs. You can use 'PropTypes' to describe your component interface. All the props passed from a parent component to a child get validated based on the PropTypes interface assigned to the child.

This section will show you to make components type safe with PropTypes. I will omit the changes for the following chapters, since they add unnecessary code refactorings, but you should update them along the way to keep your components interface type safe.

First, install a separate package for React:

```
npm install prop-types
```

Now, you can import the PropTypes.

```
import React, { Component } from 'react';
import axios from 'axios';
import PropTypes from 'prop-types';
```

Let's start to assign a props interface to the components:

```
const Button = ({
  onClick,
  className = '',
  children,
```

```
  children,
}) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

Button.propTypes = {
  onClick: PropTypes.func,
  className: PropTypes.string,
  children: PropTypes.node,
};
```

Essentially, we want to take every argument from the function signature and assign a PropType to it. The basic PropTypes for primitives and complex objects are:

- PropTypes.array
- PropTypes.bool
- PropTypes.func
- PropTypes.number
- PropTypes.object
- PropTypes.string

There are two more PropTypes that can define a renderable fragment (node), e.g. a string, and a React element:

- PropTypes.node
- PropTypes.element

You already used the `node` PropType for the Button component. There are more PropType definitions in the official React documentation.

At the moment, all the defined PropTypes for the Button are optional. The parameters can be `null` or `undefined`. But for several props you want to enforce that they be defined. To do this, we make it a requirement that these props are passed to the component.

```
Button.propTypes = {
  onClick: PropTypes.func.isRequired,
  className: PropTypes.string,
  children: PropTypes.node.isRequired,
};
```

The `className` is not required, because it can default to an empty string. Next you will define a PropType interface for the Table component:

```
Table.propTypes = {
  list: PropTypes.array.isRequired,
  onDismiss: PropTypes.func.isRequired,
};
```

You can define the content of an array PropType more explicitly:

```
Table.propTypes = {
  list: PropTypes.arrayOf(
    PropTypes.shape({
      objectID: PropTypes.string.isRequired,
      author: PropTypes.string,
      url: PropTypes.string,
      num_comments: PropTypes.number,
      points: PropTypes.number,
    })
  ).isRequired,
  onDismiss: PropTypes.func.isRequired,
};
```

Only the `objectID` is required, because some of the code depends on it. The other properties are only displayed, so they are not required. Moreover you cannot be sure the Hacker News API always has a defined property for each object in the array.

You can also define default props in your component. The `className` property has an ES6 default parameter in the component signature:

```
const Button = ({
  onClick,
  className = '',
  children
}) =>
  ...
```

Replace it with the internal React default prop:

```
const Button = ({
  onClick,
  className,
  children
```

```
}) =>
  <button
    onClick={onClick}

    className={className}
    type="button"
  >
    {children}
  </button>

Button.defaultProps = {
  className: '',
};
```

Like the ES6 default parameter, the default prop ensures the property is set to a default value when the parent component doesn't specify it. The PropType type check happens after the default prop is evaluated.

If you run your tests again, you might see PropType errors for your components on your command line. It happens when we don't define all props for components in the tests that are required in the PropType definition. The tests themselves all pass correctly though. Make sure to pass all required props to the components in your tests to avoid these errors.

Exercises

- Define the PropType interface for the Search component
- Add and update the PropType interfaces when you add and update components in the next chapters

Further Reading:

- Read about React PropTypes

Q

Which of the following is an example of complex object PropType?

Debugging with React Developer Tools

In this lesson, we'll learn how to debug React code with React Developer Tools

Introducing React Developer Tools

React Developer Tools lets you inspect the React components hierarchy, props and state. It comes as a browser extension for Chrome and Firefox and as a standalone app that works with other environments. Once installed, the extension icon will light up on websites using React. On such pages, you will see a tab called “React” in your browser’s developer tools.

Debugging the Hacker News App

Let’s try it on our Hacker News application. On most browsers, a quick way to bring the *dev tools* up is to right-click on the page and than hit “Inspect”. Do it when your application is loaded, then click on the “React” tab. You should see its elements hierarchy, being `<App>` the root element. If you expand it, you will find instances of your `<Search>`, `<Table>` and `<Button>` components, as well.

The extension shows on the side pane the component’s state and props for the selected element. For instance, if you click on `<App>`, you will see that it has no props, but it already has a state. A very straightforward debugging technique is monitoring your application’s state changes from user interaction.

First, check the “Highlight Updates” option, usually above the elements tree. Second, type a different search term in the application’s input field. Only `searchTerm` will be changed in the component’s state. We already knew that would happen, but now we can see it working as planned.

Finally, press the “Search” button. The `searchKey` state will immediately changes to same value as `searchTerm`, and then the response object is added to `results`. The asynchronous nature of your code is now visible.

If you right-click on any element, a dropdown menu will show several useful

options. For instance, you could copy the element's props or name, find the

corresponding DOM node, or jump to the application's source code in the browser. The last option is very useful for inserting breakpoints and debugging your JavaScript functions.

Exercises

- Install the React Developer Tools extension on your favorite browser
  - Run your Hacker News Clone application and inspect it using the extension
  - Experiment with state and props changes
  - Watch what happens when you trigger an asynchronous request
  - Perform several requests, including repeated ones. Watch the cache mechanism working

Further Reading:

- Read about how to debug your JavaScript functions in the browser
- Read about (and use) the React Profiler

Recap

In this lesson, we the chapter focused on keeping code organized, testing, as well as useful tools for debugging a React application.

The chapter focused on keeping code organized in a scaling application. We covered the best practices for structuring your folders and files. We also covered testing, an important practice to keep source code robust, as well as useful tools for debugging a React application. This chapter stepped back from the practical application for a moment, these topics could be covered in detail before moving on.

You have learned how to organize and test React code! Let's recap the last chapter:

- **React**
  - PropTypes let you define type checks for components
  - Jest allows you to write snapshot tests for your components
  - Enzyme allows you to write unit tests for your components
  - React Developer Tools is a helpful debugging tool
- **ES6**
  - import and export statements help you to organize your code
- **General**
  - code organization allows you to scale your application with best practices

You can find the source code in the official repository.

Ref a DOM Element

In this lesson, we'll learn how to interact with DOM nodes in React.

Why interact with the DOM?

Sometimes you need to interact with your DOM nodes in React. The `ref` attribute gives you access to a node in your elements. That is usually an anti pattern in React, because you should use its declarative way of doing things and its unidirectional data flow. We learned about it when we introduced the first search input field, but there are certain cases where you need access to the DOM node.

The official documentation mentions three:

- to use the DOM API to manage focus, media playback etc
- to invoke imperative DOM node animations
- to integrate with a third-party library that needs the DOM node (e.g. D3.js)

Referring to the DOM in the Search component

We'll use the Search component as an example. When the application renders for the first time, the input field should be focused. This is one case where we need access to the DOM API. The `ref` attribute can be used in both functional stateless components and ES6 class components. In this example, we need a lifecycle method, so the approach is showcased using the `ref` attribute with an ES6 class component.

The initial step is to refactor the functional stateless component to an ES6 class component.

```
class Search extends Component {
  render() {
    const {
      value,
```

```
      onChange,
      onSubmit,
      children
    } = this.props;

    return (
      <form onSubmit={onSubmit}>
        <input
          type="text"
          value={value}
          onChange={onChange}
        />
        <button type="submit">
          {children}
        </button>
      </form>
    );
  }
}
```

The `this` object of an ES6 class component helps us to reference the DOM element with the `ref` attribute.

```
class Search extends Component {
  render() {
    const {
      value,
      onChange,
      onSubmit,
      children
    } = this.props;

    return (
      <form onSubmit={onSubmit}>
        <input
          type="text"
          value={value}
          onChange={onChange}
          ref={el => this.input = el}
        />
        <button type="submit">
          {children}
        </button>
      </form>
    );
  }
}
```

Now you can focus the input field when the component mounted by using the `this` object, the appropriate lifecycle method, and the DOM API.

```
class Search extends Component {
  componentDidMount() {
    if (this.input) {
      this.input.focus();
```

```
    this.input.focus();
    }
  }

  render() {
    const {
      value,
      onChange,
      onSubmit,
      children
    } = this.props;

    return (
      <form onSubmit={onSubmit}>
        <input
          type="text"
          value={value}
          onChange={onChange}
          ref={el => this.input = el}
        />
        <button type="submit">
          {children}
        </button>
      </form>
    );
  }
}
```

The input field should be focused when the application renders. We access to the `ref` in a functional stateless component without the `this` object using the following functional stateless component:

```
const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) => {
  let input;
  return (
    <form onSubmit={onSubmit}>
      <input
        type="text"
        value={value}
        onChange={onChange}
        ref={el => this.input = el}
      />
      <button type="submit">
        {children}
      </button>
    </form>
  );
}
```

Now we can access the input DOM element. In our case it wouldn't help much since there's no lifecycle method in a functional stateless component to trigger

the focus. So we are not going to make use of the input variable here. In the future, though, you may encounter cases where it makes sense to use a functional stateless component with the `ref` attribute.

Further Reading

- Read about the usage of the ref attribute in React
- Read about the ref attribute in general in React

Loading ...

In this lesson, we'll learn how to add a 'loading' message to our application while results are being fetched.

Displaying a Loading message

Now we get back to the application, where we'll show a loading indicator when a search request submits to the Hacker News API. The request is asynchronous, so you should show your user feedback that something is happening.

Defining the Loading component and state property

Let's define a reusable Loading component in the *src/App.js* file like in the following.

```
const Loading = () =>
  <div>Loading ...</div>
```

Now we need a property to store the loading state. Based on the loading state, we can decide to show the Loading component later.

```
class App extends Component {
  _isMounted = false;

  constructor(props) {
    super(props);

    this.state = {
      results: null,
      searchKey: '',
      searchTerm: DEFAULT_QUERY,
      error: null,
      isLoading: false,
    };

    ...
  }

  ...

}
```

Setting the isLoading property

The initial value of that `isLoading` property is false. We don't load anything before the App component is mounted. When the request is made, the loading state is set to true. The request will succeed eventually, and you can set the loading state to false.

```
class App extends Component {

  ...

  setSearchTopStories(result) {
    ...

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      },
      isLoading: false
    });
  }

  fetchSearchTopStories(searchTerm, page = 0) {
    this.setState({ isLoading: true });

    axios(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}&${PAR
      .then(result => this._isMounted && this.setSearchTopStories(result.data))
      .catch(error => this._isMounted && this.setState({ error }));
  }

  ...

}
```

When to display the loading message

In the last step, we used the Loading component in the App component. A conditional rendering based on the loading state will decide whether to show a Loading component or the Button component. The latter is the button to fetch more data.

```
class App extends Component {

  ...

  render() {
    const {
      searchTerm,
```

```
      results,
      searchKey,
      error,

      isLoading
    } = this.state;

    ...

    return (
      <div className="page">
        ...
        <div className="interactions">
          { isLoading
            ? <Loading />
            : <Button
                onClick={() => this.fetchSearchTopStories(searchKey, page + 1)}
              >
              More
            </Button>
          }
        </div>
      </div>
    );
  }
}
```

Initially, the Loading component will show when you start your application, because you make a request on `componentDidMount()`. There is no Table component, because the list is empty. When the response returns from the Hacker News API, the result is shown, the loading state is set to false, and the Loading component disappears. Instead, the “More” button to fetch more data appears. Once you fetch more data, the button will disappear again and the Loading component will appear.

The Hacker News App with the Loading message

```
import React, { Component } from 'react';
require('./App.css');

const DEFAULT_QUERY = 'redux';
const DEFAULT_HPP = '100';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      results: null,
```

```
    searchKey: '',
    searchTerm: DEFAULT_QUERY,
    error: null,
    isLoading: false,
  };

  this.needsToSearchTopstories = this.needsToSearchTopstories.bind(this);
  this.setSearchTopstories = this.setSearchTopstories.bind(this);
  this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
  this.onSearchChange = this.onSearchChange.bind(this);
  this.onSearchSubmit = this.onSearchSubmit.bind(this);
  this.onDismiss = this.onDismiss.bind(this);
}

needsToSearchTopstories(searchTerm) {
  return !this.state.results[searchTerm];
}

setSearchTopstories(result) {
  const { hits, page } = result;
  const { searchKey, results } = this.state;

  const oldHits = results && results[searchKey]
    ? results[searchKey].hits
    : [];

  const updatedHits = [
    ...oldHits,
    ...hits
  ];

  this.setState({
    results: {
      ...results,
      [searchKey]: { hits: updatedHits, page }
    },
    isLoading: false
  });
}

fetchSearchTopstories(searchTerm, page = 0) {
  this.setState({ isLoading: true });

  fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}`)
    .then(response => response.json())
    .then(result => this.setSearchTopstories(result))
    .catch(e => this.setState({ error: e }));
}

componentDidMount() {
  const { searchTerm } = this.state;
  this.setState({ searchKey: searchTerm });
  this.fetchSearchTopstories(searchTerm);
}

onSearchChange(event) {
  this.setState({ searchTerm: event.target.value });
}

onSearchSubmit(event) {
  const { searchTerm } = this.state;
  this.setState({ searchKey: searchTerm });
```

```
    if (this.needsToSearchTopstories(searchTerm)) {
      this.fetchSearchTopstories(searchTerm);
    }

    event.preventDefault();
  }

  onDismiss(id) {
    const { searchKey, results } = this.state;
    const { hits, page } = results[searchKey];

    const isNotId = item => item.objectID !== id;
    const updatedHits = hits.filter(isNotId);

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      }
    });
  }

  render() {
    const {
      searchTerm,
      results,
      searchKey,
      error,
      isLoading
    } = this.state;

    const page = (
      results &&
      results[searchKey] &&
      results[searchKey].page
    ) || 0;

    const list = (
      results &&
      results[searchKey] &&
      results[searchKey].hits
    ) || [];

    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
          >
            Search
          </Search>
        </div>
        { error
          ? <div className="interactions">
            <p>Something went wrong.</p>
          </div>
          : <Table
            list={list}
            onDismiss={this.onDismiss}
```

```
          />
        }
        <div className="interactions">
          { isLoading
            ? <Loading />
            : <Button
              onClick={() => this.fetchSearchTopstories(searchKey, page + 1)}>
              More
            </Button>
          }
        </div>
      </div>
    );
  }
}


const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
    />
    <button type="submit">
      {children}
    </button>
  </form>

const Table = ({ list, onDismiss }) =>
  <div className="table">
    { list.map(item =>
      <div key={item.objectID} className="table-row">
        <span style={{ width: '40%' }}>
          <a href={item.url}>{item.title}</a>
        </span>
        <span style={{ width: '30%' }}>
          {item.author}
        </span>
        <span style={{ width: '10%' }}>
          {item.num_comments}
        </span>
        <span style={{ width: '10%' }}>
          {item.points}
        </span>
        <span style={{ width: '10%' }}>
          <Button
            onClick={() => onDismiss(item.objectID)}
            className="button-inline"
          >
            Dismiss
          </Button>
        </span>
      </div>
    )}
  </div>
```

```
const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

const Loading = () =>
  <div>Loading ...</div>

export default App;

export {
  Button,
  Search,
  Table,
};
```

Exercises:

- Use a library such as Font Awesome to show a loading icon instead of the “Loading ...” text in the above!

Higher-Order Components

In this lesson, you will learn how to implement higher-order components in React.

What are Higher-order components?

Higher-order components (HOC) are an advanced concept in React. HOCs are an equivalent to higher-order functions. They take any input, usually a component, but also optional arguments, and return a component as output. The returned component is an enhanced version of the input component, and it can be used in your JSX.

HOCs are used for different use cases. They can prepare properties, manage state, or alter the representation of a component. One case is to use a HOC as a helper for a conditional rendering. Imagine you have a List component that renders a list of items or nothing, because the list is empty or null. The HOC could shield away that the list would render nothing when there is no list. On the other hand, the plain List component doesn't need to bother anymore about an non existent list, as it only cares about rendering the list.

Basic example of an HOC

Let's do a simple HOC that takes a component as input and returns a component. Lets place it in the *App.js* file.

```
function withFoo(Component) {
  return function(props) {
    return <Component { ...props } />;
  }
}
```

It is a useful convention to prefix a HOC with `with`. Since you are using JavaScript ES6, you can express the HOC better with an ES6 arrow function.

```
const withEnhancement = (Component) => (props) =>
  <Component { ...props } />
```

HOCs and conditional rendering: Loading component

In our example, the input component stays the same as the output, so nothing happens. The output component should show the Loading component when the loading state is true, otherwise it should show the input component. A conditional rendering is a great use case for an HOC.

```
const withLoading = (Component) => (props) =>
  props.isLoading
    ? <Loading />
    : <Component { ...props } />
```

Based on the loading property, you can apply a conditional rendering. The function will return the Loading component or the input component. In general, it can be very efficient to spread an object like the props object in the previous example as input for a component. See the difference in the following code snippet:

```
// before you would have to destructure the props before passing them
const { firstname, lastname } = props;
<UserProfile firstname={firstname} lastname={lastname} />

// but you can use the object spread operator to pass all object properties
<UserProfile { ...props } />
```

We passed all the props including the `isLoading` property by spreading the object into the input component. The input component may not care about the `isLoading` property. You can use the ES6 rest destructuring to avoid it:

```
const withLoading = (Component) => ({ isLoading, ...rest }) =>
  isLoading
    ? <Loading />
    : <Component { ...rest } />
```

It takes one property out of the object, but keeps the remaining object, which also works with multiple properties. More can be read about it in Mozilla's destructuring assignment.

Now you can use the HOC in JSX. Maybe you want to show either the "More" button or the Loading component. The Loading component is already encapsulated in the HOC, but an input component is missing. For showing either a Button component or a Loading component, the Button is the input

either a Button component or a Loading component, the Button is the input

component of the HOC. The enhanced output component is a ButtonWithLoading component.

```
const Button = ({
  onClick,
  className = '',
  children,
}) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

const Loading = () =>
  <div>Loading ...</div>

const withLoading = (Component) => ({ isLoading, ...rest }) =>
  isLoading
    ? <Loading />
    : <Component { ...rest } />

const ButtonWithLoading = withLoading(Button);
```

Everything is defined now. As a last step, you have to use the `ButtonWithLoading` component, which receives the loading state as an additional property. While the HOC consumes the loading property, all other props get passed to the Button component.

```
class App extends Component {

  ...

  render() {
    ...
    return (
      <div className="page">
        ...
        <div className="interactions">
          <ButtonWithLoading
            isLoading={isLoading}
            onClick={() => this.fetchSearchTopStories(searchKey, page + 1)}
          >
            More
          </ButtonWithLoading>
        </div>
      </div>
    );
  }
```

```
}
```

Why Snapshot tests will fail now

Note that when you run your tests again, your snapshot test for the App component fails. The diff might look like the following on the command line:

```
  <button
     className=""
     onClick={[Function]}
     type="button"
   >
     More
   </button>
   <div>
     Loading ...
   </div>
```

You can either fix the component now, when you think there is something wrong about it, or can accept the new snapshot of it. Because you introduced the Loading component in this chapter, you can accept the altered snapshot test on the command line in the interactive test.

Higher-order components are an advanced pattern in React. They have multiple purposes: improved reusability of components, greater abstraction, composability of components, and manipulations of props, state and view. I encourage you to read gentle introduction to higher-order components. It gives you another approach to learn them, shows you an elegant way to use them in a functional programming way, and solves the problem of conditional rendering with higher-order components.

```
import React, { Component } from 'react';
require('./App.css');

const DEFAULT_QUERY = 'redux';
const DEFAULT_HPP = '100';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
```

```
    results: null,
    searchKey: '',
    searchTerm: DEFAULT_QUERY,

    error: null,
    isLoading: false,
  };

  this.needsToSearchTopstories = this.needsToSearchTopstories.bind(this);
  this.setSearchTopstories = this.setSearchTopstories.bind(this);
  this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
  this.onSearchChange = this.onSearchChange.bind(this);
  this.onSearchSubmit = this.onSearchSubmit.bind(this);
  this.onDismiss = this.onDismiss.bind(this);
}

needsToSearchTopstories(searchTerm) {
  return !this.state.results[searchTerm];
}

setSearchTopstories(result) {
  const { hits, page } = result;
  const { searchKey, results } = this.state;

  const oldHits = results && results[searchKey]
    ? results[searchKey].hits
    : [];

  const updatedHits = [
    ...oldHits,
    ...hits
  ];

  this.setState({
    results: {
      ...results,
      [searchKey]: { hits: updatedHits, page }
    },
    isLoading: false
  });
}

fetchSearchTopstories(searchTerm, page = 0) {
  this.setState({ isLoading: true });

  fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}`)
    .then(response => response.json())
    .then(result => this.setSearchTopstories(result))
    .catch(e => this.setState({ error: e }));
}

componentDidMount() {
  const { searchTerm } = this.state;
  this.setState({ searchKey: searchTerm });
  this.fetchSearchTopstories(searchTerm);
}

onSearchChange(event) {
  this.setState({ searchTerm: event.target.value });
}

onSearchSubmit(event) {
  const { searchTerm } = this.state;
```

```
    this.setState({ searchKey: searchTerm });

    if (this.needsToSearchTopstories(searchTerm)) {
      this.fetchSearchTopstories(searchTerm);
    }

    event.preventDefault();
  }

  onDismiss(id) {
    const { searchKey, results } = this.state;
    const { hits, page } = results[searchKey];

    const isNotId = item => item.objectID !== id;
    const updatedHits = hits.filter(isNotId);

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      }
    });
  }

  render() {
    const {
      searchTerm,
      results,
      searchKey,
      error,
      isLoading
    } = this.state;

    const page = (
      results &&
      results[searchKey] &&
      results[searchKey].page
    ) || 0;

    const list = (
      results &&
      results[searchKey] &&
      results[searchKey].hits
    ) || [];

    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
          >
            Search
          </Search>
        </div>
        { error
          ? <div className="interactions">
            <p>Something went wrong.</p>
          </div>
          : <Table
            list={list}
```

```
            onDismiss={this.onDismiss}
          />
        }
        <div className="interactions">
          <ButtonWithLoading
            isLoading={isLoading}
            onClick={() => this.fetchSearchTopstories(searchKey, page + 1)}>
            More
          </ButtonWithLoading>
        </div>
      </div>
    );
  }
}

const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
    />
    <button type="submit">
      {children}
    </button>
  </form>

const Table = ({ list, onDismiss }) =>
  <div className="table">
    { list.map(item =>
      <div key={item.objectID} className="table-row">
        <span style={{ width: '40%' }}>
          <a href={item.url}>{item.title}</a>
        </span>
        <span style={{ width: '30%' }}>
          {item.author}
        </span>
        <span style={{ width: '10%' }}>
          {item.num_comments}
        </span>
        <span style={{ width: '10%' }}>
          {item.points}
        </span>
        <span style={{ width: '10%' }}>
          <Button
            onClick={() => onDismiss(item.objectID)}
            className="button-inline"
          >
            Dismiss
          </Button>
        </span>
      </div>
    )}
  </div>

const Button = ({ onClick, className = '', children }) =>
  <button
```

```
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

const Loading = () =>
  <div>Loading ...</div>

const withLoading = (Component) => ({ isLoading, ...rest }) =>
  isLoading ? <Loading /> : <Component { ...rest } />

const ButtonWithLoading = withLoading(Button);

export default App;

export {
  Button,
  Search,
  Table,
};
```

Quick quiz on Higher Order Components in React!

Q

Can Higher Order Components be used to

COMPLETED 0%

1 of 1

Exercises:

- Experiment with the HOC you have created
- Think about a use case where another HOC would make sense
    - Implement the HOC, if there is a use case

Further Reading

- Read a gentle introduction to higher-order components

Advanced Sorting

In this lesson, you'll learn an advanced data sorting technique to sort our app's table by column

Sorting our table component by column

We implemented a client and server-side search interaction earlier. Since you have a Table component, it makes sense to enhance it with advanced interactions. Next, we'll introduce a sort functionality for each column by using the column headers of the Table.

It is possible to write your own sort function, but I prefer to use a utility library like Lodash for these cases. There are other options, but this is the one I prefer. We have Lodash running on the code widgets in this course but if you are following along on a local setup, install Lodash for our sort function:

```
npm install lodash
```

Now we import the sort functionality of Lodash to our *App.js* file:

```
import React, { Component } from 'react';
import fetch from 'isomorphic-fetch';
import { sortBy } from 'lodash';
import './App.css';
```

Creating `SORTS` to reference any function

Now we have several columns in Table: title, author, comments and points columns. You can define sort functions where each takes a list and returns a list of items sorted by a specific property. Additionally, you will need a default sort function that doesn't sort, but returns the unsorted list. This will be the initial state.

```
...

const SORTS = {
```

```
  NONE: list => list,
  TITLE: list => sortBy(list, 'title'),
  AUTHOR: list => sortBy(list, 'author'),

  COMMENTS: list => sortBy(list, 'num_comments').reverse(),
  POINTS: list => sortBy(list, 'points').reverse(),
};

class App extends Component {
  ...
}

...
```

Two of the sort functions return a reversed list. That's to see the items with the highest comments and points, rather than the items with the lowest counts when the list is sorted for the first time.

The `SORTS` object allows you to reference any sort function now.

Again, the App component is responsible for storing the state of the sort. The initial state will be the default sort function, which doesn't sort at all and returns the input list as output.

```
this.state = {
  results: null,
  searchKey: '',
  searchTerm: DEFAULT_QUERY,
  error: null,
  isLoading: false,
  sortKey: 'NONE',
};
```

Once we choose a different `sortKey`, like the `AUTHOR` key, we sort the list with the appropriate sort function from the `SORTS` object.

Setting `sortKey` in local state

Now we define a new class method in App component that sets a `sortKey` to the local component state, then `sortKey` can be used to retrieve the sorting function to apply it to the list:

```
class App extends Component {
  _isMounted = false;

  constructor(props) {

    ...

    this.needsToSearchTopStories = this.needsToSearchTopStories.bind(this);
```

```
    this.setSearchTopStories = this.setSearchTopStories.bind(this);
    this.fetchSearchTopStories = this.fetchSearchTopStories.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
    this.onSort = this.onSort.bind(this);
  }

  ...

  onSort(sortKey) {
    this.setState({ sortKey });
  }

  ...

}
```

Passing `sortKey` to the table component

The next step is to pass the method and `sortKey` to the Table component.

```
class App extends Component {

  ...

  render() {
    const {
      searchTerm,
      results,
      searchKey,
      error,
      isLoading,
      sortKey
    } = this.state;

    ...

    return (
      <div className="page">
        ...
        <Table
          list={list}
          sortKey={sortKey}
          onSort={this.onSort}
          onDismiss={this.onDismiss}
        />
        ...
      </div>
    );
  }
}
```

The Table component is responsible for sorting the list. It takes one of the `SORT` functions by `sortKey` and passes the list as input, after which it keeps mapping over the sorted list.

mapping over the sorted list.

```
const Table = ({
  list,
  sortKey,
  onSort,
  onDismiss
}) =>
  <div className="table">
    {SORTS[sortKey](list).map(item =>
      <div key={item.objectID} className="table-row">
        ...
      </div>
    )}
  </div>
```

In theory, the list should get sorted by one of the functions. But the default sort is set to `NONE`, so nothing is sorted yet, as nothing executes the `onSort()` method to change the `sortKey`.

Implementing sort in the Table

We extend the Table with a row of column headers that use Sort components in columns to sort each column:

```
const Table = ({
  list,
  sortKey,
  onSort,
  onDismiss
}) =>
  <div className="table">
    <div className="table-header">
      <span style={{ width: '40%' }}>
        <Sort
          sortKey={'TITLE'}
          onSort={onSort}
        >
          Title
        </Sort>
      </span>
      <span style={{ width: '30%' }}>
        <Sort
          sortKey={'AUTHOR'}
          onSort={onSort}
        >
          Author
        </Sort>
      </span>
      <span style={{ width: '10%' }}>
        <Sort
          sortKey={'COMMENTS'}
          onSort={onSort}
        >
```

```
          Comments
        </Sort>
      </span>

      <span style={{ width: '10%' }}>
        <Sort
          sortKey={'POINTS'}
          onSort={onSort}
        >
          Points
        </Sort>
      </span>
      <span style={{ width: '10%' }}>
        Archive
      </span>
    </div>
    {SORTS[sortKey](list).map(item =>
      ...
    )}
  </div>
```

Each Sort component gets a specific `sortKey` and the general `onSort()` function. Internally, it calls the method with the `sortKey` to set the specific key.

```
const Sort = ({ sortKey, onSort, children }) =>
  <Button onClick={() => onSort(sortKey)}>
    {children}
  </Button>
```

As you can see, the Sort component reuses your common Button component. On a button click, each individual passed `sortKey` is set by the `onSort()` method, so the list is sorted when column headers are selected.

Now we'll improve the look of the button in the column header. Let's give it a proper `className`:

```
const Sort = ({ sortKey, onSort, children }) =>
  <Button
    onClick={() => onSort(sortKey)}
    className="button-inline"
  >
    {children}
```

This was done to improve the UI. In the next lesson, we'll continue to build upon our search functionality. We'll first reverse-sort our table.

Our app so far:

```
import React, { Component } from 'react';
import { sortBy } from 'lodash';
import classNames from 'classnames';
require('./App.css');

const DEFAULT_QUERY = 'redux';
const DEFAULT_HPP = '100';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';

const SORTS = {
  NONE: list => list,
  TITLE: list => sortBy(list, 'title'),
  AUTHOR: list => sortBy(list, 'author'),
  COMMENTS: list => sortBy(list, 'num_comments').reverse(),
  POINTS: list => sortBy(list, 'points').reverse(),
};

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      results: null,
      searchKey: '',
      searchTerm: DEFAULT_QUERY,
      error: null,
      isLoading: false,
      sortKey: 'NONE',
    };

    this.needsToSearchTopstories = this.needsToSearchTopstories.bind(this);
    this.setSearchTopstories = this.setSearchTopstories.bind(this);
    this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
    this.onSort = this.onSort.bind(this);
  }

  onSort(sortKey) {
    this.setState({ sortKey });
  }

  needsToSearchTopstories(searchTerm) {
    return !this.state.results[searchTerm];
  }

  setSearchTopstories(result) {
    const { hits, page } = result;
    const { searchKey, results } = this.state;

    const oldHits = results && results[searchKey]
      ? results[searchKey].hits
      : [];

    const updatedHits = [
      ...oldHits,
```

```
    ...hits
  ];

  this.setState({
    results: {
      ...results,
      [searchKey]: { hits: updatedHits, page }
    },
    isLoading: false
  });
}

fetchSearchTopstories(searchTerm, page = 0) {
  this.setState({ isLoading: true });

  fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}`)
    .then(response => response.json())
    .then(result => this.setSearchTopstories(result))
      .catch(e => this.setState({ error: e }));
}

componentDidMount() {
  const { searchTerm } = this.state;
  this.setState({ searchKey: searchTerm });
  this.fetchSearchTopstories(searchTerm);
}

onSearchChange(event) {
  this.setState({ searchTerm: event.target.value });
}

onSearchSubmit(event) {
  const { searchTerm } = this.state;
  this.setState({ searchKey: searchTerm });

  if (this.needsToSearchTopstories(searchTerm)) {
    this.fetchSearchTopstories(searchTerm);
  }

  event.preventDefault();
}

onDismiss(id) {
  const { searchKey, results } = this.state;
  const { hits, page } = results[searchKey];

  const isNotId = item => item.objectID !== id;
  const updatedHits = hits.filter(isNotId);

  this.setState({
    results: {
      ...results,
      [searchKey]: { hits: updatedHits, page }
    }
  });
}

render() {
  const {
    searchTerm,
    results,
    searchKey,
```

```
      error,
      isLoading,
      sortKey,
    } = this.state;

    const page = (
      results &&
      results[searchKey] &&
      results[searchKey].page
    ) || 0;

    const list = (
      results &&
      results[searchKey] &&
      results[searchKey].hits
    ) || [];

    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
          >
            Search
          </Search>
        </div>
        { error
          ? <div className="interactions">
            <p>Something went wrong.</p>
          </div>
          : <Table
                list={list}
                  sortKey={sortKey}
              onSort={this.onSort}
            onDismiss={this.onDismiss}
                />
        }
        <div className="interactions">
          <ButtonWithLoading
            isLoading={isLoading}
            onClick={() => this.fetchSearchTopstories(searchKey, page + 1)}>
            More
          </ButtonWithLoading>
        </div>
      </div>
    );
  }
}

const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
```

```
    />
    <button type="submit">
      {children}
    </button>
  </form>

const Table = ({
  list,
  sortKey,
  onSort,
  onDismiss
}) => {
  const sortedList = SORTS[sortKey](list);

  return(
    <div className="table">
      <div className="table-header">
        <span style={{ width: '40%' }}>
          <Sort
            sortKey={'TITLE'}
            onSort={onSort}
          >
            Title
          </Sort>
        </span>
        <span style={{ width: '30%' }}>
          <Sort
            sortKey={'AUTHOR'}
            onSort={onSort}
          >
            Author
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'COMMENTS'}
            onSort={onSort}
          >
            Comments
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'POINTS'}
            onSort={onSort}
          >
            Points
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          Archive
        </span>
      </div>
      { SORTS[sortKey](list).map(item =>
        <div key={item.objectID} className="table-row">
          <span style={{ width: '40%' }}>
            <a href={item.url}>{item.title}</a>
          </span>
          <span style={{ width: '30%' }}>
            {item.author}
          </span>
          <span style={{ width: '10%' }}>
```

```
            {item.num_comments}
          </span>
          <span style={{ width: '10%' }}>
            {item.points}
          </span>
          <span style={{ width: '10%' }}>
            <Button
              onClick={() => onDismiss(item.objectID)}
              className="button-inline"
            >
              Dismiss
            </Button>
          </span>
        </div>      )}
    </div>
  );
}

const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

const Loading = () =>
  <div>Loading ...</div>

const withLoading = (Component) => ({ isLoading, ...rest }) =>
  isLoading ? <Loading /> : <Component { ...rest } />

const ButtonWithLoading = withLoading(Button);

const Sort = ({
  sortKey,
  activeSortKey,
  onSort,
  children
}) => {
  const sortClass = classNames(
    'button-inline',
  );

  return (
    <Button
      onClick={() => onSort(sortKey)}
      className={sortClass}
    >
      {children}
    </Button>
  );
}

export default App;

export {
  Button,
  Search,
  Table,
};
```

Advanced Sorting: implementing reverse sort

In this lesson, you'll continue to implement our advanced data sorting technique. By the end of this lesson, we will be able to sort and reverse sort our app's table by column

In the previous lesson, we did some initial ground work to get our search functionality working. In this chapter, we'll finish the sort functionality with reverse sort. The list should perform a reverse sort once you click a Sort component twice. First, you need to define the reverse state with a boolean. The sort can be either reversed or non-reversed.

```
this.state = {
  results: null,
  searchKey: '',
  searchTerm: DEFAULT_QUERY,
  error: null,
  isLoading: false,
  sortKey: 'NONE',
  isSortReverse: false,
};
```

Now in your sort method, you can evaluate if the list is reverse sorted. It is reverse if the `sortKey` in the state is the same as the incoming `sortKey` and the reverse state is not already set to true.

```
onSort(sortKey) {
  const isSortReverse = this.state.sortKey === sortKey && !this.state.isSortReverse;
  this.setState({ sortKey, isSortReverse });
}
```

Again, we pass the reverse prop to your Table component:

```
class App extends Component {

  ...

  render() {
    const {
      searchTerm,
      results,
```

```
      searchKey,
      error,
      isLoading,

      sortKey,
      isSortReverse
    } = this.state;

    ...

    return (
      <div className="page">
        ...
        <Table
          list={list}
          sortKey={sortKey}
          isSortReverse={isSortReverse}
          onSort={this.onSort}
          onDismiss={this.onDismiss}
        />
        ...
      </div>
    );
  }
}
```

The Table has to have an arrow function block body to compute the data now:

```
const Table = ({
  list,
  sortKey,
  isSortReverse,
  onSort,
  onDismiss
}) => {
  const sortedList = SORTS[sortKey](list);
  const reverseSortedList = isSortReverse
    ? sortedList.reverse()
    : sortedList;

  return(
    <div className="table">
      <div className="table-header">
        ...
      </div>
      {reverseSortedList.map(item =>
        ...
      )}
    </div>
  );
}
```

Finally, we want to give the user visual feedback to distinguish which column is actively sorted. Each Sort component has its specific `sortKey` already, which can be used to identify the activated sort. We pass the `sortKey` from the internal component state as active sort key to your Sort component:

```
const Table = ({
  list,
  sortKey,
  isSortReverse,
  onSort,
  onDismiss
}) => {
  const sortedList = SORTS[sortKey](list);
  const reverseSortedList = isSortReverse
    ? sortedList.reverse()
    : sortedList;

  return(
    <div className="table">
      <div className="table-header">
        <span style={{ width: '40%' }}>
          <Sort
            sortKey={'TITLE'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Title
          </Sort>
        </span>
        <span style={{ width: '30%' }}>
          <Sort
            sortKey={'AUTHOR'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Author
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'COMMENTS'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Comments
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'POINTS'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Points
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          Archive
        </span>
      </div>
      {reverseSortedList.map(item =>
          ...
      )}
```

```
    </div>
  );
}
```

Now the user will know whether sort is active based on the `sortKey` and `activeSortKey` . Give your Sort component an extra `className` attribute, in case it is sorted, to give visual feedback:

```
const Sort = ({
  sortKey,
  activeSortKey,
  onSort,
  children
}) => {
  const sortClass = ['button-inline'];

  if (sortKey === activeSortKey) {
    sortClass.push('button-active');
  }

  return (
    <Button
      onClick={() => onSort(sortKey)}
      className={sortClass.join(' ')}
    >
      {children}
    </Button>
  );
}
```

We can define `sortClass` more efficiently using a library called classnames, which is installed using npm:

```
npm install classnames
```

After installation, we import it on top of the *src/App.js* file.

```
import React, { Component } from 'react';
import fetch from 'isomorphic-fetch';
import { sortBy } from 'lodash';
import classNames from 'classnames';
import './App.css';
```

Now we can define `className` with conditional classes:

```
const Sort = ({
  sortKey,
  activeSortKey,
  onSort,
  children
```

```
}) => {
  const sortClass = classNames(
    'button-inline',

    { 'button-active': sortKey === activeSortKey }
  );

  return (
    <Button
      onClick={() => onSort(sortKey)}
      className={sortClass}
    >
      {children}
    </Button>
  );
}
```

There will be failing snapshot and unit tests for the Table component. Since we intentionally changed again our component representations, we accept the snapshot tests, but we still need to fix the unit test. In *src/App.test.js* , provide a `sortKey` and the `isSortReverse` boolean for the Table component.

```
...

describe('Table', () => {

  const props = {
    list: [
      { title: '1', author: '1', num_comments: 1, points: 2, objectID: 'y' },
      { title: '2', author: '2', num_comments: 1, points: 2, objectID: 'z' },
    ],
    sortKey: 'TITLE',
    isSortReverse: false,
  };

  ...

});
```

We may need to accept the failing snapshot tests again for the Table component, because we provided extended props for it. The advanced sort interaction is finally complete.

App with Table sorting:

```
import React, { Component } from 'react';
import { sortBy } from 'lodash';
import classNames from 'classnames';
require('./App.css');

const DEFAULT_QUERY = 'redux';
const DEFAULT_HPP = '100';
```

```
const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';

const PARAM_PAGE = 'page=';

const SORTS = {
  NONE: list => list,
  TITLE: list => sortBy(list, 'title'),
  AUTHOR: list => sortBy(list, 'author'),
  COMMENTS: list => sortBy(list, 'num_comments').reverse(),
  POINTS: list => sortBy(list, 'points').reverse(),
};

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      results: null,
      searchKey: '',
      searchTerm: DEFAULT_QUERY,
      error: null,
      isLoading: false,
      sortKey: 'NONE',
      isSortReverse: false,
    };

    this.needsToSearchTopstories = this.needsToSearchTopstories.bind(this);
    this.setSearchTopstories = this.setSearchTopstories.bind(this);
    this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
    this.onSort = this.onSort.bind(this);
  }

  onSort(sortKey) {
    const isSortReverse = this.state.sortKey === sortKey && !this.state.isSortReverse;
    this.setState({ sortKey, isSortReverse });
  }

  needsToSearchTopstories(searchTerm) {
    return !this.state.results[searchTerm];
  }

  setSearchTopstories(result) {
    const { hits, page } = result;
    const { searchKey, results } = this.state;

    const oldHits = results && results[searchKey]
      ? results[searchKey].hits
      : [];

    const updatedHits = [
      ...oldHits,
      ...hits
    ];

    this.setState({
      results: {
        ...results,
```

```
      [searchKey]: { hits: updatedHits, page }
    },
    isLoading: false
  });
}

fetchSearchTopstories(searchTerm, page = 0) {
  this.setState({ isLoading: true });

  fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}`)
    .then(response => response.json())
    .then(result => this.setSearchTopstories(result))
      .catch(e => this.setState({ error: e }));
}

componentDidMount() {
  const { searchTerm } = this.state;
  this.setState({ searchKey: searchTerm });
  this.fetchSearchTopstories(searchTerm);
}

onSearchChange(event) {
  this.setState({ searchTerm: event.target.value });
}

onSearchSubmit(event) {
  const { searchTerm } = this.state;
  this.setState({ searchKey: searchTerm });

  if (this.needsToSearchTopstories(searchTerm)) {
    this.fetchSearchTopstories(searchTerm);
  }

  event.preventDefault();
}

onDismiss(id) {
  const { searchKey, results } = this.state;
  const { hits, page } = results[searchKey];

  const isNotId = item => item.objectID !== id;
  const updatedHits = hits.filter(isNotId);

  this.setState({
    results: {
      ...results,
      [searchKey]: { hits: updatedHits, page }
    }
  });
}

render() {
  const {
    searchTerm,
    results,
    searchKey,
    error,
    isLoading,
    sortKey,
    isSortReverse
  } = this.state;
```

```
    const page = (
      results &&
      results[searchKey] &&
      results[searchKey].page
    ) || 0;

    const list = (
      results &&
      results[searchKey] &&
      results[searchKey].hits
    ) || [];

    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
          >
            Search
          </Search>
        </div>
        { error
          ? <div className="interactions">
            <p>Something went wrong.</p>
          </div>
          : <Table
                list={list}
                  sortKey={sortKey}
                isSortReverse={isSortReverse}
              onSort={this.onSort}
            onDismiss={this.onDismiss}
                />
        }
        <div className="interactions">
          <ButtonWithLoading
            isLoading={isLoading}
            onClick={() => this.fetchSearchTopstories(searchKey, page + 1)}>
            More
          </ButtonWithLoading>
        </div>
      </div>
    );
  }
}

const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
    />
    <button type="submit">
      {children}
    </button>
```

```
  </form>

const Table = ({
  list,
  sortKey,
  isSortReverse,
  onSort,
  onDismiss
}) => {
  const sortedList = SORTS[sortKey](list);
  const reverseSortedList = isSortReverse
    ? sortedList.reverse()
    : sortedList;

  return(
    <div className="table">
      <div className="table-header">
        <span style={{ width: '40%' }}>
          <Sort
            sortKey={'TITLE'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Title
          </Sort>
        </span>
        <span style={{ width: '30%' }}>
          <Sort
            sortKey={'AUTHOR'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Author
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'COMMENTS'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Comments
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'POINTS'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Points
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          Archive
        </span>
      </div>
      { reverseSortedList.map(item =>
        <div key={item.objectID} className="table-row">
          <span style={{ width: '40%' }}>
            <a href={item.url}>{item.title}</a>
          </span>
```

```
          <span style={{ width: '30%' }}>
            {item.author}
          </span>
          <span style={{ width: '10%' }}>
            {item.num_comments}
          </span>
          <span style={{ width: '10%' }}>
            {item.points}
          </span>
          <span style={{ width: '10%' }}>
            <Button
              onClick={() => onDismiss(item.objectID)}
              className="button-inline"
            >
              Dismiss
            </Button>
          </span>
        </div>
      )}
    </div>
  );
}

const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

const Loading = () =>
  <div>Loading ...</div>

const withLoading = (Component) => ({ isLoading, ...rest }) =>
  isLoading ? <Loading /> : <Component { ...rest } />

const ButtonWithLoading = withLoading(Button);

const Sort = ({
  sortKey,
  activeSortKey,
  onSort,
  children
}) => {
  const sortClass = classNames(
    'button-inline',
    { 'button-active': sortKey === activeSortKey }
  );

  return (
    <Button
      onClick={() => onSort(sortKey)}
      className={sortClass}
    >
      {children}
    </Button>
  );
}

export default App;
```

```
export {
  Button,
  Search,
  Table,
};
```

Exercises:

- Use a library like Font Awesome to indicate the (reverse) sort. It could be an arrow up or arrow down icon next to each Sort header

Further Reading

- Read about the classnames library

Recap

This chapter focused on implementing advanced React components. You will learnt how to implement higher-order components, and we dove into more advanced topics in React.

You have learned advanced component techniques in React! Let's recap the chapter:

- **React**
  - The `ref` attribute to reference DOM elements
  - Higher-order components are a common way to build advanced components
  - Implementation of advanced interactions in React
  - Conditional classNames with a neat helper library
- **ES6**
  - Rest destructuring to split up objects and arrays

You can always find the source code in the official repository.

```
import React, { Component } from 'react';
import { sortBy } from 'lodash';
import classNames from 'classnames';
require('./App.css');

const DEFAULT_QUERY = 'redux';
const DEFAULT_HPP = '100';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';

const SORTS = {
  NONE: list => list,
  TITLE: list => sortBy(list, 'title'),
  AUTHOR: list => sortBy(list, 'author'),
  COMMENTS: list => sortBy(list, 'num_comments').reverse(),
  POINTS: list => sortBy(list, 'points').reverse(),
};

class App extends Component {
```

```
  constructor(props) {
    super(props);


    this.state = {
      results: null,
      searchKey: '',
      searchTerm: DEFAULT_QUERY,
      error: null,
      isLoading: false,
      sortKey: 'NONE',
      isSortReverse: false,
    };

    this.needsToSearchTopstories = this.needsToSearchTopstories.bind(this);
    this.setSearchTopstories = this.setSearchTopstories.bind(this);
    this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
    this.onSort = this.onSort.bind(this);
  }

  onSort(sortKey) {
    const isSortReverse = this.state.sortKey === sortKey && !this.state.isSortReverse;
    this.setState({ sortKey, isSortReverse });
  }

  needsToSearchTopstories(searchTerm) {
    return !this.state.results[searchTerm];
  }

  setSearchTopstories(result) {
    const { hits, page } = result;
    const { searchKey, results } = this.state;

    const oldHits = results && results[searchKey]
      ? results[searchKey].hits
      : [];

    const updatedHits = [
      ...oldHits,
      ...hits
    ];

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      },
      isLoading: false
    });
  }

  fetchSearchTopstories(searchTerm, page = 0) {
    this.setState({ isLoading: true });

    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}`)
      .then(response => response.json())
      .then(result => this.setSearchTopstories(result))
        .catch(e => this.setState({ error: e }));
  }
```

```
  componentDidMount() {
    const { searchTerm } = this.state;
    this.setState({ searchKey: searchTerm });
    this.fetchSearchTopstories(searchTerm);
  }

  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  onSearchSubmit(event) {
    const { searchTerm } = this.state;
    this.setState({ searchKey: searchTerm });

    if (this.needsToSearchTopstories(searchTerm)) {
      this.fetchSearchTopstories(searchTerm);
    }

    event.preventDefault();
  }

  onDismiss(id) {
    const { searchKey, results } = this.state;
    const { hits, page } = results[searchKey];

    const isNotId = item => item.objectID !== id;
    const updatedHits = hits.filter(isNotId);

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      }
    });
  }

  render() {
    const {
      searchTerm,
      results,
      searchKey,
      error,
      isLoading,
      sortKey,
      isSortReverse
    } = this.state;

    const page = (
      results &&
      results[searchKey] &&
      results[searchKey].page
    ) || 0;

    const list = (
      results &&
      results[searchKey] &&
      results[searchKey].hits
    ) || [];

    return (
      <div className="page">
        <div className="interactions">
```

```
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
          >
            Search
          </Search>
        </div>
        { error
          ? <div className="interactions">
            <p>Something went wrong.</p>
          </div>
          : <Table
                  list={list}
                sortKey={sortKey}
                  isSortReverse={isSortReverse}
                onSort={this.onSort}
              onDismiss={this.onDismiss}
          />
        }
        <div className="interactions">
          <ButtonWithLoading
            isLoading={isLoading}
            onClick={() => this.fetchSearchTopstories(searchKey, page + 1)}>
            More
          </ButtonWithLoading>
        </div>
      </div>
    );
  }
}

const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
    />
    <button type="submit">
      {children}
    </button>
  </form>

const Table = ({
  list,
  sortKey,
  isSortReverse,
  onSort,
  onDismiss
}) => {
  const sortedList = SORTS[sortKey](list);
  const reverseSortedList = isSortReverse
    ? sortedList.reverse()
    : sortedList;

  return(
```

```
    <div className="table">
      <div className="table-header">
        <span style={{ width: '40%' }}>
          <Sort
            sortKey={'TITLE'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Title
          </Sort>
        </span>
        <span style={{ width: '30%' }}>
          <Sort
            sortKey={'AUTHOR'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Author
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'COMMENTS'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Comments
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'POINTS'}
            onSort={onSort}
            activeSortKey={sortKey}
          >
            Points
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          Archive
        </span>
      </div>
      { reverseSortedList.map(item =>
        <div key={item.objectID} className="table-row">
          <span style={{ width: '40%' }}>
            <a href={item.url}>{item.title}</a>
          </span>
          <span style={{ width: '30%' }}>
            {item.author}
          </span>
          <span style={{ width: '10%' }}>
            {item.num_comments}
          </span>
          <span style={{ width: '10%' }}>
            {item.points}
          </span>
          <span style={{ width: '10%' }}>
            <Button
              onClick={() => onDismiss(item.objectID)}
              className="button-inline"
            >
              Dismiss
```

```
            </Button>
          </span>
        </div>
      )}
    </div>
  );
}

const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

const Loading = () =>
  <div>Loading ...</div>

const withLoading = (Component) => ({ isLoading, ...rest }) =>
  isLoading ? <Loading /> : <Component { ...rest } />

const ButtonWithLoading = withLoading(Button);

const Sort = ({
  sortKey,
  activeSortKey,
  onSort,
  children
}) => {
  const sortClass = classNames(
    'button-inline',
    { 'button-active': sortKey === activeSortKey }
  );

  return (
    <Button
      onClick={() => onSort(sortKey)}
      className={sortClass}
    >
      {children}
    </Button>
  );
}

export default App;

export {
  Button,
  Search,
  Table,
};
```

Lifting State

We have already covered the basics of state management in React in the previous chapters by using React's local state, so this chapter will dig a bit deeper. In this lesson, we'll learn what lifting state in React is and how it is used.

What does ‘Lifting State’ mean?

Only the App component is a stateful ES6 component in your application. It handles a lot of application state and logic in its class methods. Moreover, we pass a lot of properties to the Table component, most of which are only used in there. It’s not important that the App component knows about them, so the sort functionality could be moved into the Table component.

Moving a sub-state from one component to another is known as lifting state. We want to move state that isn’t used in the App component into the Table component, down from parent to child component. To deal with state and class methods in the Table component, it has to become an ES6 class component.

Refactoring the Table component

Refactoring from functional stateless components to ES6 class components is simple. Your Table component as a functional stateless component:

```
const Table = ({
  list,
  sortKey,
  isSortReverse,
  onSort,
  onDismiss
}) => {
  const sortedList = SORTS[sortKey](list);
  const reverseSortedList = isSortReverse
    ? sortedList.reverse()
    : sortedList;

  return(
    ...
  );
}
```

Your Table component as an ES6 class component:

```
class Table extends Component {

  render() {
    const {
      list,
      sortKey,
      isSortReverse,
      onSort,
      onDismiss
    } = this.props;

    const sortedList = SORTS[sortKey](list);
    const reverseSortedList = isSortReverse
      ? sortedList.reverse()
      : sortedList;

    return(
      ...
    );
  }
}
```

Since you want to deal with state and methods in your component, you have to add a constructor and initial state.

```
class Table extends Component {

  constructor(props) {
    super(props);

    this.state = {};
  }

  render() {
    ...
  }
}
```

Now you can move state and class methods regarding the sort functionality from your App component down to your Table component.

```
class Table extends Component {
  constructor(props) {
    super(props);

    this.state = {
      sortKey: 'NONE',
      isSortReverse: false,
    };

    this.onSort = this.onSort.bind(this);
```

```
  }

  onSort(sortKey) {

    const isSortReverse = this.state.sortKey === sortKey &&
      !this.state.isSortReverse;

    this.setState({ sortKey, isSortReverse });
  }

  render() {
    ...
  }
}
```

Don't forget to remove the moved state and `onSort()` class method from your App component.

```
class App extends Component {
  _isMounted = false;

  constructor(props) {
    super(props);

    this.state = {
      results: null,
      searchKey: '',
      searchTerm: DEFAULT_QUERY,
      error: null,
      isLoading: false,
    };

    this.setSearchTopStories = this.setSearchTopStories.bind(this);
    this.fetchSearchTopStories = this.fetchSearchTopStories.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.needsToSearchTopStories = this.needsToSearchTopStories.bind(this);
  }

  ...

}
```

Making the Table component more lightweight

To do this, we move props that are passed to it from the App component, because they are handled internally in the Table component.

```
class App extends Component {

  ...

  render() {
    const {
```

```
      searchTerm,
      results,
      searchKey,

      error,
      isLoading
    } = this.state;

    ...

    return (
      <div className="page">
        ...
        { error
          ? <div className="interactions">
            <p>Something went wrong.</p>
          </div>
          : <Table
            list={list}
            onDismiss={this.onDismiss}
          />
        }
        ...
      </div>
    );
  }
}
```

Now in your Table component you can use the internal `onSort()` method and the internal Table state.

```
class Table extends Component {

  ...

  render() {
    const {
      list,
      onDismiss
    } = this.props;

    const {
      sortKey,
      isSortReverse,
    } = this.state;

    const sortedList = SORTS[sortKey](list);
    const reverseSortedList = isSortReverse
      ? sortedList.reverse()
      : sortedList;

    return(
      <div className="table">
        <div className="table-header">
          <span style={{ width: '40%' }}>
            <Sort
              sortKey={'TITLE'}
              onSort={this.onSort}
              activeSortKey={sortKey}
```

```
            >
              Title
            </Sort>

          </span>
          <span style={{ width: '30%' }}>
            <Sort
              sortKey={'AUTHOR'}
              onSort={this.onSort}
              activeSortKey={sortKey}
            >
              Author
            </Sort>
          </span>
          <span style={{ width: '10%' }}>
            <Sort
              sortKey={'COMMENTS'}
              onSort={this.onSort}
              activeSortKey={sortKey}
            >
              Comments
            </Sort>
          </span>
          <span style={{ width: '10%' }}>
            <Sort
              sortKey={'POINTS'}
              onSort={this.onSort}
              activeSortKey={sortKey}
            >
              Points
            </Sort>
          </span>
          <span style={{ width: '10%' }}>
            Archive
          </span>
        </div>
        { reverseSortedList.map((item) =>
          ...
        )}
      </div>
    );
  }
}
```

We made a crucial refactoring by moving functionality and state closer into another component, and other components got more lightweight. Again, the component API of the Table got lighter because it deals internally with the sort functionality.

Lifting state can go the other way as well: from child to parent component. It is called as lifting state up. Imagine you were dealing with local state in a child component, and you want to fulfil a requirement to show the state in your parent component as well. You would have to lift up the state to your parent component. Moreover, imagine you want to show the state in a sibling component of your child component. Again, you would lift the state up to your

parent component. The parent component deals with the local state, but exposes it to both child components.

```
import React, { Component } from 'react';
import { sortBy } from 'lodash';
import classNames from 'classnames';
require('./App.css');

const DEFAULT_QUERY = 'redux';
const DEFAULT_HPP = '100';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';

const SORTS = {
  NONE: list => list,
  TITLE: list => sortBy(list, 'title'),
  AUTHOR: list => sortBy(list, 'author'),
  COMMENTS: list => sortBy(list, 'num_comments').reverse(),
  POINTS: list => sortBy(list, 'points').reverse(),
};

class App extends Component {

  constructor(props) {
    super(props);

    this.state = {
      results: null,
      searchKey: '',
      searchTerm: DEFAULT_QUERY,
      error: null,
      isLoading: false,
    };

    this.needsToSearchTopstories = this.needsToSearchTopstories.bind(this);
    this.setSearchTopstories = this.setSearchTopstories.bind(this);
    this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  needsToSearchTopstories(searchTerm) {
    return !this.state.results[searchTerm];
  }

  setSearchTopstories(result) {
    const { hits, page } = result;
    const { searchKey, results } = this.state;

    const oldHits = results && results[searchKey]
      ? results[searchKey].hits
      : [];

    const updatedHits = [
      ...oldHits,
      ...hits
    ];
```

```
    this.setState({
      results: {

        ...results,
        [searchKey]: { hits: updatedHits, page }
      },
      isLoading: false
    });
  }

  fetchSearchTopstories(searchTerm, page = 0) {
    this.setState({ isLoading: true });

    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}`)
      .then(response => response.json())
      .then(result => this.setSearchTopstories(result))
        .catch(e => this.setState({ error: e }));
  }

  componentDidMount() {
    const { searchTerm } = this.state;
    this.setState({ searchKey: searchTerm });
    this.fetchSearchTopstories(searchTerm);
  }

  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  onSearchSubmit(event) {
    const { searchTerm } = this.state;
    this.setState({ searchKey: searchTerm });

    if (this.needsToSearchTopstories(searchTerm)) {
      this.fetchSearchTopstories(searchTerm);
    }

    event.preventDefault();
  }

  onDismiss(id) {
    const { searchKey, results } = this.state;
    const { hits, page } = results[searchKey];

    const isNotId = item => item.objectID !== id;
    const updatedHits = hits.filter(isNotId);

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      }
    });
  }

  render() {
    const {
      searchTerm,
      results,
      searchKey,
      error,
      isLoading
```

```
    } = this.state;

    const page = (
      results &&
      results[searchKey] &&
      results[searchKey].page
    ) || 0;

    const list = (
      results &&
      results[searchKey] &&
      results[searchKey].hits
    ) || [];

    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
          >
            Search
          </Search>
        </div>
        { error
          ? <div className="interactions">
            <p>Something went wrong.</p>
          </div>
          : <Table
                  list={list}
                  onDismiss={this.onDismiss}
                />
        }
        <div className="interactions">
          <ButtonWithLoading
            isLoading={isLoading}
            onClick={() => this.fetchSearchTopstories(searchKey, page + 1)}>
            More
          </ButtonWithLoading>
        </div>
      </div>
    );
  }
}

const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
    />
    <button type="submit">
      {children}
    </button>
  </form>
```

```
class Table extends Component {

  constructor(props) {
    super(props);

    this.state = {
      sortKey: 'NONE',
      isSortReverse: false,
    };

    this.onSort = this.onSort.bind(this);
  }

  onSort(sortKey) {
    const isSortReverse = this.state.sortKey === sortKey && !this.state.isSortReverse;
    this.setState({ sortKey, isSortReverse });
  }

  render() {
    const {
      list,
      onDismiss
    } = this.props;

    const {
      sortKey,
      isSortReverse,
    } = this.state;

    const sortedList = SORTS[sortKey](list);
    const reverseSortedList = isSortReverse
      ? sortedList.reverse()
      : sortedList;

    return(
      <div className="table">
        <div className="table-header">
          <span style={{ width: '40%' }}>
            <Sort
              sortKey={'TITLE'}
              onSort={this.onSort}
              activeSortKey={sortKey}
            >
              Title
            </Sort>
          </span>
          <span style={{ width: '30%' }}>
            <Sort
              sortKey={'AUTHOR'}
              onSort={this.onSort}
              activeSortKey={sortKey}
            >
              Author
            </Sort>
          </span>
          <span style={{ width: '10%' }}>
            <Sort
              sortKey={'COMMENTS'}
              onSort={this.onSort}
              activeSortKey={sortKey}
            >
```

```
              Comments
            </Sort>
          </span>
          <span style={{ width: '10%' }}>
            <Sort
              sortKey={'POINTS'}
              onSort={this.onSort}
              activeSortKey={sortKey}
            >
              Points
            </Sort>
          </span>
          <span style={{ width: '10%' }}>
            Archive
          </span>
        </div>
        { reverseSortedList.map(item =>
          <div key={item.objectID} className="table-row">
            <span style={{ width: '40%' }}>
              <a href={item.url}>{item.title}</a>
            </span>
            <span style={{ width: '30%' }}>
              {item.author}
            </span>
            <span style={{ width: '10%' }}>
              {item.num_comments}
            </span>
            <span style={{ width: '10%' }}>
              {item.points}
            </span>
            <span style={{ width: '10%' }}>
              <Button
                onClick={() => onDismiss(item.objectID)}
                className="button-inline"
              >
                Dismiss
              </Button>
            </span>
          </div>
        )}
      </div>
    );
  }
}

const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

const Loading = () =>
  <div>Loading ...</div>

const withLoading = (Component) => ({ isLoading, ...rest }) =>
  isLoading ? <Loading /> : <Component { ...rest } />

const ButtonWithLoading = withLoading(Button);
```

```
const Sort = ({
  sortKey,
  activeSortKey,
  onSort,
  children
}) => {
  const sortClass = classNames(
    'button-inline',
    { 'button-active': sortKey === activeSortKey }
  );

  return (
    <Button
      onClick={() => onSort(sortKey)}
      className={sortClass}
    >
      {children}
    </Button>
  );
}

export default App;

export {
  Button,
  Search,
  Table,
};
```

Exercises:

- read more about lifting state in React
- read more about lifting state in learn React before using Redux

Revisited: setState()

In this chapter we revisit the React setState() method. We learn that setState() is asynchronous and not only takes objects as arguments but also takes functions.

`setState()` can take functions as input

So far, we have used React `setState()` to manage your internal component state. We can pass an object to the function where it updates partially the local state.

```
this.setState({ value: 'hello'});
```

But `setState()` doesn't take only an object. In its second version, you can pass a function to update the state.

```
this.setState((prevState, props) => {
  ...
});
```

Use cases of functions as input

There is one crucial case where it makes sense to use a function over an object: when you update the state depending on the previous state or props. If you don't use a function, the local state management can cause bugs. The React `setState()` method is asynchronous. React batches `setState()` calls and executes them eventually. Sometimes, the previous state or props changes between before we can rely on it in our `setState()` call.

```
const { oneCount } = this.state;
const { anotherCount } = this.props;
this.setState({ count: oneCount + anotherCount });
```

Imagine that `oneCount` and `anotherCount`, thus the state or the props, change somewhere else asynchronously when you call `setState()`. In a growing

application, you have more than one `setState()` call across your application.

Since `setState()` executes asynchronously, you could rely in the example on stale values.

With the function approach, the function in `setState()` is a callback that operates on the state and props at the time of executing the callback function. Even though `setState()` is asynchronous, with a function it takes the state and props at the time when it is executed.

```
this.setState((prevState, props) => {
  const { oneCount } = prevState;
  const { anotherCount } = props;
  return { count: oneCount + anotherCount };
});
```

In our code, the `setSearchTopStories()` method relies on the previous state, and this is a good example to use a function over an object in `setState()`. Right now, it looks like the following code:

```
setSearchTopStories(result) {
  const { hits, page } = result;
  const { searchKey, results } = this.state;

  const oldHits = results && results[searchKey]
    ? results[searchKey].hits
    : [];

  const updatedHits = [
    ...oldHits,
    ...hits
  ];

  this.setState({
    results: {
      ...results,
      [searchKey]: { hits: updatedHits, page }
    },
    isLoading: false
  });
}
```

Here, we extracted values from the state, but updated the state depending on the previous state asynchronously. Now we'll use the functional approach to prevent bugs from a stale state:

```
setSearchTopStories(result) {
```

```
  const { hits, page } = result;

  this.setState(prevState => {

    ...
  });
}
```

We can move the whole block we implemented into the function by directing it to operate on the `prevState` instead of the `this.state`.

```
setSearchTopStories(result) {
  const { hits, page } = result;

  this.setState(prevState => {
    const { searchKey, results } = prevState;

    const oldHits = results && results[searchKey]
      ? results[searchKey].hits
      : [];

    const updatedHits = [
      ...oldHits,
      ...hits
    ];

    return {
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      },
      isLoading: false
    };
  });
}
```

That will fix the issue with a stale state, but there is still one more improvement. Since it is a function, you can extract the function for improved readability. One more advantage to use a function over an object is that function can live outside of the component. We still have to use a higher-order function to pass the result to it since we want to update the state based on the fetched result from the API.

```
setSearchTopStories(result) {
  const { hits, page } = result;
  this.setState(updateSearchTopStoriesState(hits, page));
}
```

The `updateSearchTopStoriesState()` function has to return a function. It is a higher-order function that can be defined outside the App component. Note

how the function signature changes slightly now.

```
const updateSearchTopStoriesState = (hits, page) => (prevState) => {
  const { searchKey, results } = prevState;

  const oldHits = results && results[searchKey]
    ? results[searchKey].hits
    : [];

  const updatedHits = [
    ...oldHits,
    ...hits
  ];

  return {
    results: {
      ...results,
      [searchKey]: { hits: updatedHits, page }
    },
    isLoading: false
  };
};

class App extends Component {
  ...
}
```

The function instead of object approach in `setState()` fixes potential bugs, while increasing the readability and maintainability of your code. Further, it becomes testable outside of the App component. I advise exporting and testing it as practice.

Exercises:

- Export `updateSearchTopStoriesState` from the file. Write a test for it which passes the a payload (hits, page) and a made up previous state and finally expect a new state
- Refactor your `setState()` methods to use a function, but only when it makes sense, because it relies on props or state
- Run your tests again and verify that everything is up to date

Further Reading

- read more about React using state correctly

Quick quiz on setState()!

1 `setState()` runs synchronously.

COMPLETED 0%

1 of 2

Taming the State

In this lesson you learn why you would consider using a third-party state management library such as Redux or MobX.

When do you need state management?

Previous chapters have shown you that state management can be a crucial topic in larger applications, as React and a lot of other SPA frameworks struggle with it. As applications get more complex, the big challenge in web applications is to tame and control the state.

How React Makes State Management simple

Compared to other solutions, React has already taken a big step forward. A unidirectional data flow and a simple API to manage state in components is indispensable. These concepts make it easier to reason about your state and your state changes. It also makes it easier to reason about it on a component level and on an application level to a certain degree.

It is possible to introduce bugs by operating on stale state when using an object over a function in `setState()`. We lift state around to share or hide necessary state across components. Sometimes a component needs to lift up state, because its sibling component depends on it. Perhaps the component is far away in the component tree, so the stated needs to be shared across the whole component tree. Components are more involved in state management, as the main responsibility of components is representing the UI.

Because of this, there are standalone solutions to take care of state management. Libraries like Redux or MobX are both feasible solutions in a React application. They come with extensions, react-redux and mobx-react, to integrate them into the React view layer. Redux and MobX are outside of the scope of this course, but I encourage you to study the different ways to handle scaling state management as your React applications become more complex.

Further Reading:

Further Reading:

- Read about external state management and how to learn it
- Check out my second book about state management in React called Taming the State in React

Recap

Congratulations, you have learned to bootstrap your first React application! This lesson will provide you a recap to what you have learned so far and will also share how your App.js code looks by now.

Let's recap what we have covered in this chapter:

- **React**
  - Create-react-app bootstraps a React application
  - JSX mixes up HTML and JavaScript to define the output of React components in their render methods
  - Components, instances, and elements are different items in React
  - `ReactDOM.render()` is an entry point for a React application to hook React into the DOM
  - Built-in JavaScript functionalities can be used in JSX
  - Map can be used to render a list of items as HTML elements
- **ES6**
  - Variable declarations with `const` and `let` can be used for specific use cases
  - Use const over let in React applications
  - Arrow functions can be used to keep your functions concise
  - Classes are used to define components in React by extending them

Now that you've completed the first chapter, it's advisable to experiment with the source code you have written so far and see what changes you can make on your own. You can find the source code in the official repository.

Your `src/App.js` should look like the following by now:

```
import React, { Component } from 'react';
require('./App.css');

const list = [
  {
    title: 'React',
```

```
    url: 'https://reactjs.org/',
    author: 'Jordan Walke',
    num_comments: 3,
    points: 4,
    objectID: 0,
  },
  {
    title: 'Redux',
    url: 'https://redux.js.org/',
    author: 'Dan Abramov, Andrew Clark',
    num_comments: 2,
    points: 5,
    objectID: 1,
  },
];

class App extends Component {
  render() {
    return (
      <div className="App">
        { list.map(item =>
          <div key={item.objectID}>
            <span>
              <a href={item.url}>{item.title}</a>
            </span>
            <span>{item.author}</span>
            <span>{item.num_comments}</span>
            <span>{item.points}</span>
          </div>
        )}
      </div>
    );
  }
}

export default App;
```

Eject

The last chapters will show you how to deploy your application to production. We will uncover more about how to deploy applications in the create-react-app in this lesson.

The following knowledge is not necessary to deploy your application to production, but it still bears mentioning. *create-react-app* comes with one feature to keep it extendable, but also to prevent a vendor lock-in. A vendor lock-in usually happens when you buy into a technology but there is no escape hatch from using it in the future. Fortunately, in *create-react-app* you have such an escape hatch with "eject".

In your package.json you will find the scripts to *start*, *test*, and *build* your application. The last script is *eject*. However, it is important you know that it is a **one-way operation**. That means **once you eject, you can't go back!** It is advisable to stay in the safe environment of *create-react-app* if you have just started creating applications in React.

If you feel comfortable enough to run `npm run eject`, the command copies all the configuration and dependencies to your *package.json* and a new *config/* folder. It converts the whole project into a custom setup with tooling that includes Babel and Webpack, and grants full control over all these tools.

Its official documentation says create-react-app is suitable for small to middle sized projects, so you shouldn't feel obligated to use the "eject" command until you're ready.

Further Reading

- Read about eject

Deploy your App

In this lesson, we'll see how the free hosting service Heroku is used to deploy a React application

No application should stay on localhost; eventually, it has to go live to see how it will perform in real-life scenarios. Heroku is a platform as a service where you can host your application, and it offers seamless integration with React. Specifically, it's possible to deploy a create-react-app in minutes, with a a zero-configuration deployment which follows the philosophy of create-react-app.

There are two requirements before an application can be deployed to Heroku:

- install the Heroku CLI
- create a free Heroku account

If you have installed Homebrew, you can install the Heroku CLI from command line:

```
brew update
brew install heroku-toolbelt
```

Now you can use git and Heroku CLI to deploy your application:

```
git init
heroku create -b https://github.com/mars/create-react-app-buildpack.git
git add .
git commit -m "react-create-app on Heroku"
git push heroku master
heroku open
```

That's all there is to it. If you run into problems, check these resources for troubleshooting options:

- Git and GitHub Essentials
- Deploying React with Zero Configuration
- Heroku Buildpack for create-react-app

Now you can use git and Heroku CLI to deploy your application.

Requirements

To get a better understanding of this course, we are listing down some pre-requisites for you.

To follow this course, you should be familiar with the basics of web development, i.e how to use HTML, CSS, and JavaScript. It also makes sense to understand how APIs work, as they will be covered thoroughly. Also, I encourage you to join the official Slack Group to be a part of a growing React community where you can learn from and help others.

Editor and Terminal

For the lessons, you will need a text editor or an IDE and terminal (command line tool). I have provided a setup guide if you need additional help. Optionally, we recommend you keep your projects in GitHub while conducting the exercises in this course. There is a short guide on how to use these tools. Github has excellent version control, so you can see what changes were made if you make a mistake or just want a more direct way to follow along.

Node and NPM

Finally, you will need an installation of node and npm. Both are used to manage libraries you will need along the way. In this course, you will install external node packages via npm (node package manager). These node packages can be libraries or whole frameworks.

You can verify your versions of node and npm on the command line. If you don't get any output in the terminal, you need to install node and npm first. These are my versions at the time of writing this course:

```
node --version
*v8.9.4
```

```
npm --version
*v5.6.0
```

npm and node (Optional)

This lesson is a crash course in node and npm. It is not exhaustive, but it will cover all of the necessary tools. If you are familiar with both of them, you can skip this section.

The *node package manager* (npm) installs external node packages from the command line. These packages can be a set of utility functions, libraries, or whole frameworks, and they are the dependencies of your application. You can either install these packages to your global node package folder, or to your local project folder.

Global node packages are accessible from everywhere in the terminal, and only need to be installed to the global directory once. Install a global package by typing the following into a terminal:

```
npm install -g <package>
```

The `-g` flag tells npm to install the package globally. Local packages are used in your application by default. For our purposes, we will install React to the local directory terminal by typing:

```
npm install react
```

The installed package will automatically appear in a folder called *node_modules/* and will be listed in the *package.json* file next to your other dependencies.

To initialize the *node_modules/* folder and the *package.json* file for your project, use the following npm command. Then, you can install new local packages via npm:

```
npm init -y
```

The `-y` flag initializes all the defaults in your *package.json*. After initializing

The `-y` flag initializes all the defaults in your *package.json*. After initializing your npm project, you are ready to install new packages via `npm install <package>`.

The *package.json* file allows you to share your project with other developers without sharing all the node packages. It will contain references to all node packages used in your project, called **dependencies**. Other users can copy a project without the dependencies using the references in *package.json*, where the references make it easy to install all packages using `npm install`. A `npm install` script will take all the dependencies listed in the *package.json* file and install them in the *node_modules/* folder.

Finally, there's one more command to cover about npm:

```
npm install --save-dev <package>
```

The `--save-dev` flag indicates that the node package is only used in the development environment, meaning it won't be used in when the application is deployed to a the server or used in production. It is useful for testing an application using a node package, but want to exclude it from your production environment.

Some of you may want to use other package managers to work with node packages in your applications. **Yarn** is a dependency manager that works similar to **npm**. It has its own list of commands, but you still have access to the same npm registry. Yarn was created to solve issues npm couldn't, but both tools have evolved to the point where either will suffice today.

Exercises:

- Set up a throw away npm project using the terminal:
  - Create a new folder with `mkdir <folder_name>`
  - Navigate into the folder with `cd <folder_name>`
  - Execute `npm init -y` or `npm init`
  - Install a local package like React with `npm install react`
  - Check the *package.json* file and the *node_modules/* folder
  - Attempt to uninstall and reinstall the *react* node package

Further Readings:

- Read about npm
- Read about yarn package manager

Installation

This lesson explains how to get started with building a React application. You will also learn about what CDN is how you can add React to your application through CDN.

There are many approaches to getting started with a React application. The first we'll explore is a CDN, short for content delivery network. Don't worry too much about CDNs now, because you will not use them in this course, but it makes sense to explain them briefly. Many companies use CDNs to host files publicly for their consumers. Some of these files are libraries like React, since the bundled React library is just a *react.js* JavaScript file.

To get started with React by using a CDN, find the `<script>` tag in your web page HTML that points to a CDN URL. You will need two libraries: *react* and *react-dom*.

```
<script
  crossorigin
  src="https://unpkg.com/react@16/umd/react.development.js"
></script>

<script
  crossorigin
  src="https://unpkg.com/react-dom@16/umd/react-dom.development.js"
></script>
```

You can also get React into your application by initializing it as node project. With a *package.json* file, you can install *react* and *react-dom* from the command line. However, the folder must be initialized as a npm project using `npm init -y` with a *package.json* file. You can install multiple node packages with npm:

```
npm install react react-dom
```

This approach is often used to add React to an existing application managed with npm.

You may also have to deal with Babel to make your application aware of JSX (the React syntax) and JavaScript ES6. Babel transpiles your code–that is, it converts it to vanilla JavaScript–so most modern browsers can interpret JavaScript ES6 and JSX. Because of this difficult setup, Facebook introduced *create-react-app* as a zero-configuration React solution. The next section will show you how to set up your application using this bootstrapping tool.

Further Reading:

- Read about React installations

Zero-Configuration Setup

This section will show you how to set up your application using create-react-app.

In the Road to learn React, we will be using create-react-app to bootstrap your application. It's an opinionated yet zero-configuration starter kit for React introduced by Facebook in 2016, recommended for beginners by 96% of React users. In *create-react-app* the tooling and configuration evolve in the background, while the focus is on the application implementation.

To get started, install the package to your global node packages, which keeps it available on the command line to bootstrap new React applications:

```
npm install -g create-react-app
```

You can check the version of *create-react-app* to verify a successful installation on your command line:

```
create-react-app --version
*v1.5.1
```

Now you are ready to bootstrap your first React application. The example will be referred to as *hackernews*, but you may choose any name you like. First, navigate into the folder:

```
create-react-app hackernews
cd hackernews
```

Now you can open the application in your editor. The following folder structure, or a variation of it depending on the *create-react-app* version, should be presented to you:

```
hackernews/
  README.md
```

```
node_modules/
package.json
.gitignore

public/
  favicon.ico
  index.html
  manifest.json
src/
  App.css
  App.js
  App.test.js
  index.css
  index.js
  logo.svg
  registerServiceWorker.js
```

Breaking down

This is a breakdown of the folders and files:

- **README.md:** The .md extension indicates the file is a markdown file. Markdown is used as a lightweight markup language with plain text formatting syntax. Many source code projects come with a *README.md* file to give you initial instructions about the project. When pushing your project to a platform such as GitHub, the *README.md* file usually displays information about the content contained in the repository. Because you used *create-react-app*, your *README.md* should be the same as the official create-react-app GitHub repository.

- **node_modules/:** This folder contains all node packages that have been installed via npm. Since you used *create-react-app*, there should already be a couple of node modules installed for you. You will rarely touch this folder, because node packages are generally installed and uninstalled with npm from the command line.

- **package.json:** This file shows you a list of node package dependencies and other project configurations.

- **.gitignore:** This file displays all files and folders that shouldn't be added to your git repository when using git; such files and folders should only be located in your local project. The *node_modules/* folder is one example. It is enough to share the *package.json* file with others, so they can install dependencies on their end with `npm install` without your dependency folder.

- **public/:** This folder holds development files, such as *public/index.html*. The index is displayed on localhost:3000 when developing your app. The boilerplate takes care of relating this index with all the scripts in *src/*.
- **build/** This folder is created when you build the project for production, as it holds all of the production files. When building your project for production, all the source code in the *src/* and *public/* folders are bundled and placed in the build folder.
- **manifest.json** and **registerServiceWorker.js:** These files won't be used for this project, so you can ignore them for now.

In the beginning, everything you need is located in the *src/* folder. The main focus lies on the *src/App.js* file which is used to implement React components. It will be used to implement your application, but later you might want to split up your components into multiple files, where each file maintains one or more components on its own.

Additionally, you will find a *src/App.test.js* file for your tests, and a *src/index.js* as an entry point to the React world. You will get to know both files intimately in a later chapter. There is also a *src/index.css* and a *src/App.css* file to style your general application and components, which comes with the default style when you open them. You will modify them later as well.

The *create-react-app* application is a npm project you can use to install and uninstall node packages. It comes with the following npm scripts for your command line:

```
# Runs the application in http://localhost:3000
npm start

# Runs the tests
npm test

# Builds the application for production
npm run build
```

The scripts are defined in your *package.json*, and your basic React application is bootstrapped. The following exercises will finally allow you to run your bootstrapped application in a browser.

Exercises:

- `npm start` your application and visit the application in your browser (Exit the command by pressing Control + C)
- Run the `npm test` script
- Run the `npm run build` script and verify that a *build/* folder was added to your project (you can remove it afterward. Note that the build folder can be used later on to deploy your application)
- Familiarize yourself with the folder structure
- Check the content of the files

Further Reading:

- Read about npm scripts and create-react-app

Introduction to JSX

Get to know the basics of JSX and its syntax in React. You will also learn more about the App component and where to use it.

As mentioned before, *create-react-app* has already bootstrapped a basic application for you, and all files come with their own default implementations.

For now, the only file we will modify is the `src/App.js` file.

```
import React, { Component } from 'react';
import logo from './logo.svg';
import './App.css';

class App extends Component {
  render() {
    return (
      <div className="App">
        <header className="App-header">
          <img src={logo} className="App-logo" alt="logo" />
          <h1 className="App-title">Welcome to React</h1>
        </header>
        <p className="App-intro">
          To get started, edit <code>src/App.js</code> and save to reload.
        </p>
      </div>
    );
  }
}

export default App;
```

Don't worry if you're confused by the import/export statements and class declaration now. These are features of JavaScript ES6 we will revisit in a later chapter.

In the file you should see a **React ES6 class component** with the name App. This is a component declaration. After you have declared a component, you can use it as an element anywhere in your application. It will produce an **instance** of your **component** or, in other words, the component gets instantiated.

```
// component declaration
class App extends Component {
  ...
}

// component usage (also called instantiation for a class)
// creates an instance of the component
<App />
```

The returned **element** is specified in the `render()` method. The components you instantiated earlier are made up of elements, so it is important to understand the differences between a component, an instance of a component, and an element.

You should see where the App component is instantiated, else you couldn't see the rendered output in a browser. The App component is only the declaration, but not the usage. You can instantiate the component anywhere in your JSX with `<App />`. You will see later where this happens in this application.

The content in the render block may look similar to HTML, but it is actually JSX. JSX allows you to mix HTML and JavaScript. It is powerful, but it can be confusing when you are used to separating the two languages. It is a good idea to start by using basic HTML in your JSX. Open the `App.js` file and remove all unnecessary HTML code as shown:

```
import React, { Component } from 'react';
import './App.css';

class App extends Component {
  render() {
    return (
      <div className="App">
        <h2>Welcome to the Road to learn React</h2>
      </div>
    );
  }
}

export default App;
```

Now, you only return HTML in your `render()` method without any JavaScript. Let's define the "Welcome to the Road to learn React" as a variable. A variable is set in JSX by curly braces.

```
import React, { Component } from 'react';
import './App.css';


class App extends Component {
  render() {
    var helloWorld = 'Welcome to the Road to learn React';
    return (
      <div className="App">
        <h2>{helloWorld}</h2>
      </div>
    );
  }
}

export default App;
```

Start your application on the command line with `npm start` to verify the changes you've made.

You might have noticed the `className` attribute. It reflects the standard `class` attribute in HTML. JSX had replaced a handful of internal HTML attributes, but you can find all the supported HTML attributes in React's documentation, which all follow the camelCase convention. On your way to learn React, expect to run across more JSX specific attributes.

Exercises:

- Define more variables and render them in JSX
  - Use a complex object to represent a user with a first name and last name
  - Render the user properties in JSX

Further Readings:

- Read about JSX
- Read about React components, elements and instances

ReactDOM

A brief introduction to ReactDOM and render() method in React application.

The App component is located in your entry point to the React world: the *src/index.js* file.

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';
import './index.css';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);
```

`ReactDOM.render()` uses a DOM node in your HTML to replace it with JSX. It's a way to integrate React in any foreign application easily, and you can use `ReactDOM.render()` multiple times across your application. You can use it to bootstrap simple JSX syntax, a React component, multiple React components, or an entire application. In a plain React application, you would only use it once to bootstrap the component tree.

`ReactDOM.render()` expects two arguments. The first argument is for rendering the JSX. The second argument specifies the place where the React application hooks into your HTML. It expects an element with an `id='root'`, found in the *public/index.html* file.

```
ReactDOM.render(
  <h1>Hello React World</h1>,
  document.getElementById('root')
);
```

During implementation, `ReactDOM.render()` takes your App component, though it can also pass simple JSX. It doesn't require a component instance.

Exercises:

- Open the *public/index.html* to see where the React application hooks into your HTML

Futher Reading:

- Read about rendering elements in React

Hot Module Replacement

A brief introduction to Hot Module Replacement and its use in React App.

Hot Module Replacement (HMR) is a tool for reloading your application in the browser without the page refresh. Hot Module Replacement can be used in the `src/index.js` file to improve your experience as a developer. By default, *create-react-app* will cause the browser to refresh the page whenever its source code is modified. Try it by changing the `helloWorld` variable in your `src/App.js` file, which should cause the browser to refresh the page. There is a better way of handling source code changes during development, however.

You can activate HMR in *create-react-app* by adding the following configuration to your `src/index.js` file:

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';
import './index.css';

ReactDOM.render(
  <App />,
  document.getElementById('root')
);

if (module.hot) {
  module.hot.accept();
}
```

Again, change the `helloWorld` variable in your `src/App.js` file. The browser shouldn't refresh, but the application will reload and show the correct output. HMR comes with multiple advantages:

Imagine you are debugging your code with `console.log()` statements. These statements will stay in your developer console, even though you changed your code because the browser doesn't refresh the page anymore. In a growing application, page refreshes delay productivity; HMR removes this obstacle by eliminating the incremental time loss it takes for a browser to reload.

The most useful benefit of HMR is that you can keep the application state after the application reloads. For instance, assume you have a dialog or wizard in your application with multiple steps, and you are on step 3. Without HMR, you make changes to the source code and your browser refreshes the page. You would then have to open the dialog again and navigate from step 1 to step 3 each time. With HMR your dialog stays open at step 3, so you can debug from the exact point you're working on. With the time saved from page loads, this makes HMR an invaluable tool for React developers.

Exercises:

- Change your `src/App.js` source code a few times to see HMR in action

Further Reading:

- Watch the first 10 minutes of Live React: Hot Reloading with Time Travel by Dan Abramov

Recap

This chapter will dug a deeper at state management in React than using React's local state. It expanded on the best practices, how to apply them, and why you could consider using a third-party state management library.

Your *src/App.js* should look like the following by now:

```
import React, { Component } from 'react';
import { sortBy } from 'lodash';
import classNames from 'classnames';
require('./App.css');

const DEFAULT_QUERY = 'redux';
const DEFAULT_HPP = '100';

const PATH_BASE = 'https://hn.algolia.com/api/v1';
const PATH_SEARCH = '/search';
const PARAM_SEARCH = 'query=';
const PARAM_PAGE = 'page=';

const SORTS = {
  NONE: list => list,
  TITLE: list => sortBy(list, 'title'),
  AUTHOR: list => sortBy(list, 'author'),
  COMMENTS: list => sortBy(list, 'num_comments').reverse(),
  POINTS: list => sortBy(list, 'points').reverse(),
};

const updateSearchTopstoriesState = (hits, page) => (prevState) => {
  const { searchKey, results } = prevState;

  const oldHits = results && results[searchKey]
    ? results[searchKey].hits
    : [];

  const updatedHits = [
    ...oldHits,
    ...hits
  ];

  return {
    results: {
      ...results,
      [searchKey]: { hits: updatedHits, page }
    },
    isLoading: false
  };
};

class App extends Component {
```

```
  constructor(props) {
    super(props);

    this.state = {
      results: null,
      searchKey: '',
      searchTerm: DEFAULT_QUERY,
      error: null,
      isLoading: false,
    };

    this.needsToSearchTopstories = this.needsToSearchTopstories.bind(this);
    this.setSearchTopstories = this.setSearchTopstories.bind(this);
    this.fetchSearchTopstories = this.fetchSearchTopstories.bind(this);
    this.onSearchChange = this.onSearchChange.bind(this);
    this.onSearchSubmit = this.onSearchSubmit.bind(this);
    this.onDismiss = this.onDismiss.bind(this);
  }

  needsToSearchTopstories(searchTerm) {
    return !this.state.results[searchTerm];
  }

  setSearchTopstories(result) {
    const { hits, page } = result;
    this.setState(updateSearchTopstoriesState(hits, page));
  }

  fetchSearchTopstories(searchTerm, page = 0) {
    this.setState({ isLoading: true });

    fetch(`${PATH_BASE}${PATH_SEARCH}?${PARAM_SEARCH}${searchTerm}&${PARAM_PAGE}${page}`)
      .then(response => response.json())
      .then(result => this.setSearchTopstories(result))
        .catch(e => this.setState({ error: e }));
  }

  componentDidMount() {
    const { searchTerm } = this.state;
    this.setState({ searchKey: searchTerm });
    this.fetchSearchTopstories(searchTerm);
  }

  onSearchChange(event) {
    this.setState({ searchTerm: event.target.value });
  }

  onSearchSubmit(event) {
    const { searchTerm } = this.state;
    this.setState({ searchKey: searchTerm });

    if (this.needsToSearchTopstories(searchTerm)) {
      this.fetchSearchTopstories(searchTerm);
    }

    event.preventDefault();
  }

  onDismiss(id) {
    const { searchKey, results } = this.state;
    const { hits, page } = results[searchKey];
```

```
    const isNotId = item => item.objectID !== id;
    const updatedHits = hits.filter(isNotId);

    this.setState({
      results: {
        ...results,
        [searchKey]: { hits: updatedHits, page }
      }
    });
  }

  render() {
    const {
      searchTerm,
      results,
      searchKey,
      error,
      isLoading
    } = this.state;

    const page = (
      results &&
      results[searchKey] &&
      results[searchKey].page
    ) || 0;

    const list = (
      results &&
      results[searchKey] &&
      results[searchKey].hits
    ) || [];

    return (
      <div className="page">
        <div className="interactions">
          <Search
            value={searchTerm}
            onChange={this.onSearchChange}
            onSubmit={this.onSearchSubmit}
          >
            Search
          </Search>
        </div>
        { error
          ? <div className="interactions">
            <p>Something went wrong.</p>
          </div>
          : <Table
                list={list}
                 onDismiss={this.onDismiss}
              />
        }
        <div className="interactions">
          <ButtonWithLoading
            isLoading={isLoading}
            onClick={() => this.fetchSearchTopstories(searchKey, page + 1)}>
            More
          </ButtonWithLoading>
        </div>
      </div>
    );
```

```
  }
}

const Search = ({
  value,
  onChange,
  onSubmit,
  children
}) =>
  <form onSubmit={onSubmit}>
    <input
      type="text"
      value={value}
      onChange={onChange}
    />
    <button type="submit">
      {children}
    </button>
  </form>

class Table extends Component {

  constructor(props) {
    super(props);

    this.state = {
      sortKey: 'NONE',
      isSortReverse: false,
    };

    this.onSort = this.onSort.bind(this);
  }

  onSort(sortKey) {
    const isSortReverse = this.state.sortKey === sortKey && !this.state.isSortReverse;
    this.setState({ sortKey, isSortReverse });
  }

  render() {
    const {
      list,
      onDismiss
    } = this.props;

    const {
      sortKey,
      isSortReverse,
    } = this.state;

    const sortedList = SORTS[sortKey](list);
    const reverseSortedList = isSortReverse
      ? sortedList.reverse()
      : sortedList;

    return(
      <div className="table">
        <div className="table-header">
          <span style={{ width: '40%' }}>
            <Sort
              sortKey={'TITLE'}
              onSort={this.onSort}
              activeSortKey={sortKey}
```

```
          >
            Title
          </Sort>
        </span>
        <span style={{ width: '30%' }}>
          <Sort
            sortKey={'AUTHOR'}
            onSort={this.onSort}
            activeSortKey={sortKey}
          >
            Author
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'COMMENTS'}
            onSort={this.onSort}
            activeSortKey={sortKey}
          >
            Comments
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          <Sort
            sortKey={'POINTS'}
            onSort={this.onSort}
            activeSortKey={sortKey}
          >
            Points
          </Sort>
        </span>
        <span style={{ width: '10%' }}>
          Archive
        </span>
      </div>
      { reverseSortedList.map(item =>
        <div key={item.objectID} className="table-row">
          <span style={{ width: '40%' }}>
            <a href={item.url}>{item.title}</a>
          </span>
          <span style={{ width: '30%' }}>
            {item.author}
          </span>
          <span style={{ width: '10%' }}>
            {item.num_comments}
          </span>
          <span style={{ width: '10%' }}>
            {item.points}
          </span>
          <span style={{ width: '10%' }}>
            <Button
              onClick={() => onDismiss(item.objectID)}
              className="button-inline"
            >
              Dismiss
            </Button>
          </span>
        </div>
      )}
    </div>
  );
}
```

```
}

const Button = ({ onClick, className = '', children }) =>
  <button
    onClick={onClick}
    className={className}
    type="button"
  >
    {children}
  </button>

const Loading = () =>
  <div>Loading ...</div>

const withLoading = (Component) => ({ isLoading, ...rest }) =>
  isLoading ? <Loading /> : <Component { ...rest } />

const ButtonWithLoading = withLoading(Button);

const Sort = ({
  sortKey,
  activeSortKey,
  onSort,
  children
}) => {
  const sortClass = classNames(
    'button-inline',
    { 'button-active': sortKey === activeSortKey }
  );

  return (
    <Button
      onClick={() => onSort(sortKey)}
      className={sortClass}
    >
      {children}
    </Button>
  );
}

export default App;

export {
  Button,
  Search,
  Table,
};
```

You have learned advanced state management in React! Let's recap the last chapter:

React

- Lift state management up and down to suitable components
- setState() can use a function to prevent stale state bugs
- Existing external solutions that help you to tame the state

You can find the source code in the official repository.

Outline

Some final words and what you can learn after taking this course!

Outline

So now we've reached the end of The Road to Learn React. I hope you enjoyed it, and I hope it helped you to gain some traction in React. If you liked the course, tell your friends about it.

So, where can you go from here? I recommend you extend the application on your own, so eventually, you can start creating your very own React projects. Consider doing this before you dive into another course or tutorial. Do it for one week, take it to production by deploying it, and reach out to me me or others to showcase it. I am always interested in seeing what my readers built and helping them along.

If you are looking for more extensions for your application, I recommend several learning paths after you've mastered the basics:

- **Connecting to a Database and/or Authentication:** In a growing React application, you may want to persist data eventually. The data should be stored in a database so it can survive after a browser session, and so it can be shared across different users using your application. The simplest way to introduce a database is Firebase. In this tutorial, you will find a step-by-step guide on how to use Firebase authentication in React. Beyond that, you can use Firebase's real-time database to store user entities.

- **State Management:** You have used React `this.setState()` and `this.state` to manage and access local component state. That's a good start. In a larger application, however, you will experience the limits of React's local component state. It is imperative you learn to use third-party state management libraries like Redux or MobX. On the Road to React platform, you will find the course "Taming the State in React" that

platform, you will find the course "Taming the State in React" that teaches about advanced local state management using Redux and MobX.

The course comes with an ebook as well, but I recommend you dive into the source code and screencasts too.

- **Tooling with Webpack and Babel:** We used *create-react-app* to set up the application we created for this course. At some point, you may want to learn the tooling around it, which enables you to setup your own project without *create-react-app*. I recommend a minimal setup with Webpack and Babel, after which you can apply additional tooling on your own. For instance, you could use ESLint to follow a unified code style.

- **Code Organization:** Recall if you will the chapter about code organization. You can apply these changes now if you haven't already. It will help organize your components in structured files and folders (modules), and it will help you understand the principles of code splitting, reusability, maintainability, and module API design. Your applications will eventually grow and need to be structured into modules, so it's better to start now.

- **Testing:** The course only scratched the surface of testing. If you are not familiar with the topic, you should dive deeper into unit testing and integration testing, especially with React applications. I would recommend sticking to Enzyme and Jest for implementations, to refine your approaches with unit and snapshot tests.

- **React Component Syntax:** The best practices for implementing React components evolve over time. You will find many ways to write your React components, especially React class components, in other learning resources. A GitHub repository called react-alternative-class-component-syntax is a great way to learn alternative ways to write React class components. By using class field declarations, you can write them even more concisely in the future.

- **UI Components:** Many beginners make the mistake of introducing UI component libraries too early in their projects. It is more practical to learn how to implement and use a dropdown, checkbox, or dialog in React with standard HTML elements. Most of these components will manage their own local state. A checkbox has to know whether it is

checked or unchecked, so you should implement them as controlled components. After we covered the foundational implementations,

introducing a UI component library with checkboxes and dialogs as React components should be easier.

- **Routing:** You can implement routing for your application with react-router. So far, there is only one page in your application. React Router helps manage multiple pages across multiple URLs. When you introduce routing to your application, you don't make any requests to your web server to request the next page. The router will do everything for you on the client-side. So get your hands dirty and introduce routing in your application.

- **Type Checking:** Earlier, we used React PropTypes to define component interfaces. It is a good practice to prevent bugs, but the PropTypes are only checked on runtime. You can go further by introducing static type checking at compile time. TypeScript and Flow are popular type systems for React.

- **React Native:** React Native brings your application to mobile devices, such as iOS and Android applications. Once you've mastered React, the learning curve for React Native shouldn't be that steep, as they share the same principles. The only difference with mobile is the layout components.

- **Other Projects:** There are plenty of tutorials out there that use only React to build exciting applications, which provide good practice to apply what you've learned in this course before you move on to intermediate projects. Here are a few of my own:

    - A paginated and infinite scrolling list
    - Showcasing tweets on a Twitter wall
    - Connecting your React application to Stripe for charging money.

I invite you to visit my website to find more interesting topics about web development and software engineering. You can also subscribe to my Newsletter to get updates about articles and courses. Furthermore, the Road to React course platform offers more advanced lessons about the React ecosystem.

Finally, I hope to find more Patrons who are willing to support my content, as there are many students who cannot afford to pay for educational content. That's why I put lots of my content out there for free. Supporting me on Patreon allows me to educate others for free.

Thank you for taking the Road to learn React.

Regards,

Robin Wieruch